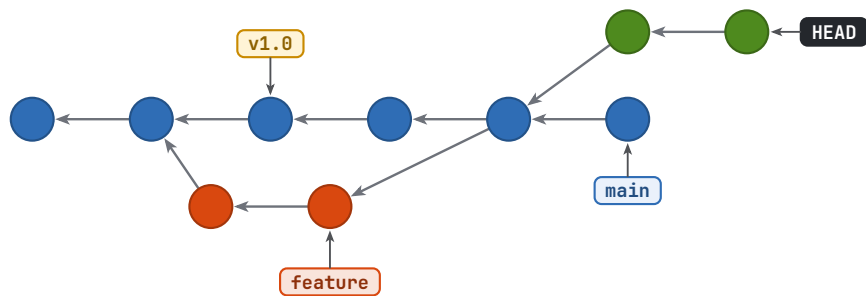


Git von Grund auf

Kommandozeile am Mac, sauberes Mergen, Worktrees, Pull Requests und CI/CD mit Gitea Actions



Inhalt

Bevor es losgeht	4	6 Korrekt mergen	22
Aufbau	4	6.1 Die Merge-Basis	22
Konventionen	4	6.2 Fast-Forward	22
		6.3 Der echte Merge (Drei-Wege-Merge)	22
		6.4 Die Strategien im Vergleich	23
		6.5 Konflikte lösen	24
TEIL I · GRUNDLAGEN		7 Rebase im Detail	26
1 Das mentale Modell	5	7.1 Was Rebase tut	26
1.1 Schnappschüsse statt Änderungen	5	7.2 Die goldene Regel	26
1.2 Die drei Bereiche	5	7.3 Interaktiver Rebase	26
1.3 Das Objektmodell	6	7.4 Der Fixup-Workflow	27
1.4 Der Commit-Graph	7	7.5 Branches umhängen: <code>--onto</code>	28
1.5 Branches sind nur Zeiger	7	7.6 Während eines Rebase	28
2 Git am Mac einrichten	8	8 Commits nachträglich ändern	29
2.1 Terminal-Grundlagen in fünf Minuten	8	8.1 Übersicht	29
2.2 Git installieren	8	8.2 Die letzte Commit-Nachricht ändern	29
2.3 Die Konfiguration	9	8.3 Vergessene Dateien nachreichen	29
2.4 Der Editor für Commit-Nachrichten	10	8.4 Die Nachricht eines älteren Commits ändern	30
2.5 SSH-Schlüssel einrichten	11	8.5 Einen Commit aufteilen	30
2.6 Globale Ignore-Datei	11	8.6 Wenn der Commit schon gepusht ist	30
2.7 Branch im Prompt anzeigen	12	8.7 Autor und E-Mail korrigieren	31
3 Der tägliche Arbeitsablauf	13	8.8 Große Umbauten: Dateien aus der gesamten Historie entfernen	31
3.1 Repository anlegen oder klonen	13	9 Rückgängig machen und Rettung	32
3.2 Den Zustand lesen: <code>git status</code>	13	9.1 Die Werkzeuge im Überblick	32
3.3 Änderungen vormerken	14	9.2 <code>restore</code> : Dateien zurücksetzen	32
3.4 Comitten	14	9.3 <code>reset</code> : den Branch verschieben	32
3.5 Diffs lesen	15	9.4 <code>revert</code> : sicher rückgängig nach dem Push	33
3.6 Die Historie ansehen	15	9.5 <code>reflog</code> : das Sicherheitsnetz	33
3.7 Dateien löschen, verschieben, ignorieren	15	9.6 <code>stash</code> : Änderungen beiseitelegen	34
3.8 Der Kreislauf im Überblick	16	9.7 <code>cherry-pick</code> : einzelne Commits übernehmen	34
4 Branches	17	9.8 <code>clean</code> : unversionierte Dateien löschen	36
4.1 Anlegen und wechseln	17	9.9 Notfall-Tabelle	36
4.2 Branches verwalten	17	10 Worktrees: mehrere Branches gleichzeitig	37
4.3 Wechseln mit ungesicherten Änderungen	18	10.1 Das Problem	37
4.4 Detached HEAD	18	10.2 Das Konzept	37
4.5 Namenskonventionen	18	10.3 Die Befehle	38
		10.4 Beispiel: Hotfix mitten im Feature	38
		10.5 Beispiel: Pull Request lokal prüfen	38
		10.6 Beispiel: parallele Arbeitsstränge mit KI-Agenten	38
		10.7 Stolpersteine	39
TEIL II · ZUSAMMENARBEIT UND HISTORIE			
5 Remotes: Arbeiten mit dem Server	19		
5.1 Remotes verwalten	19		
5.2 Remote-Tracking-Branches	19		
5.3 fetch, pull und push	19		
5.4 Wenn der Push abgelehnt wird	20		
5.5 <code>git pull</code> richtig konfigurieren	20		
5.6 Aufräumen und Remote-Branches löschen	21		

10.8	Eine sinnvolle Ordnerstruktur	39
11	Tags, Referenzen und Werkzeuge	40
11.1	Tags	40
11.2	Commits benennen	40
11.3	Bereiche: zwei und drei Punkte	41
11.4	Suchen in der Historie	41
11.5	bisect: den schuldigen Commit finden	42
11.6	Kleine Helfer	42

TEIL III · GITEA, PULL REQUESTS UND CI/CD

12	Gitea und Pull Requests	43
12.1	Ein lokales Projekt mit Gitea verbinden	43
12.2	Der Branch-basierte Arbeitsablauf	43
12.3	Einen Pull Request erstellen	43
12.4	Review	44
12.5	Einen Pull Request aktualisieren	44
12.6	Mergen: die Gitea-Merge-Stile	45
12.7	Branch-Schutz	45
12.8	Pull Requests lokal auschecken	46
13	Gitea Actions: die ci.yml verstehen	47
13.1	Wie Gitea Actions funktioniert	47
13.2	YAML in zehn Minuten	48
13.3	Der erste Workflow	48
13.4	Die Anatomie: Workflow, Jobs, Steps	49
13.5	Auslöser: on	49
13.6	Jobs im Detail	50
13.7	Steps im Detail	50
13.8	Ausdrücke und Kontexte	51
13.9	Daten zwischen Jobs: Artefakte und Caches	51
13.10	Unterschiede zu GitHub Actions	52
13.11	Vertrauensgrenzen von Workflows	52
13.12	Das Beispielprojekt	52
14	Runner, Secrets, Deployment und SonarQube	54
14.1	Der Runner	54
14.2	Secrets und Variablen	55
14.3	Deployment per SSH	56
14.4	Container-Images in der Gitea-Registry	58
14.5	SonarQube: statische Codeanalyse	59
15	Die komplette Pipeline	61
15.1	ci.yml: jede Änderung prüfen	61
15.2	deploy.yml: ausliefern	62
15.3	Ein Feature von Anfang bis Ende	64
15.4	Fehlersuche	65
16	Dependency-Bots mit Renovate	66
16.1	Der sichere Ablauf	66

16.2	Ein Bot-Konto statt eines persönlichen Kontos	66
16.3	Zentraler Lauf auf einem Runner	67
16.4	Die globale Konfiguration	68
16.5	Konfiguration pro Repository	68
16.6	Betrieb und Fehlersuche	69
17	CI-Gates, Secrets und Repository-Struktur	70
17.1	Vier Stufen statt eines einzigen großen Jobs	70
17.2	Welche Checks wirklich sinnvoll sind	70
17.3	Referenzworkflow für Pull Requests	72
17.4	Build und Deployment: nur das geprüfte Artefakt	72
17.5	Secrets, Variablen und Job-Tokens	73
17.6	Deploy Keys richtig einordnen	73
17.7	Einzelne Repositories oder Organisation?	74
17.8	Eine pragmatische Zielarchitektur	74

ANHANG

A	Befehlsübersicht	75
B	Glossar und Quellen	76
	Weiterführende Quellen	77
	Literaturverzeichnis	78

Bevor es losgeht

Dieses Handbuch ist für jemanden geschrieben, der Git seit Jahren benutzt, aber vor allem über eine Handvoll auswendig gelernter Befehle: `add`, `commit`, `push`, `pull`, gelegentlich ein `Branch`. Das funktioniert, bis etwas Unerwartetes passiert: ein Merge-Konflikt, ein abgelehnter Push, eine falsche Commit-Nachricht, die schon auf dem Server liegt, oder eine CI-Pipeline, die rot wird, ohne dass klar ist, warum. Genau an diesen Stellen hilft kein weiterer Befehl, sondern nur ein brauchbares Bild davon, was Git intern tut.

Deshalb beginnt das Buch mit dem mentalen Modell (Kapitel 1). Alles Weitere, vom Mergen über Rebase bis zu Worktrees, ist danach eine Anwendung dieses Modells und lässt sich ohne Auswendiglernen herleiten.

Aufbau

- **Teil I, Grundlagen:** das Modell hinter Git, die Einrichtung am Mac (Terminal, Homebrew, SSH-Schlüssel, Konfiguration), der tägliche Arbeitsablauf und Branches.
- **Teil II, Zusammenarbeit und Historie:** Remotes, korrektes Mergen mit allen Strategien und Konfliktlösung, Rebase, das nachträgliche Ändern von Commits und Commit-Nachrichten, Rückgängig machen und Rettung, Worktrees sowie Werkzeuge wie `bisect` und `blame`.
- **Teil III, Gitea und CI/CD:** Pull Requests in Gitea, Aufbau einer `ci.yml`, Runner-Betrieb, Secrets, Deployment per SSH und SonarQube, zum Schluss eine komplette Pipeline.
- **Anhang:** Befehlsübersicht zum Nachschlagen und ein Glossar.

Konventionen

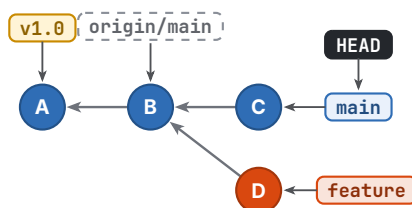
Befehle zum Eintippen stehen in grauen Kästen:

```
git status
```

Ausgaben von Git sind gestrichelt umrandet:

```
On branch main
nothing to commit, working tree clean
```

In Commit-Graphen ist jeder Kreis ein Commit. **Die Pfeile zeigen immer vom neueren Commit auf seinen Vorgänger** (Elterncommit), weil Git es intern genau so speichert: Ein Commit kennt seine Eltern, aber nicht seine Nachfolger [1]. Die Zeit läuft also von links nach rechts, die Pfeile zeigen nach links.



Beschriftete Kästchen sind Referenzen: `main` ist ein lokaler Branch, `origin/main` ein Remote-Tracking-Branch, `v1.0` ein Tag und `HEAD` zeigt, wo du gerade stehst. Gestrichelte Kreise sind Commits, die nach einer Operation nicht mehr erreichbar sind (aber noch existieren, dazu mehr in Kapitel 9).

Als durchgängiges Beispiel dient ein neutrales Projekt namens `demo`. Die Programmiersprache spielt keine Rolle: Alle Prüf- und Build-Schritte laufen über ein `Makefile` mit den Zielen `make lint`, `make test` und `make build`. So lässt sich jede Pipeline aus diesem Buch auf Python, TypeScript, Swift oder Typst übertragen, indem man nur das `Makefile` anpasst. Server heißen `gitea.example.com` (Gitea, SSH auf Port 2222), `sonar.example.com` und `app.example.com` (Zielserver für das Deployment).

Jedes Kapitel lässt sich für sich nachschlagen. Wer von vorne liest, sollte die Befehle in einem Wegwerf-Repository mittippen (`mkdir /tmp/spielwiese && cd /tmp/spielwiese && git init`) und nach jedem Schritt `git log --oneline --graph --all` ansehen.

1 Das mentale Modell

Git wirkt kompliziert, weil es über 150 Befehle mit unzähligen Optionen hat [2]. Der Kern ist aber erstaunlich klein: eine Datenbank aus unveränderlichen Schnappschüssen, die zu einem Graphen verkettet sind, plus ein paar verschiebbare Namensschilder, die auf Punkte in diesem Graphen zeigen. Wer diese drei Dinge verstanden hat (Schnappschüsse, Graph, Namensschilder), kann fast jeden Befehl selbst herleiten.

1.1 Schnappschüsse statt Änderungen

Viele ältere Versionsverwaltungen speichern pro Version die Änderungen gegenüber der Vorversion. Git speichert dagegen bei jedem Commit **den vollständigen Zustand aller Dateien** des Projekts [3]. Das klingt verschwenderisch, ist es aber nicht: Unveränderte Dateien werden nicht neu abgelegt, sondern der neue Schnappschuss verweist auf die bereits vorhandene Version [3]. Eine Änderungsansicht (ein *Diff*) berechnet Git erst dann, wenn du sie anforderst, indem es zwei Schnappschüsse vergleicht.

Daraus folgen zwei Eigenschaften, die später immer wieder wichtig werden:

1. **Commits sind unveränderlich.** Ein Commit wird über eine Prüfsumme seines Inhalts identifiziert [1]. Ändert man auch nur ein Zeichen der Nachricht, entsteht ein neuer Commit mit neuer Prüfsumme. „Einen Commit ändern“ heißt in Git immer: einen neuen, korrigierten Commit erzeugen und den alten links liegen lassen.
2. **Nichts geht leicht verloren.** Was einmal committet wurde, bleibt in der Objektdatenbank, auch wenn kein Branch mehr darauf zeigt. Erst nach Wochen räumt Git solche verwaisten Objekte automatisch weg [4]. Das ist die Grundlage aller Rettungsaktionen in Kapitel 9.

1.2 Die drei Bereiche

Wenn du an einem Git-Projekt arbeitest, gibt es drei Orte, an denen eine Datei in unterschiedlichen Versionen existieren kann. Diese Unterscheidung ist der wichtigste einzelne Schlüssel zum Verständnis von Befehlen wie `add`, `restore` oder `reset`.

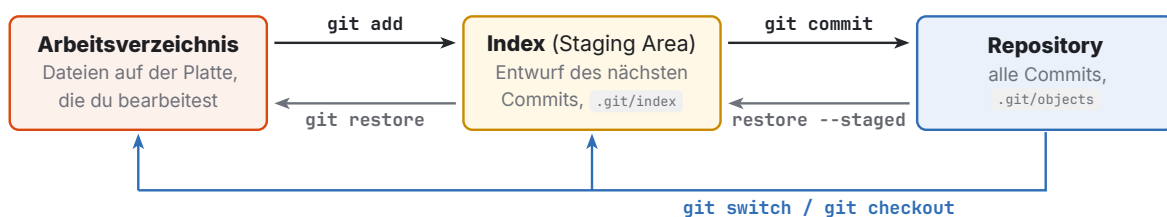


Abb. 1: Die drei Bereiche und die Befehle, die Inhalte zwischen ihnen bewegen.

- Das **Arbeitsverzeichnis** (*working tree*) ist der normale Projektordner mit den Dateien, die du im Editor öffnest [5].
- Der **Index** (*staging area*) ist ein Zwischenbereich. Er enthält den Stand, der beim nächsten `git commit` festgeschrieben wird [3]. Mit `git add` kopierst du den aktuellen Inhalt einer Datei in den Index. Änderst du die Datei danach noch einmal, liegt im Index weiterhin die ältere Fassung, bis du erneut `git add` ausführst [6].
- Das **Repository** ist der versteckte Ordner `.git` im Projektverzeichnis. Dort liegen alle Commits, Branches und Einstellungen [3]. Löscht du `.git`, ist das Projekt kein Git-Projekt mehr, die Dateien im Arbeitsverzeichnis bleiben aber erhalten.

Der Index erscheint anfangs überflüssig, ist aber eines der nützlichsten Werkzeuge: Er erlaubt, aus einem Durcheinander von Änderungen gezielt nur einen Teil zu committen, etwa den Bugfix, aber nicht die Debug-Ausgaben daneben (siehe `git add -p` in Kapitel 3).

MERKE

`git status` zeigt dir jederzeit die Unterschiede zwischen diesen drei Bereichen: *Changes to be committed* sind Unterschiede zwischen Index und letztem Commit, *Changes not staged for commit* sind Unterschiede zwischen Arbeitsverzeichnis und Index, *Untracked files* sind Dateien, die Git noch nie im Index hatte [7].

1.3 Das Objektmodell

Im Ordner `.git/objects` speichert Git vier Arten von Objekten. Jedes wird über den SHA-1-Hash seines Inhalts adressiert, eine 40-stellige Hexadezimalzahl wie `3f2a9c1e...` [1], [3]. Meist reichen die ersten sieben Zeichen, um ein Objekt eindeutig anzusprechen [8].

Objekt	Inhalt
blob	Der Inhalt einer Datei, ohne Namen und ohne Rechte. Zwei Dateien mit identischem Inhalt teilen sich einen Blob [1].
tree	Ein Verzeichnis: eine Liste von Namen, Dateirechten und Verweisen auf Blobs (Dateien) oder weitere Trees (Unterordner) [1].
commit	Ein Verweis auf den Wurzel-Tree des Projekts, dazu die Elterncommits, Autor, Committer, Zeitstempel und die Nachricht [1].
tag	Ein annotierter Tag: Verweis auf einen Commit plus Name, Autor, Datum und Nachricht (siehe Kapitel 11) [5].

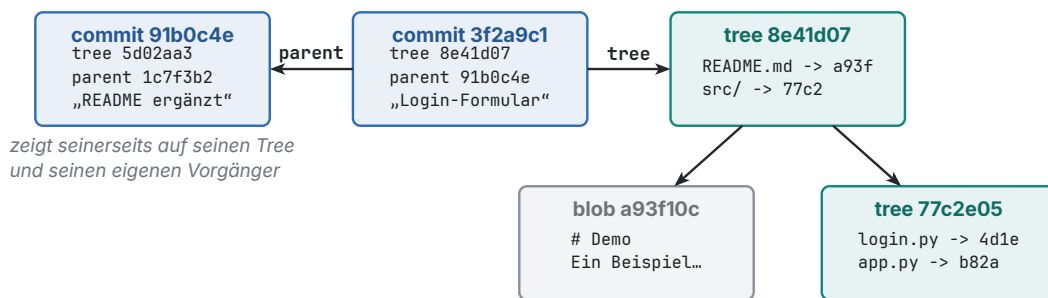


Abb. 2: Ein Commit zeigt auf einen Tree (den Schnappschuss) und auf seinen Elterncommit.

Du kannst dir jedes Objekt ansehen. Das ist nie nötig für die tägliche Arbeit, aber ein guter Weg, das Modell einmal selbst zu sehen:

```
git cat-file -p HEAD          # den aktuellen Commit anzeigen
git cat-file -p HEAD^{tree}   # den zugehörigen Tree anzeigen
```

```
tree 8e41d07b2c...
parent 91b0c4e5a1...
author Philipp Haindl <philipp@example.com> 1790251200 +0200
committer Philipp Haindl <philipp@example.com> 1790251200 +0200

Login-Formular
```

Weil der Hash eines Commits über Tree, Eltern, Autor, Zeit und Nachricht berechnet wird, hängt er indirekt von der gesamten Vorgeschichte ab [1]. Ändert sich ein früherer Commit, ändern sich zwangsläufig die Hashes aller späteren. Genau deshalb bekommen beim Rebase oder beim Ändern einer alten Commit-Nachricht alle nachfolgenden Commits neue Hashes [9].

TIPP

Git 3.0, für das noch kein Veröffentlichungstermin feststeht, soll neue Repositories standardmäßig auf SHA-256 umstellen (64 statt 40 Hex-Zeichen) und den Standard-Branch `main` statt `master` nennen [10]. Bestehende Repositories funktionieren unverändert weiter. Für das Verständnis ändert sich nichts.

1.4 Der Commit-Graph

Jeder Commit (außer dem allerersten) verweist auf mindestens einen Elterncommit. Ein gewöhnlicher Commit hat genau ein Elternteil, ein Merge-Commit hat zwei (selten mehr) [11]. So entsteht ein gerichteter, azyklischer Graph (DAG) [5]: Man kann immer nur rückwärts in die Vergangenheit laufen, nie im Kreis.

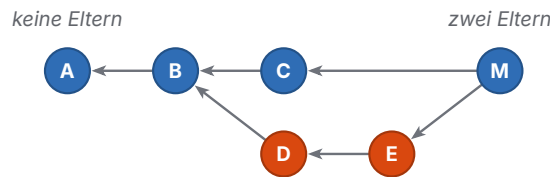


Abb. 3: Ein Graph mit Verzweigung. M ist ein Merge-Commit mit den Eltern C und E.

Ein wichtiger Begriff ist die **Erreichbarkeit**: Ein Commit gehört „zu einem Branch“, wenn man ihn vom Branch-Ende aus entlang der Pfeile erreichen kann [5]. In der Abbildung sind von M aus alle Commits erreichbar. Diese Idee steckt hinter Fragen wie „Ist mein Feature schon in main?“ (`git branch --merged` [12]) oder „Welche Commits hat feature, die main noch nicht hat?“ (`git log main..feature`, Kapitel 11 [13]).

1.5 Branches sind nur Zeiger

Ein Branch ist in Git kein Ordner und keine Kopie, sondern **eine Datei, die einen einzigen Commit-Hash enthält**. Der Branch `main` ist die Datei `.git/refs/heads/main` mit 41 Bytes Inhalt [11], [14]. Einen Branch anzulegen kostet deshalb praktisch nichts, und es ist völlig normal, Dutzende davon zu haben.

Woher weiß Git, auf welchem Branch du gerade arbeitest? Dafür gibt es `HEAD`, eine weitere kleine Datei (`.git/HEAD`), die normalerweise nicht auf einen Commit, sondern auf einen Branch verweist [14]:

```
cat .git/HEAD
cat .git/refs/heads/main
```

```
ref: refs/heads/main
3f2a9c1e0b8d4f6a7c2e9b1d0a5f8e3c7b6d4a21
```

Beim Committen passiert Folgendes: Git erzeugt aus dem Index einen neuen Commit, dessen Elternteil der bisherige Commit von `HEAD` ist, und schiebt dann den Branch, auf den `HEAD` zeigt, auf den neuen Commit weiter [11], [14]. `HEAD` selbst bewegt sich dabei nicht, denn er zeigt ja auf den Branch, und der Branch wandert mit.



Abb. 4: Ein Commit schiebt nur den aktuellen Branch weiter. feature bleibt stehen.

MERKE

Das komplette Modell in fünf Sätzen: **(1)** Ein Commit ist ein unveränderlicher Schnappschuss mit Verweis auf seine Eltern. **(2)** Commits bilden einen Graphen, in dem Pfeile in die Vergangenheit zeigen. **(3)** Ein Branch ist ein verschiebbarer Zeiger auf einen Commit. **(4)** `HEAD` zeigt auf den aktuellen Branch. **(5)** Zwischen deinen Dateien und dem Repository liegt der Index als Entwurf des nächsten Commits.

2 Git am Mac einrichten

Bevor es um Git selbst geht, lohnt sich eine halbe Stunde für eine saubere Grundausstattung: ein aktuelles Git über Homebrew, eine durchdachte globale Konfiguration, SSH-Schlüssel mit Schlüsselbund-Anbindung und eine Shell, die den aktuellen Branch anzeigt. Das spart später viele kleine Irritationen.

2.1 Terminal-Grundlagen in fünf Minuten

Das Terminal (Programme -> Dienstprogramme -> Terminal, oder schneller über `⌘` `Leertaste` und „Terminal“) startet seit macOS Catalina standardmäßig die Shell **zsh** [15]. Alternativen wie iTerm2 oder Ghostty funktionieren genauso. Die wichtigsten Befehle zum Navigieren:

Befehl	Bedeutung
<code>pwd</code>	Zeigt das aktuelle Verzeichnis (<i>print working directory</i>).
<code>ls -la</code>	Listet alle Dateien, auch versteckte wie <code>.git</code> oder <code>.gitignore</code> , mit Details.
<code>cd projekte/demo</code>	Wechselt in ein Verzeichnis. <code>cd ..</code> geht eine Ebene hoch, <code>cd -</code> zurück zum vorherigen, <code>cd</code> allein ins Home-Verzeichnis <code>~</code> .
<code>mkdir -p a/b/c</code>	Legt Verzeichnisse an, inklusive fehlender Zwischenebenen.
<code>open .</code>	Öffnet das aktuelle Verzeichnis im Finder. <code>open datei.pdf</code> öffnet mit dem Standardprogramm.
<code>cat</code> , <code>less</code>	Datei ausgeben bzw. seitenweise anzeigen.
<code>code .</code>	Öffnet das Verzeichnis in VS Code (wenn der Befehl installiert ist, siehe unten) [16].

Zwei Dinge sparen im Alltag am meisten Zeit: Die `Tab`-Taste vervollständigt Pfade, Befehle und bei Git sogar Branch-Namen [17]. Mit `↑` holst du frühere Befehle zurück, mit `ctrl R` durchsuchst du die gesamte Befehlshistorie.

Der Pager: Warum `git log` „hängen bleibt“

Längere Ausgaben wie `git log` oder `git diff` zeigt Git im Programm `less` an [18]. Das wirkt beim ersten Mal wie ein Hänger, ist aber ein Betrachter mit eigenen Tasten [19]:

Taste	Wirkung	Taste	Wirkung
<code>Leertaste</code>	eine Seite weiter	<code>b</code>	eine Seite zurück
<code>/</code> Text	vorwärts suchen	<code>n</code>	nächster Treffer
<code>g</code> / <code>G</code>	Anfang / Ende	<code>q</code>	beenden

Hilfe gibt es auf zwei Stufen: `git log -h` zeigt eine kurze Optionsübersicht im Terminal [20], `git help log` öffnet die vollständige Handbuchseite [21].

2.2 Git installieren

macOS bringt kein vollwertiges Git mit. Beim ersten Aufruf von `git` bietet das System die Installation der *Xcode Command Line Tools* an [22], die ein von Apple gepflegtes Git enthalten [23]. Das funktioniert, hinkt aber der offiziellen Version meist einige Releases hinterher [23]. Empfehlenswert ist Git über Homebrew, den Paketmanager für macOS [24]:

```
# 1. Homebrew installieren (einmalig; Befehl von brew.sh)
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# 2. Homebrew in die Shell einbinden (Apple Silicon; der Installer zeigt den Befehl an)
echo 'eval "$$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
```



```
eval "$(/opt/homebrew/bin/brew shellenv)"

# 3. Git installieren und prüfen
brew install git
which -a git
git --version
```

```
/opt/homebrew/bin/git
/usr/bin/git
git version 2.55.0
```

`which -a` zeigt alle gefundenen Git-Programme in der Reihenfolge, in der die Shell sie sucht. Steht `/opt/homebrew/bin/git` oben, wird das Homebrew-Git verwendet. Aktualisieren lässt es sich jederzeit mit `brew upgrade git`.

2.3 Die Konfiguration

Git liest Einstellungen aus drei Ebenen, wobei die spezifischere gewinnt [18]:

Ebene	Datei	Gilt für
--system	/opt/homebrew/etc/gitconfig	alle Benutzer des Rechners (selten gebraucht)
--global	~/.gitconfig	alle Repositories deines Benutzers
--local	.git/config im Projekt	nur dieses eine Repository (Standard, wenn nichts angegeben ist)

Mit `git config --list --show-origin` siehst du alle aktiven Werte und aus welcher Datei sie stammen [18]. Das ist die erste Anlaufstelle, wenn sich Git „komisch“ verhält. Die folgende Datei ist eine erprobte Grundkonfiguration, jede Zeile ist in der Tabelle darunter erklärt. Du kannst sie direkt als `~/.gitconfig` speichern und Name und E-Mail anpassen.

`~/.gitconfig`

```
[user]
  name = Philipp Haindl
  email = philipp@example.com
[init]
  defaultBranch = main
[core]
  editor = nano
  excludesFile = ~/.config/git/ignore
[pull]
  rebase = true
[push]
  autoSetupRemote = true
[fetch]
  prune = true
[rebase]
  autoStash = true
  autoSquash = true
  updateRefs = true
[merge]
  conflictStyle = zdiff3
[rerere]
  enabled = true
[diff]
  algorithm = histogram
  colorMoved = default
[commit]
  verbose = true
[branch]
  sort = -committerdate
[tag]
  sort = version:refname
```

```
[help]
  autocorrect = prompt
[alias]
  st = status -sb
  lg = log --graph --oneline --decorate
  lga = log --graph --oneline --decorate --all
  last = log -1 --stat
  unstage = restore --staged
  amend = commit --amend --no-edit
```

Einstellung	Wirkung
<code>init.defaultBranch</code>	Neue Repositories beginnen mit <code>main</code> statt <code>master</code> [25].
<code>pull.rebase = true</code>	<code>git pull</code> setzt eigene, noch nicht gepushte Commits auf den neuen Remote-Stand, statt einen Merge-Commit zu erzeugen [26]. Hält die Historie linear (Kapitel 5).
<code>push.autoSetupRemote</code>	Beim ersten <code>git push</code> eines neuen Branches wird der Upstream automatisch gesetzt. Kein <code>-u origin feature</code> mehr nötig [27].
<code>fetch.prune</code>	Entfernt lokale <code>origin/...</code> -Branches, die auf dem Server gelöscht wurden, etwa nach einem gemergten Pull Request [28].
<code>rebase.autoStash</code>	Uncommittete Änderungen werden vor einem Rebase automatisch beiseitegelegt und danach wiederhergestellt [29].
<code>rebase.autoSquash</code>	<code>fixup!</code> -Commits werden beim interaktiven Rebase automatisch an die richtige Stelle sortiert (Kapitel 7) [29].
<code>rebase.updateRefs</code>	Beim Rebase aufeinander aufbauender Branches werden die Zwischen-Banches mitverschoben [29].
<code>merge.conflictStyle = zdiff3</code>	Konfliktmarkierungen zeigen zusätzlich den gemeinsamen Ursprung [30]. Das macht Konflikte deutlich leichter lesbar (Kapitel 6).
<code>rerere.enabled</code>	Git merkt sich, wie du einen Konflikt gelöst hast, und wendet die Lösung beim nächsten identischen Konflikt selbst an [31].
<code>diff.algorithm = histogram</code>	Liefert bei verschobenen Codeblöcken meist verständlichere Diffs als der Standardalgorithmus [32].
<code>commit.verbose</code>	Zeigt beim Schreiben der Commit-Nachricht im Editor den Diff darunter an [18].
<code>branch.sort</code>	<code>git branch</code> listet die zuletzt benutzten Branches zuerst [12].
<code>help.autocorrect = prompt</code>	Bei Tippfehlern wie <code>git stauts</code> fragt Git, ob <code>status</code> gemeint war [18].

Unterschiedliche Identitäten pro Ordner

Wer beruflich und privat mit verschiedenen E-Mail-Adressen committet, kann die Identität abhängig vom Ordner setzen. Alle Repositories unter `~/work/` bekommen dann automatisch die Arbeitsadresse [18]:

```
# in ~/.gitconfig ergänzen
[includeIf "gitdir:~/work/"]
  path = ~/.gitconfig-work
```

~/gitconfig-work

```
[user]
  email = philipp.haendl@firma.example
```

Prüfen lässt sich das im jeweiligen Repository mit `git config user.email`.

2.4 Der Editor für Commit-Nachrichten

Ohne Konfiguration öffnet Git für Commit-Nachrichten und interaktive Rebases den Editor `vi` (am Mac ist das Vim) [33]. Wer dort unfreiwillig landet: `i` schaltet in den Schreibmodus, `Esc` zurück, dann `:wq` und `↵` speichert und beendet, `:q!` beendet ohne Speichern (bei einer Commit-Nachricht bricht das den Commit ab). Angenehmer sind:

```
git config --global core.editor nano          # einfacher Terminal-Editor
git config --global core.editor "code --wait" # VS Code
```

Für VS Code muss einmalig der Shell-Befehl `code` installiert werden: In VS Code `⌘` `⇧` `P` drücken und *Shell Command: Install 'code' command in PATH* wählen [16]. Das `--wait` ist wichtig, damit Git wartet, bis du den Tab schließt [34]. Bei `nano` speichert `ctrl` `O` und `ctrl` `X` beendet [35].

2.5 SSH-Schlüssel einrichten

Für die Verbindung zu Gitea gibt es zwei Wege: HTTPS mit Benutzername und Token oder SSH mit einem Schlüsselpaar. SSH ist bequemer und sicherer, sobald es einmal eingerichtet ist. Der private Schlüssel verlässt deinen Mac nie, auf den Server kommt nur der öffentliche Teil.

```
# Schlüsselpaar erzeugen (Ed25519 ist der aktuelle Standard)
ssh-keygen -t ed25519 -C "philipp@macbook"
# Speicherort bestätigen (~/.ssh/id_ed25519) und eine Passphrase vergeben
```

Damit du die Passphrase nicht bei jedem Push eintippen musst, speichert macOS sie im Schlüsselbund [36], [37]. Dazu gehört ein Eintrag in der SSH-Konfiguration, in dem du auch gleich einen Kurznamen für den Gitea-Server anlegen kannst [38]:

`~/.ssh/config`

```
Host *
  AddKeysToAgent yes
  UseKeychain yes
  IdentityFile ~/.ssh/id_ed25519

# Kurzname für den eigenen Gitea-Server mit SSH auf Port 2222
Host gitea
  HostName gitea.example.com
  Port 2222
  User git
```

```
ssh-add --apple-use-keychain ~/.ssh/id_ed25519 # Passphrase einmal in den Schlüsselbund
pbcopy < ~/.ssh/id_ed25519.pub                # öffentlichen Schlüssel in die Zwischenablage
```

In Gitea fügst du den Schlüssel unter *Einstellungen* -> *SSH/GPG-Schlüssel* -> *Schlüssel hinzufügen* ein. Danach testest du die Verbindung [39]:

```
ssh -T gitea
```

```
Hi there, philipp! You've successfully authenticated with the key named MacBook,
but Gitea does not provide shell access.
```

Durch den Kurznamen funktionieren nun Remote-Adressen wie `gitea:team/demo.git`. Ohne Eintrag in `~/.ssh/config` müsste die volle Form mit Port geschrieben werden: `ssh://git@gitea.example.com:2222/team/demo.git`.

ACHTUNG

Den privaten Schlüssel (`id_ed25519` ohne `.pub`) niemals kopieren, hochladen oder in ein Repository legen. Für CI-Deployments erzeugst du später eigene, eingeschränkte Schlüssel (Kapitel 14), statt deinen persönlichen Schlüssel wiederzuverwenden.

2.6 Globale Ignore-Datei

Manche Dateien entstehen nicht durch das Projekt, sondern durch deinen Rechner, allen voran `.DS_Store`, das der Finder in jedem geöffneten Ordner anlegt. Solche Einträge gehören nicht in die `.gitignore` jedes Projekts, sondern in eine globale Datei:

```
~/ .config/git/ignore
```

```
.DS_Store
.AppleDouble
.*
*.swp
*~
```

Diese Datei liest Git automatisch [40]. Der Eintrag `core.excludesFile` in der Konfiguration oben macht es nur explizit.

2.7 Branch im Prompt anzeigen

Sehr hilfreich ist es, den aktuellen Branch direkt in der Eingabezeile zu sehen. zsh bringt dafür alles mit [17]. In die Datei `~/ .zshrc` eintragen und ein neues Terminalfenster öffnen:

```
~/ .zshrc
```

```
# Tab-Vervollständigung (inklusive Git-Branches)
autoload -Uz compinit && compinit

# Git-Branch im Prompt
autoload -Uz vcs_info
precmd() { vcs_info }
zstyle ':vcs_info:git:*' formats ' (%b)'
zstyle ':vcs_info:git:*' actionformats ' (%b|a)'
setopt PROMPT_SUBST
PROMPT='%F{cyan}%~%F{yellow}${vcs_info_msg_0_}%f %# '
```

Das Ergebnis sieht so aus: `~/projekte/demo (feature/login) %`. Das `actionformats` zeigt zusätzlich an, wenn gerade ein Rebase oder Merge läuft, etwa `(main|rebase-i)` [41]. Wer mehr möchte (Anzeige von uncommitteten Änderungen, Ahead/Behind-Zähler), findet mit *Starship* (`brew install starship`) einen fertigen, schnellen Prompt [42].

SPEZIELL AM MAC

Groß-/Kleinschreibung: Das Mac-Dateisystem APFS unterscheidet standardmäßig nicht zwischen `readme.md` und `README.md` [43], Linux-Server und Git selbst aber schon [18]. Eine reine Umbenennung der Schreibweise ist deshalb heikel. Zuverlässig funktioniert sie in zwei Schritten:

```
git mv readme.md tmp.md && git mv tmp.md README.md.
```

Versteckte Dateien: Im Finder blendet   versteckte Dateien wie `.git` und `.gitignore` ein und aus.

Umlaute in Dateinamen: macOS speichert Umlaute intern zerlegt (a + Trema), Linux zusammengesetzt. Git gleicht das über `core.precomposeUnicode` aus [18], das am Mac automatisch aktiv ist. Trotzdem gilt: Dateinamen in Repositories am besten ohne Umlaute und Leerzeichen.

3 Der tägliche Arbeitsablauf

Dieses Kapitel deckt ab, was du jeden Tag brauchst: ein Repository anlegen oder klonen, Änderungen lesen, gezielt vormerken, sauber committen und die Historie durchsuchen.

3.1 Repository anlegen oder klonen

Ein neues Projekt machst du mit `git init` zu einem Repository. Das legt nur den Ordner `.git` an, deine Dateien bleiben unberührt [25]. Ein bestehendes Projekt vom Server holst du mit `git clone`:

```
# Neues Projekt
mkdir demo && cd demo
git init

# Bestehendes Projekt klonen (legt den Ordner demo an)
git clone gitea:team/demo.git
git clone gitea:team/demo.git anderer-ordnername
```

`git clone` erledigt vier Dinge auf einmal: Es legt das Verzeichnis und `.git` an, lädt alle Commits herunter, registriert die Quelle unter dem Namen `origin` und checkt den Standard-Branch des Servers (meist `main`) aus [44], [45].

3.2 Den Zustand lesen: `git status`

`git status` ist der Befehl, den du am häufigsten ausführen solltest, am besten vor und nach jedem anderen Befehl. Die Kurzform ist kompakter und für den Alltag meist besser [7]:

```
git status -sb
```

```
## main...origin/main [ahead 1]
M  src/login.py
M  README.md
MM src/app.py
A  src/token.py
?? notizen.txt
```

Die erste Zeile zeigt den Branch, seinen Upstream und ob du Commits voraus (*ahead*) oder hinterher (*behind*) bist. Darunter stehen zwei Statusspalten: **links der Index, rechts das Arbeitsverzeichnis** [7].

Code	Bedeutung
M (links)	Geändert und vorgemerkt: kommt so in den nächsten Commit.
M (rechts)	Geändert, aber noch nicht vorgemerkt.
MM	Vorgemerkt und danach erneut geändert. Im Index liegt die ältere Fassung!
A	Neue Datei, vorgemerkt.
D , D	Gelöscht (vorgemerkt bzw. nur im Arbeitsverzeichnis).
R	Umbenannt (Git erkennt Umbenennungen über ähnlichen Inhalt [46]).
??	Unbekannte Datei, die Git noch nie im Index hatte (<i>untracked</i>).
UU	Konflikt, beide Seiten haben die Datei geändert (Kapitel 6) [7].

3.3 Änderungen vormerken

```
git add src/login.py      # eine Datei
git add src/              # alles in einem Ordner
git add .                 # alles im aktuellen Verzeichnis und darunter
git add -p                # stückweise, interaktiv (sehr empfehlenswert)
git restore --staged datei # Vormerkung zurücknehmen, Änderung bleibt erhalten
```

`git add -p` (*patch*) zeigt jede Änderung als einzelnen Abschnitt (*hunk*) und fragt, ob er in den Index soll [6]. So baust du saubere Commits auch dann, wenn du in einer Datei mehrere unabhängige Dinge geändert hast [47]:

Taste	Wirkung	Taste	Wirkung
<code>y</code>	Abschnitt vormerken	<code>n</code>	Abschnitt überspringen
<code>s</code>	in kleinere Abschnitte teilen	<code>e</code>	Abschnitt im Editor zuschneiden
<code>q</code>	beenden	<code>?</code>	Hilfe zu allen Optionen

TIPP

Eine gute Routine vor jedem Commit: `git add -p`, dann `git diff --staged` zur Kontrolle dessen, was wirklich committet wird, dann `git commit`. Das verhindert fast alle „Ups, das sollte nicht mit rein“-Momente.

3.4 Committen

```
git commit                # öffnet den Editor für die Nachricht
git commit -m "Kurzer Titel"
git commit -am "Titel"    # add + commit für bereits bekannte Dateien (nicht für neue!)
```

Gute Commits

Ein guter Commit enthält **eine logische Änderung**: einen Bugfix, eine Funktion, eine Umbenennung [48]. Er sollte für sich verständlich sein und das Projekt in einem funktionierenden Zustand hinterlassen. Das zählt sich beim Review, bei `git revert` und bei `git bisect` aus. Faustregel: Wenn du im Titel „und“ schreiben musst, sind es vermutlich zwei Commits.

Für die Nachricht hat sich eine feste Form durchgesetzt:

Commit-Nachricht

Login-Sperre nach fünf Fehlversuchen einführen

Bisher konnten Passwörter beliebig oft probiert werden. Nach fünf Fehlversuchen innerhalb von zehn Minuten wird das Konto jetzt für 15 Minuten gesperrt. Der Zähler liegt in Redis, damit die Sperre über mehrere Instanzen hinweg gilt.

Refs: #42

- **Titelzeile:** höchstens etwa 50 Zeichen [49], ohne Punkt am Ende, im Imperativ („einführen“, nicht „eingeführt“; englisch „Add“, nicht „Added“) [48]. Sie erscheint in `git log --oneline` und in Gitea-Listen [49].
- **Leerzeile**, dann der **Textkörper** mit Zeilen bis etwa 72 Zeichen [48]. Er erklärt **warum** etwas geändert wurde [48]. Das **Was** steht ohnehin im Diff.
- **Trailer** am Ende wie `Refs: #42` oder `Co-authored-by: Name <mail>` verknüpfen den Commit mit Issues oder Mitautoren [50].

Viele Teams nutzen zusätzlich *Conventional Commits*, bei denen der Titel mit einem Typ beginnt: `feat: Login-Sperre einführen`, `fix:`, `docs:`, `refactor:`, `test:`, `chore:`. Das erleichtert automatisch erzeugte Changelogs [51], ist aber Geschmackssache. Wichtiger als das Format ist, dass es im Projekt einheitlich ist.

3.5 Diffs lesen

`git diff` vergleicht immer zwei Zustände. Welche, hängt von den Argumenten ab, und genau das verwirrt am Anfang [52]:

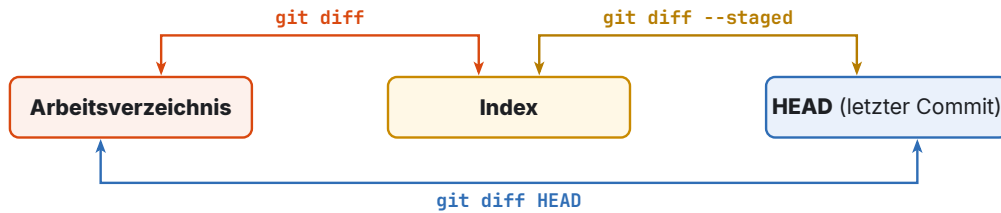


Abb. 5: Was `git diff` vergleicht, abhängig von den Argumenten.

```
git diff           # noch nicht vorgemerkte Änderungen
git diff --staged  # was in den nächsten Commit geht
git diff HEAD      # alles seit dem letzten Commit
git diff main feature  # Unterschied zwischen zwei Branches
git diff --stat    # nur Übersicht: welche Dateien, wie viele Zeilen
git diff --word-diff # wortweise statt zeilenweise (ideal für LaTeX, Typst, Markdown)
```

Ein Diff-Abschnitt beginnt mit einer Kopfzeile wie `@@ -12,7 +12,9 @@`: Im alten Stand ging es um 7 Zeilen ab Zeile 12, im neuen um 9 Zeilen ab Zeile 12. Zeilen mit `-` wurden entfernt, mit `+` hinzugefügt, ohne Präfix sind sie unveränderter Kontext [53].

TIPP

Für Texte (Paper, Skripte, Folien in LaTeX oder Typst) ist `git diff --word-diff` Gold wert [32], denn ein einziges geändertes Wort markiert sonst den ganzen Absatz. Noch besser funktioniert das, wenn im Quelltext jeder Satz in einer eigenen Zeile steht.

3.6 Die Historie ansehen

Befehl	Zeigt
<code>git log</code>	alle Commits des aktuellen Branches, neueste zuerst [54]
<code>git log --oneline --graph --all</code>	kompakter Graph aller Branches (als Alias <code>git lga</code>)
<code>git log -p</code>	jeden Commit mit vollständigem Diff
<code>git log --stat</code>	jeden Commit mit Liste der geänderten Dateien
<code>git log -n 5</code>	nur die letzten fünf Commits
<code>git log --since="2 weeks ago"</code>	Commits der letzten zwei Wochen
<code>git log --author="Haindl"</code>	nur Commits eines Autors
<code>git log -- src/login.py</code>	nur Commits, die diese Datei betreffen
<code>git log --follow -- datei</code>	Datei-Historie auch über Umbenennungen hinweg [54]
<code>git show a1b2c3d</code>	einen einzelnen Commit mit Nachricht und Diff [55]
<code>git show HEAD:src/app.py</code>	eine Datei so, wie sie im letzten Commit aussah [55]

Das doppelte `--` trennt Optionen von Dateipfaden. Es ist nur nötig, wenn ein Pfad mit einem Branch-Namen verwechselt werden könnte, schadet aber nie [20].

3.7 Dateien löschen, verschieben, ignorieren

```
git rm datei.txt      # löschen und Löschung vormerken
git rm --cached .env  # nicht mehr versionieren, Datei aber behalten
git mv alt.py neu.py  # umbenennen und vormerken
```

Welche Dateien Git gar nicht erst beachten soll, steht in der Datei `.gitignore` im Projekt. Sie wird selbst committet, damit sie für alle gilt [40]:

Muster	Bedeutung
*.log	alle Dateien mit Endung <code>.log</code> , in jedem Ordner
/build	nur <code>build</code> im Wurzelverzeichnis des Projekts
build/	jeder Ordner namens <code>build</code> , egal wo
**/tmp	<code>tmp</code> in beliebiger Tiefe
!wichtig.log	Ausnahme: diese Datei doch versionieren
.env	lokale Umgebungsvariablen und Geheimnisse

Wenn eine Datei trotz Eintrag nicht ignoriert wird, ist sie meistens schon versioniert: `.gitignore` wirkt nur auf Dateien, die Git noch nicht kennt [40]. Dann hilft `git rm --cached datei` [56]. Warum eine Datei ignoriert wird, verrät `git check-ignore -v datei` [57].

ACHTUNG

Wurde eine Datei mit Passwörtern oder Tokens (etwa `.env`) jemals committet und gepusht, steht sie in der Historie, auch wenn du sie danach löschst. Das Geheimnis gilt als kompromittiert und muss **sofort ausgetauscht** werden [58]. Das Umschreiben der Historie (Kapitel 8) ist erst der zweite Schritt.

Zeilenenden und Binärdateien mit `.gitattributes`

`.gitignore` entscheidet, **was** Git verfolgt; `.gitattributes` legt fest, **wie** Git bestimmte Dateien behandelt [59]. Das ist besonders wichtig, wenn macOS-, Linux- und Windows-Rechner zusammenarbeiten:

`.gitattributes`

```
*      text=auto
*.sh   text eol=lf
*.png  binary
*.pdf  binary
```

`text=auto` normalisiert erkannte Textdateien im Repository auf LF [59]. Für Shell-Skripte erzwingt `eol=lf` zusätzlich LF im Arbeitsverzeichnis [59], damit sie in Linux-Containern nicht an unsichtbaren CRLF-Zeichen scheitern. `binary` verhindert sinnlose Text-Diffs [59]. Nach dem erstmaligen Einführen prüfst du die einmalige Normalisierung mit `git add --renormalize .` [6] und `git diff --staged`, bevor du sie committest.

Große Binärdateien mit Git LFS

Git speichert jede Version einer Datei in der Historie. Große, häufig geänderte Binärdateien wie Videos, Datensätze oder Photoshop-Dateien blähen deshalb jeden Klon dauerhaft auf. Git LFS speichert im Repository nur kleine Zeiger und die eigentlichen Dateien in einem getrennten LFS-Speicher [60]:

```
brew install git-lfs
git lfs install
git lfs track "*.psd"
git add .gitattributes
```

Vorher prüfen, ob der Git-Server LFS unterstützt und genügend Speicher hat [61]. `git lfs track` wirkt nur auf künftige Commits [62]. Bereits vorhandene große Dateien migriert `git lfs migrate import` nur durch Umschreiben der Historie [63]; das ist eine Team-Entscheidung und verlangt anschließend koordinierte neue Klone oder Force-Pushes.

3.8 Der Kreislauf im Überblick



Abb. 6: Der tägliche Ablauf. Pushen muss man nicht nach jedem Commit, aber mindestens am Ende des Tages.

4 Branches

Branches sind in Git billig und schnell [11], deshalb solltest du sie großzügig einsetzen: für jede Funktion, jeden Bugfix, jedes Experiment. `main` bleibt dadurch jederzeit in einem funktionierenden Zustand, und angefangene Arbeit stört niemanden.

4.1 Anlegen und wechseln

```
git branch feature/login      # Branch anlegen (am aktuellen Commit), nicht wechseln
git switch feature/login      # zum Branch wechseln
git switch -c feature/login    # anlegen und wechseln in einem Schritt
git switch -c fix/typo a1b2c3d # Branch an einem bestimmten Commit anlegen
git switch -                   # zurück zum vorherigen Branch
```

Früher erledigte `git checkout` all das und noch viel mehr (auch das Zurücksetzen von Dateien). Seit Git 2.23 gibt es die klareren Befehle `git switch` für Branches und `git restore` für Dateien [11], [64]. `git checkout -b name` funktioniert weiterhin und taucht in vielen Anleitungen auf, meint aber dasselbe wie `git switch -c name` [65].

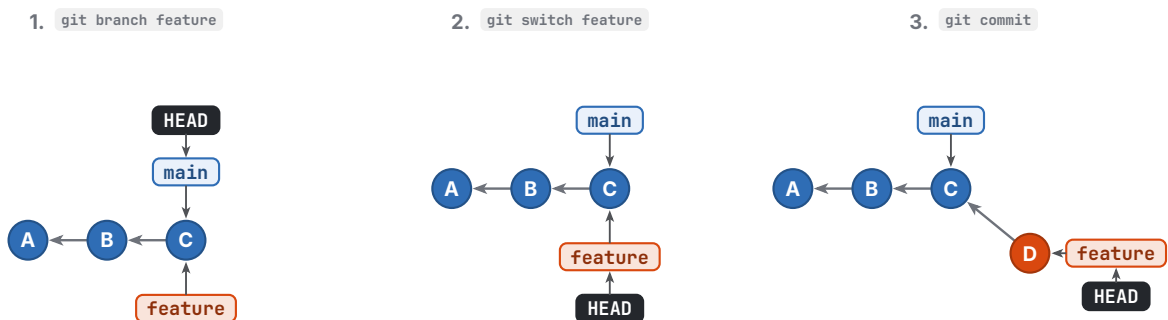


Abb. 7: Ein Branch entsteht als zweiter Zeiger auf denselben Commit. Erst beim nächsten Commit laufen die Zeiger auseinander.

Beim Wechseln passt Git das Arbeitsverzeichnis an den Schnappschuss des Ziel-Branches an: Dateien werden geändert, hinzugefügt oder entfernt [11]. Bei großen Unterschieden kann das einen Moment dauern, es passiert aber lokal ohne Netzwerk.

4.2 Branches verwalten

Befehl	Wirkung
<code>git branch</code>	lokale Branches auflisten, der aktuelle ist mit <code>*</code> markiert [12]
<code>git branch -vv</code>	zusätzlich letzter Commit, Upstream und ahead/behind [12]
<code>git branch -a</code>	auch Remote-Tracking-Branches (<code>origin/...</code>) anzeigen [12]
<code>git branch --merged</code>	Branches, deren Commits vollständig im aktuellen Branch enthalten sind [12]
<code>git branch --no-merged</code>	Branches mit noch nicht integrierten Commits [12]
<code>git branch -d feature</code>	löschen, aber nur wenn gemergt (sicher) [12]
<code>git branch -D feature</code>	löschen erzwingen, auch mit ungemergten Commits [12]
<code>git branch -m alt neu</code>	umbenennen (<code>-m neu</code> allein benennt den aktuellen Branch um) [12]

Beim Löschen eines Branches wird nur der Zeiger samt seinem eigenen Reflog entfernt [12]. Die Commits bleiben zunächst in der Datenbank und sind, sofern du sie ausgecheckt hattest, über das Reflog von `HEAD`

noch Wochen lang auffindbar [4], [66] (Kapitel 9). `-D` ist also weniger gefährlich, als es klingt, sollte aber trotzdem nur mit Absicht verwendet werden.

TIPP

Nach einem gemergten Pull Request räumst du lokal so auf: `git switch main`, `git pull`, `git branch -d feature/login`. Weigert sich Git mit „not fully merged“ [67], obwohl der PR gemergt ist, wurde vermutlich per Squash oder Rebase gemergt [30]: Die Commits in `main` sind dann neue Kopien, nicht deine ursprünglichen Commits. Hier ist `-D` korrekt.

4.3 Wechseln mit ungesicherten Änderungen

Hast du Dateien geändert und willst den Branch wechseln, gibt es zwei Fälle. Betreffen deine Änderungen Dateien, die sich zwischen den Branches nicht unterscheiden, nimmt Git sie einfach mit [68]. Das ist praktisch, wenn du merkst, dass du auf dem falschen Branch angefangen hast: `git switch -c richtiger-branch` nimmt alle Änderungen mit in den neuen Branch.

Würden deine Änderungen dagegen überschrieben, verweigert Git den Wechsel [68]:

```
error: Your local changes to the following files would be overwritten by checkout:
    src/app.py
Please commit your changes or stash them before you switch branches.
```

Dann hast du drei Möglichkeiten: die Arbeit als Zwischenstand committen (und später aufräumen, Kapitel 8), sie mit `git stash` beiseitelegen (Kapitel 9) oder den anderen Branch in einem zweiten Arbeitsverzeichnis öffnen, ohne den aktuellen zu verlassen (Worktrees, Kapitel 10).

4.4 Detached HEAD

Normalerweise zeigt `HEAD` auf einen Branch. Wechselst du direkt zu einem Commit oder Tag, zeigt `HEAD` dagegen unmittelbar auf einen Commit [5], [65]. Git warnt dann mit *You are in 'detached HEAD' state*.

```
git switch --detach v1.0    # alten Stand ansehen
```

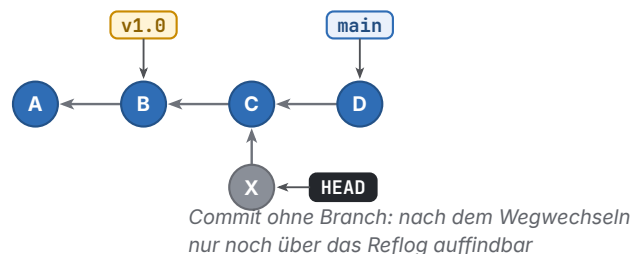


Abb. 8: Detached HEAD: Ein neuer Commit `x` hängt an keinem Branch.

Zum Ansehen, Testen oder Bauen eines alten Standes ist das völlig in Ordnung. Problematisch wird es nur, wenn du dort committest und dann wegwechselst: Dann zeigt nichts mehr auf die neuen Commits, und die Garbage Collection kann sie irgendwann entfernen [65]. Die Lösung ist einfach, solange du noch dort stehst:

```
git switch -c rettung      # macht aus dem losen Stand einen richtigen Branch
```

4.5 Namenskonventionen

Branch-Namen dürfen Schrägstriche enthalten [69], was sich für Gruppierungen anbietet: `feature/login`, `fix/csv-import`, `hotfix/2.3.1`, `docs/api`. Bewährt haben sich Kleinbuchstaben, Bindestriche statt Leerzeichen und, wo vorhanden, die Issue-Nummer: `feature/42-login-sperre`. Gitea und die Tab-Vervollständigung sortieren und gruppieren dann sinnvoll.

5 Remotes: Arbeiten mit dem Server

Bis hierhin lief alles lokal. Git ist ein verteiltes System: Dein Mac hat eine vollständige Kopie des Repositories mit allen Commits, genau wie der Gitea-Server. Remotes sind die Verbindung zwischen diesen Kopien. Dabei gilt ein Grundsatz, der viele Missverständnisse erklärt: **Git spricht nur dann mit dem Server, wenn du es ausdrücklich verlangst**, also bei `clone`, `fetch`, `pull` und `push` [70]. Alles andere, auch `git status`, arbeitet mit dem Wissensstand des letzten Kontakts [70].

5.1 Remotes verwalten

Ein Remote ist nichts anderes als ein Name für eine URL [71]. Nach `git clone` heißt er `origin` [44], [72].

```
git remote -v           # alle Remotes mit URL anzeigen
git remote add origin gitea:team/demo.git # Remote hinzufügen (nach git init)
git remote set-url origin gitea:team/demo.git # URL ändern, z.B. nach Umzug des Servers
git remote rename origin gitea # umbenennen
git remote remove altes-remote # entfernen
```

Mehrere Remotes sind möglich und manchmal nützlich, etwa `origin` für den eigenen Gitea-Server und `github` für einen öffentlichen Spiegel. Ein Push geht immer an genau ein Remote.

5.2 Remote-Tracking-Branches

Neben deinen lokalen Branches verwaltet Git für jedes Remote **Remote-Tracking-Branches** wie `origin/main`. Sie sind deine lokale, schreibgeschützte Erinnerung daran, wo der Branch `main` auf dem Server beim letzten Kontakt stand [5], [70]. Du arbeitest nie direkt auf ihnen. Sie werden nur durch `fetch`, `pull` und `push` aktualisiert.



Abb. 9: Du hast lokal `c` committet, jemand anderes hat inzwischen `d` gepusht. Dein `origin/main` weiß davon noch nichts.

In dieser Situation meldet `git status` fröhlich *Your branch is ahead of ,origin/main' by 1 commit*. Das stimmt aber nur gemessen am Stand des letzten Kontakts. Dass der Server weitergelaufen ist, erfährst du erst nach einem `git fetch` [70].

5.3 fetch, pull und push

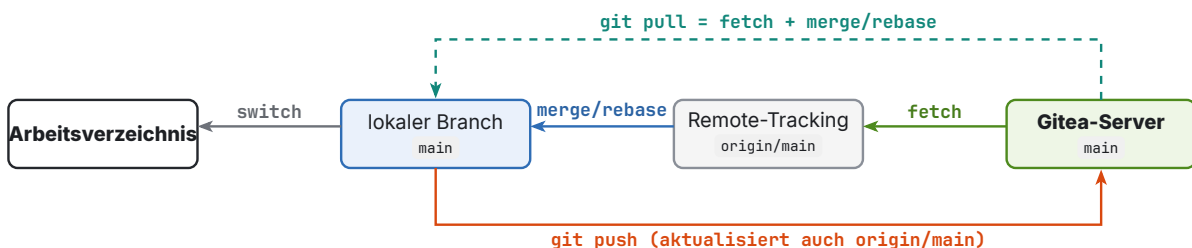


Abb. 10: Welcher Befehl was bewegt.

- `git fetch` lädt neue Commits vom Server und aktualisiert die Remote-Tracking-Branches [28]. Deine lokalen Branches und dein Arbeitsverzeichnis bleiben unangetastet [72]. `fetch` ist deshalb **immer ungefährlich** und eine gute Gewohnheit, bevor du irgendetwas integrierst.
- `git pull` ist `fetch` plus Integration: Der Remote-Stand wird in deinen aktuellen Branch eingearbeitet, je nach Konfiguration per Merge oder Rebase [26], [72].
- `git push` überträgt deine lokalen Commits und verschiebt den Branch auf dem Server. Das gelingt nur, wenn der Server-Branch ein Vorfahre deines lokalen Branches ist, der Server also nichts hat, was dir fehlt [27].

Der Upstream

Jeder lokale Branch kann einen *Upstream* haben, den Remote-Branch, mit dem er standardmäßig abgeglichen wird [5]. Erst dadurch wissen `git pull`, `git push` und `git status`, womit sie vergleichen sollen. Bei einem neuen Branch setzt man ihn beim ersten Push [27]:

```
git push -u origin feature/login # -u = --set-upstream
git branch -vv                  # zeigt den Upstream jedes Branches
```

Mit der Einstellung `push.autoSetupRemote = true` aus Kapitel 2 genügt ein schlichtes `git push` [27]. In Git-Ausdrücken lässt sich der Upstream als `@{u}` ansprechen [13]: `git log @{u}..` zeigt, welche Commits du noch nicht gepusht hast.

5.4 Wenn der Push abgelehnt wird

Die häufigste Fehlermeldung im Umgang mit Remotes:

```
! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'gitea:team/demo.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally.
```

Das ist die Situation aus Abb. 9: Server und lokaler Branch sind auseinandergefahren. Die Lösung ist **nicht** `--force`, denn damit würdest du den Commit `D` auf dem Server vernichten [27]. Stattdessen holst du den Server-Stand und setzt deine Arbeit obendrauf:

```
git pull --rebase # mit pull.rebase = true genügt git pull
git push
```



Abb. 11: Dein Commit `c` wird als `c'` auf `D` gesetzt. Danach ist ein normaler Push möglich.

5.5 git pull richtig konfigurieren

Ohne Konfiguration verlangt Git bei auseinandergefahrenen Branches eine Entscheidung [26]; mit `pull.rebase = false` erzeugt `git pull` einen Merge-Commit [26] („Merge branch ‚main‘ of ...“). Ein solcher Commit kann eine bewusste Zusammenführung dokumentieren, entsteht beim beiläufigen Pull aber oft nur als technisches Nebenprodukt. Zwei sinnvolle Einstellungen gibt es:

Einstellung	Verhalten bei auseinandergefahrenen Branches
<code>pull.rebase = true</code>	Deine lokalen Commits werden auf den Server-Stand gesetzt [26]. Lineare Historie, empfohlen für die meisten Fälle.
<code>pull.ff = only</code>	<code>pull</code> verweigert [26] und du entscheidest selbst, ob du <code>git rebase</code> oder <code>git merge</code> ausführst. Maximal kontrolliert, etwas umständlicher.

5.6 Aufräumen und Remote-Banches löschen

```
git push origin --delete feature/login # Branch auf dem Server löschen
git fetch --prune                      # verschwundene origin/...-Branches lokal entfernen
git branch -vv | grep '[: gone]'      # lokale Branches, deren Upstream gelöscht wurde
```

Gitea kann Branches nach dem Mergen eines Pull Requests automatisch löschen [73] (Kapitel 12). Mit `fetch.prune = true` verschwinden dann auch die zugehörigen `origin/...`-Einträge automatisch beim nächsten Fetch [28].

6 Korrekt mergen

Mergen heißt, die Arbeit von zwei Branches zusammenzuführen. Git kennt dafür mehrere Wege, die im Ergebnis denselben Dateistand erzeugen, aber eine unterschiedlich aussehende Historie hinterlassen. Welcher Weg „korrekt“ ist, hängt davon ab, was die Historie später erzählen soll. Dieses Kapitel erklärt die Varianten, zeigt, wie Gitea sie beim Pull Request anbietet, und behandelt ausführlich Konflikte.

Alle Beispiele gehen davon aus, dass du einen Branch `feature` in `main` integrieren willst. Gemergt wird immer **in den aktuellen Branch hinein** [30]:

```
git switch main
git pull          # main auf den neuesten Stand bringen
git merge feature # feature in main integrieren
```

6.1 Die Merge-Basis

Für jeden Merge sucht Git zuerst den jüngsten gemeinsamen Vorfahren der beiden Branches, die **Merge-Basis** [74]. Sie ist der Punkt, an dem die Wege auseinandergegangen sind, und entscheidet, welche Änderungen von welcher Seite stammen.

```
git merge-base main feature # zeigt den Hash der Merge-Basis
```

6.2 Fast-Forward

Hat `main` seit dem Abzweigen von `feature` keine neuen Commits bekommen, ist die Merge-Basis identisch mit dem Ende von `main`. Dann gibt es nichts zusammenzuführen: Git schiebt den Zeiger `main` einfach nach vorne. Das heißt *Fast-Forward*, es entsteht kein neuer Commit [30], [75].



Abb. 12: Fast-Forward: `main` wird nur weitergeschoben.

6.3 Der echte Merge (Drei-Wege-Merge)

Hat `main` inzwischen eigene Commits, sind die Branches auseinandergefahren. Git vergleicht dann drei Stände: die Merge-Basis, das Ende von `main` (*ours*) und das Ende von `feature` (*theirs*) [75]. Änderungen, die nur auf einer Seite passiert sind, werden übernommen. Haben beide Seiten dieselbe Stelle unterschiedlich geändert, entsteht ein Konflikt. Das Ergebnis wird als **Merge-Commit** mit zwei Eltern festgehalten [5], [75].



Abb. 13: Drei-Wege-Merge: Der Merge-Commit **M** hat die Eltern **E** (erstes Elternteil) und **D** (zweites).

Das **erste Elternteil** eines Merge-Commits ist immer der Branch, in dem du standest (hier `main`) [8]. Das ist später nützlich: `git log --first-parent main` zeigt nur die Hauptlinie, also pro gemergtem Feature einen Eintrag [54].

Merge-Commit erzwingen: `--no-ff`

Auch wenn ein Fast-Forward möglich wäre, kannst du einen Merge-Commit erzwingen [30]. Der Vorteil: Die Commits des Features bleiben in der Historie als zusammengehörige Gruppe erkennbar, und das ganze Feature lässt sich mit einem einzigen `git revert -m 1` zurücknehmen [76].

```
git merge --no-ff feature # immer einen Merge-Commit erzeugen
git merge --ff-only feature # nur Fast-Forward, sonst abbrechen
```

6.4 Die Strategien im Vergleich

In der Praxis wählst du zwischen vier Formen, die Gitea auch als Schaltflächen beim Mergen eines Pull Requests anbietet [77] (Kapitel 12). Ausgangslage ist jeweils die linke Seite von Abb. 13.

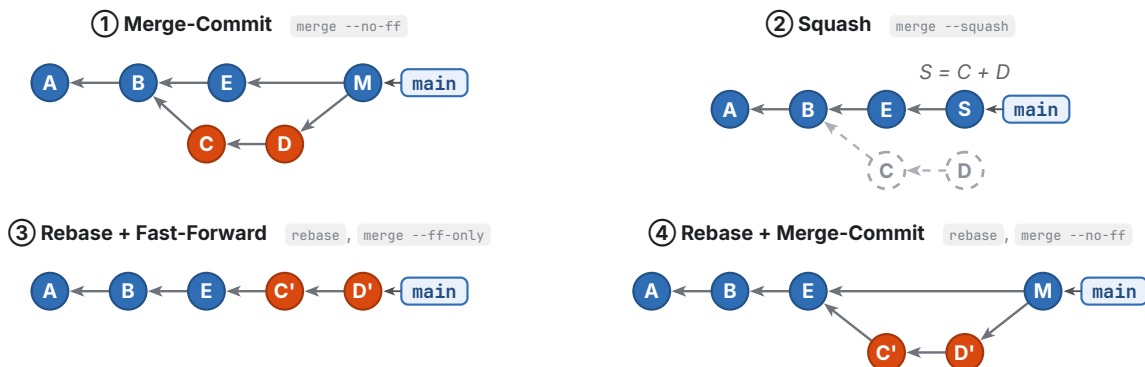


Abb. 14: Vier Ergebnisse aus derselben Ausgangslage. Der Dateistand am Ende ist in allen vier Fällen identisch.

Strategie	Vorteile	Nachteile
① Merge-Commit	Historie zeigt exakt, was wann parallel lief. Feature als Einheit revertierbar [76]. Keine Commits werden umgeschrieben.	Historie wird bei vielen Branches unübersichtlich („Gleisanlage“). Zwischencommits wie „Tippfehler“ landen in <code>main</code> .
② Squash	Ein sauberer Commit pro Feature, <code>main</code> bleibt linear und lesbar. Unordentliche Zwischencommits verschwinden.	Einzelschritte des Features gehen in <code>main</code> verloren. Der Feature-Branch gilt für Git nicht als gemergt (<code>branch -d</code> weigert sich) [12], [30].
③ Rebase + FF	Völlig lineare Historie mit allen Einzelcommits. <code>git bisect</code> findet Fehler auf Commit-Ebene [78].	Commits bekommen neue Hashes [79]. Jeder Einzelcommit sollte sauber sein, sonst landet Unordnung linear in <code>main</code> .
④ Rebase + Merge-Commit	Lineare Einzelcommits und trotzdem eine sichtbare Klammer pro Feature (<i>semi-linear</i>).	Etwas mehr Commits, aber meist der beste Kompromiss für Teams.

Welche Strategie?

Für ein kleines Team oder ein Soloprojekt mit Pull Requests hat sich folgende Faustregel bewährt:

- **Squash** für kleine Pull Requests, deren Einzelcommits niemanden interessieren (typisch: ein Bugfix, eine kleine Funktion). Das ist der häufigste Fall.
- **Rebase + Merge-Commit** (oder Rebase + FF) für größere Features, deren Commits bewusst in sinnvolle Schritte gegliedert sind.
- **Merge-Commit ohne Rebase** für langlebige Branches, die mehrfach mit `main` abgeglichen werden, etwa Release-Branches.
- Legt euch pro Repository auf **eine** Standardstrategie fest. Gitea erlaubt, nicht gewünschte Stile in den Repository-Einstellungen abzuschalten.

MERKE

Egal welche Strategie: Commits auf `main`, die schon auf dem Server liegen, werden **nicht** umgeschrieben [79]. Rebase und Squash betreffen nur den Feature-Branch vor dem Mergen.

6.5 Konflikte lösen

Ein Konflikt entsteht, wenn beide Seiten dieselbe Stelle einer Datei unterschiedlich geändert haben, oder wenn eine Seite eine Datei ändert, die die andere gelöscht hat [7], [75]. Git hält dann an und überlässt dir die Entscheidung. Das ist kein Fehler, sondern Absicht: Welche Variante inhaltlich richtig ist, kann nur ein Mensch entscheiden.

```
Auto-merging src/config.py
CONFLICT (content): Merge conflict in src/config.py
Automatic merge failed; fix conflicts and then commit the result.
```

`git status` listet die betroffenen Dateien unter *Unmerged paths* [75]. In jeder solchen Datei markiert Git die Konfliktstelle. Mit der Einstellung `merge.conflictStyle = zdiff3` aus Kapitel 2 sieht das so aus [30]:

`src/config.py`

```
<<<<<< HEAD
TIMEOUT = 30
RETRIES = 5
|||||| 91b0c4e
TIMEOUT = 10
RETRIES = 3
=====
TIMEOUT = 10
RETRIES = 3
BACKOFF = 2.0
>>>>>> feature
```

Von oben nach unten: **deine Seite** (`HEAD`, hier `main`), der **gemeinsame Ursprung** (die Merge-Basis) und **die andere Seite** (`feature`) [30]. Erst durch den Ursprung wird klar, was passiert ist: `main` hat die Werte erhöht, `feature` hat eine Zeile hinzugefügt. Die richtige Lösung übernimmt also beides:

```
TIMEOUT = 30
RETRIES = 5
BACKOFF = 2.0
```

Der Ablauf ist immer derselbe:

1. Datei im Editor öffnen, jede Konfliktstelle inhaltlich lösen und **alle Markierungszeilen entfernen**.
2. Prüfen, ob das Projekt noch funktioniert, etwa mit `make test`.
3. Die gelöste Datei mit `git add datei` als erledigt markieren.
4. Wenn alle Dateien gelöst sind: `git commit` (bei einem Merge) oder `git rebase --continue` (bei einem Rebase) [30], [75].

Wer mittendrin merkt, dass er den Merge doch nicht will, kehrt mit `git merge --abort` (bzw. `git rebase --abort`) zum Zustand vor dem Merge zurück [29], [30].

Ganze Dateien von einer Seite übernehmen

Manchmal ist klar, dass eine Seite komplett richtig ist, etwa bei generierten Dateien oder Lockfiles:

```
git checkout --ours -- package-lock.json # unsere Version nehmen
git checkout --theirs -- package-lock.json # die andere Version nehmen
git add package-lock.json
```

ACHTUNG

Beim **Rebase** sind `--ours` und `--theirs` vertauscht! Ein Rebase setzt deine Commits auf den anderen Branch. „Ours“ ist dabei der Branch, auf den du setzt (etwa `main`), „theirs“ sind deine eigenen Commits [29], [65]. Im Zweifel die Datei öffnen und nachsehen, statt blind eine Seite zu wählen.

Grafische Werkzeuge

VS Code erkennt Konfliktdateien automatisch und bietet über den Merge-Editor Schaltflächen wie *Accept Current*, *Accept Incoming* und *Accept Both* [80]. Als Werkzeug für `git mergetool` richtest du es so ein [80], [81]:

```
git config --global merge.tool vscode
git config --global mergetool.vscode.cmd 'code --wait --merge "$REMOTE" "$LOCAL" "$BASE"
"$MERGED"'
git mergetool # öffnet nacheinander jede Konfliktdatei
```

Alternativ ist mit Xcode das Apple-Werkzeug FileMerge dabei (`git mergetool --tool=opendiff`) [82].

Konflikte seltener machen

Die meisten Konflikte entstehen, weil Branches zu lange leben. Kleine Pull Requests, die in Tagen statt Wochen gemergt werden, haben wenig Konfliktpotenzial. Ein langlebiger Feature-Branch sollte regelmäßig `main` aufnehmen (per Rebase oder Merge), damit Konflikte in kleinen Portionen statt auf einen Schlag kommen. Automatische Formatierer (etwa per Pre-Commit-Hook oder in der CI) verhindern zusätzlich reine Formatierungskonflikte. Und `rerere.enabled` sorgt dafür, dass du denselben Konflikt beim wiederholten Rebase nur einmal lösen musst [31].

7 Rebase im Detail

Rebase ist das mächtigste Werkzeug, um Historie zu gestalten, und gleichzeitig das, vor dem am häufigsten gewarnt wird. Beides hat denselben Grund: Rebase erzeugt neue Commits und verwirft die alten [79]. Wer versteht, was dabei genau passiert, kann es gezielt und gefahrlos einsetzen.

7.1 Was Rebase tut

`git rebase main` (ausgeführt auf `feature`) bedeutet: „Nimm alle Commits, die `feature` hat und `main` nicht, und spiele sie der Reihe nach auf dem aktuellen Ende von `main` neu ab.“ Git berechnet dafür für jeden Commit den Diff zu seinem Elternteil, wendet ihn auf der neuen Basis an und erzeugt einen neuen Commit mit gleicher Nachricht und gleichem Autor, aber neuem Elternteil und damit **neuem Hash** [29], [79].

```
git switch feature
git rebase main
```

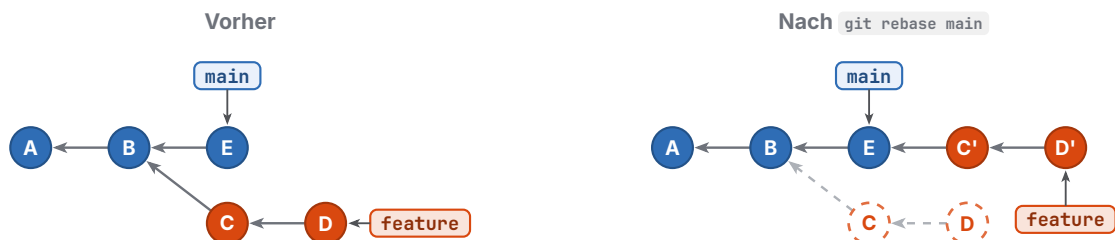


Abb. 15: `C` und `D` werden als neue Commits `C'` und `D'` auf `E` gesetzt. Die Originale bleiben unerreichbar zurück.

Danach kann `main` per Fast-Forward nachgezogen werden, die Historie ist linear. Die ursprünglichen Commits `C` und `D` existieren noch (über das Reflog auffindbar), gehören aber zu keinem Branch mehr [66].

7.2 Die goldene Regel

ACHTUNG

Rebase keine Commits, auf denen andere bereits aufbauen. Wer deine alten Commits `C` und `D` schon geholt hat und darauf weiterarbeitet, hat nach deinem Rebase eine Historie, die nicht mehr zu deiner passt. Das Ergebnis sind doppelte Commits und verwirrende Konflikte [29], [79].

In der Praxis heißt das:

- Eigene, noch nicht gepushte Commits darfst du beliebig umbauen.
- Einen eigenen Feature-Branch, den du nur für den Pull Request gepusht hast, darfst du ebenfalls umbauen. Danach ist ein `git push --force-with-lease` nötig [27] (Kapitel 8). Arbeitet jemand anderes auf demselben Branch mit, sprich das vorher ab.
- Gemeinsame Branches wie `main` werden nie umgeschrieben. Gitea verhindert das zusätzlich über Branch-Schutzregeln [83] (Kapitel 12).

7.3 Interaktiver Rebase

Mit `-i` (*interactive*) zeigt Git vor dem Abspielen eine Liste der betroffenen Commits im Editor [9], [29]. Durch Bearbeiten dieser Liste bestimmst du, was mit jedem Commit geschieht:

```
git rebase -i HEAD~4      # die letzten vier Commits bearbeiten
git rebase -i main        # alle Commits des Branches seit der Abzweigung von main
```

Editor: git-rebase-todo

```
pick 3a1f2c0 Login-Formular anlegen
pick 7b9e4d1 Validierung ergänzen
pick c02d8a3 Tippfehler
pick e5f6b72 Tests für Login

# Commands:
# p, pick = use commit
# r, reword = use commit, but edit the commit message
# ...
```

MERKE

Die Liste steht in **chronologischer Reihenfolge**, der älteste Commit oben. Das ist genau umgekehrt zu `git log`, das den neuesten zuerst zeigt [9], [29].

Befehl	Kurz	Wirkung
pick	p	Commit unverändert übernehmen.
reword	r	Commit übernehmen, aber Nachricht im Editor ändern.
edit	e	Nach diesem Commit anhalten, um ihn zu verändern (Dateien, Aufteilen). Weiter mit <code>git rebase --continue</code> .
squash	s	Mit dem vorherigen Commit verschmelzen, beide Nachrichten im Editor zusammenführen.
fixup	f	Mit dem vorherigen Commit verschmelzen, die eigene Nachricht verwerfen.
fixup -C		Verschmelzen und stattdessen die Nachricht dieses Commits verwenden [29].
drop	d	Commit weglassen (eine Zeile zu löschen wirkt genauso) [9], [29].
exec	x	Einen Shell-Befehl ausführen, z.B. <code>x make test</code> nach jedem Commit.
break	b	An dieser Stelle anhalten, weiter mit <code>--continue</code> .

Zeilen lassen sich auch umsortieren, um Commits in eine andere Reihenfolge zu bringen. Für das Beispiel oben wäre eine sinnvolle Aufräumaktion, den Tippfehler-Commit mit dem Formular-Commit zu verschmelzen:

```
pick 3a1f2c0 Login-Formular anlegen
fixup c02d8a3 Tippfehler
pick 7b9e4d1 Validierung ergänzen
pick e5f6b72 Tests für Login
```

Nach dem Speichern und Schließen des Editors arbeitet Git die Liste ab. Aus vier Commits werden drei saubere.

7.4 Der Fixup-Workflow

Noch eleganter ist es, Korrekturen gleich beim Committen als solche zu markieren [29], [49]. Angenommen, du bemerkst einen Fehler in einem früheren Commit `3a1f2c0` deines Branches:

```
git add src/login.py
git commit --fixup=3a1f2c0      # erzeugt einen Commit "fixup! Login-Formular anlegen"
# ... weiterarbeiten, weitere Fixups ...
git rebase -i main              # mit rebase.autoSquash = true werden die fixup!-Commits
                                # automatisch hinter ihr Ziel sortiert und als fixup markiert
```

Du musst in der Liste nur noch speichern. Dieses Muster ist ideal für Review-Anmerkungen in Pull Requests: Jede Korrektur wird als `fixup!` committet und gepusht, der Reviewer sieht genau, was sich geändert hat, und vor dem Mergen werden alle Fixups mit einem Rebase eingefaltet.

7.5 Branches umhängen: --onto

Manchmal baut ein Branch auf einem anderen auf, der inzwischen per Squash gemergt wurde. Ein einfaches `git rebase main` würde dann versuchen, auch die schon enthaltenen Commits erneut anzuwenden. Mit `--onto` sagst du Git genau, welche Commits es wohin setzen soll [29], [79]:

```
git rebase --onto main feature-1 feature-2
# Setze die Commits von feature-2, die nicht in feature-1 sind, auf main.
```



Abb. 16: `feature-1` wurde als Squash-Commit `s` gemergt. Nur `F` und `G` werden auf `main` übertragen.

Arbeitest du mit mehreren aufeinander aufbauenden Branches (*stacked branches*), verschiebt die Einstellung `rebase.updateRefs = true` aus Kapitel 2 beim Rebase des obersten Branches automatisch alle darunterliegenden Branch-Zeiger mit [29].

7.6 Während eines Rebase

Ein Rebase kann an jedem Commit anhalten: bei `edit`, `break` oder einem Konflikt [29]. `git status` zeigt dann an, wo du stehst (*interactive rebase in progress; onto 5e1d2f3*), und der Prompt aus Kapitel 2 zeigt `rebase-i`.

Befehl	Wirkung
<code>git rebase --continue</code>	Nach dem Lösen eines Konflikts (Datei bearbeiten, <code>git add</code>) oder nach einem <code>edit</code> weitermachen.
<code>git rebase --skip</code>	Den aktuellen Commit komplett weglassen, etwa wenn seine Änderung schon in <code>main</code> enthalten ist.
<code>git rebase --abort</code>	Alles rückgängig machen, zurück zum Zustand vor dem Rebase.

Weil ein Rebase Commit für Commit vorgeht, kann derselbe Konflikt in mehreren Commits hintereinander auftauchen. Das ist lästig, aber normal. Hier hilft `rerere` [31]. Und falls das Ergebnis nach Abschluss doch nicht gefällt: Git merkt sich den Stand vor dem Rebase in `ORIG_HEAD`, `git reset --hard ORIG_HEAD` stellt ihn wieder her [29] (solange du seitdem nichts anderes Größeres gemacht hast, sonst über das Reflog, Kapitel 9).

8 Commits nachträglich ändern

Eine falsche Commit-Nachricht, eine vergessene Datei, ein Commit, der eigentlich zwei sein sollte: Solche Korrekturen sind in Git alltäglich. Nach Kapitel 1 ist klar, was dabei technisch passiert: Commits sind unveränderlich, „ändern“ heißt immer, einen neuen Commit zu erzeugen und den Branch auf ihn umzuhängen. Die entscheidende Frage ist daher nicht *wie*, sondern **ob der Commit schon gepusht wurde**.

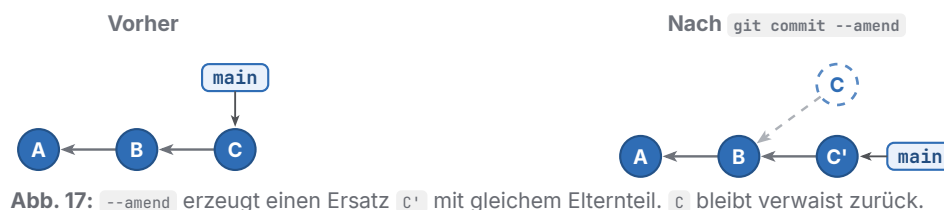
8.1 Übersicht

Situation	Befehl
Nachricht des letzten Commits ändern	<code>git commit --amend</code>
Datei im letzten Commit vergessen	<code>git add datei</code> + <code>git commit --amend --no-edit</code>
Nachricht eines älteren Commits ändern	<code>git rebase -i mit reword</code>
Nachrichten mehrerer Commits ändern	<code>git rebase -i mit mehreren reword</code>
Inhalt eines älteren Commits korrigieren	<code>git commit --fixup</code> + <code>git rebase -i --autosquash</code>
Commit in zwei aufteilen	<code>git rebase -i mit edit</code> + <code>git reset HEAD^</code>
Autor bzw. E-Mail korrigieren	<code>git commit --amend --reset-author</code>
bereits gepushten Feature-Branch aktualisieren	<code>git push --force-with-lease</code>
bereits in <code>main</code> gepusht	nicht umschreiben, siehe Abschnitt 8.6

8.2 Die letzte Commit-Nachricht ändern

Der einfachste und häufigste Fall. `--amend` ersetzt den letzten Commit durch einen neuen [49]:

```
git commit --amend # öffnet den Editor mit der alten Nachricht
git commit --amend -m "Login-Sperre einführen" # neue Nachricht direkt angeben
```



ACHTUNG

`--amend` nimmt **alles mit, was gerade im Index liegt** [49]. Wer nur die Nachricht korrigieren will, prüft vorher mit `git status`, dass nichts vorgemerkt ist. Sonst landet eine halbfertige Änderung versehentlich im korrigierten Commit.

8.3 Vergessene Dateien nachreichen

```
git add vergessene_datei.py
git commit --amend --no-edit # Nachricht unverändert lassen
```

Mit dem Alias `amend` aus der Konfiguration in Kapitel 2 verkürzt sich die zweite Zeile auf `git amend`.

8.4 Die Nachricht eines älteren Commits ändern

Hier hilft der interaktive Rebase aus Kapitel 7. Schritt für Schritt:

1. Den Commit finden. Mit `git log --oneline` siehst du, wie weit er zurückliegt:

```
e5f6b72 (HEAD -> feature/login) Tests für Login
7b9e4d1 Validierung ergänzen
3a1f2c0 Login-Formular anlegen
91b0c4e (main) README ergänzt
```

2. Den Rebase starten, und zwar ab dem Elternteil des betroffenen Commits. Der Tippfehler steckt im zweitneuesten Commit, also reichen die letzten zwei:

```
git rebase -i HEAD~2      # oder: git rebase -i 7b9e4d1^  (^ = "Elternteil von")
```

3. In der Liste pick durch reword (oder kurz r) ersetzen, speichern und den Editor schließen [29]:

```
r 7b9e4d1 Validierung ergänzen
pick e5f6b72 Tests für Login
```

4. Git öffnet den Editor ein zweites Mal, jetzt mit der Nachricht dieses Commits. Korrigieren, speichern, schließen. Git spielt die restlichen Commits automatisch ab [29]:

```
[detached HEAD 4c8d0e2] Validierung ergänzen
Successfully rebased and updated refs/heads/feature/login.
```



Abb. 18: Der korrigierte Commit und **alle danach** bekommen neue Hashes, frühere Commits bleiben unverändert.

Mehrere Nachrichten auf einmal änderst du, indem du in Schritt 3 mehrere Zeilen auf `reword` setzt. Git öffnet den Editor dann nacheinander für jede [9]. Soll auch der allererste Commit des Repositories geändert werden, der kein Elternteil hat, verwendest du `git rebase -i --root` [29].

TIPP

Abkürzung ohne Bearbeiten der Liste: `git commit --fixup=reword:7b9e4d1` fragt sofort nach der neuen Nachricht und legt einen Markierungs-Commit an [49]. Ein späteres `git rebase -i --autosquash main` wendet die Korrektur an [29].

8.5 Einen Commit aufteilen

Hat ein Commit zu viel auf einmal geändert, lässt er sich per `edit` zerlegen [9]:

```
git rebase -i HEAD~3      # beim betroffenen Commit "edit" eintragen
# Git hält nach diesem Commit an
git reset HEAD^           # Commit auflösen, Änderungen bleiben als unstaged erhalten
git add -p                # ersten Teil auswählen
git commit -m "Validierung ergänzen"
git add -p                # zweiten Teil auswählen
git commit -m "Fehlermeldungen übersetzen"
git rebase --continue
```

8.6 Wenn der Commit schon gepusht ist

Hast du einen Branch bereits gepusht und danach lokal umgeschrieben, lehnt der Server einen normalen Push ab, weil deine neue Historie die alte nicht mehr enthält [27]. Du musst den Server anweisen, seinen Branch zu ersetzen. Dafür gibt es drei Varianten:

Befehl	Verhalten
<code>git push --force</code>	Überschreibt den Server-Branch bedingungslos. Hat inzwischen jemand anderes etwas gepusht, ist es weg [27]. Vermeiden.
<code>git push --force-with-lease</code>	Überschreibt nur, wenn der Server-Branch noch dort steht, wo dein <code>origin/feature</code> ihn zuletzt gesehen hat. Sonst Abbruch mit <i>stale info</i> [27].
<code>git push --force-with-lease --force-if-includes</code>	Prüft zusätzlich, ob du den Server-Stand auch tatsächlich in deine Arbeit aufgenommen hast. Schützt vor dem Fall, dass ein zwischenzeitliches <code>git fetch</code> die „Lease“ unbemerkt erneuert hat [27].

Die Idee der „Lease“ (Pacht): Du darfst den Branch nur überschreiben, solange du nachweisen kannst, dass du den aktuellen Stand kennst. Als Alias lohnt sich

```
git config --global alias.pushf "push --force-with-lease --force-if-includes"
```

Und wenn es schon in `main` ist?

Commits auf einem geschützten, gemeinsam genutzten Branch werden nicht umgeschrieben. Gitea lehnt einen Force-Push auf `main` bei aktivem Branch-Schutz ohnehin ab [83]. Für eine unglückliche Nachricht gibt es pragmatische Wege:

- Mit der Tippfehler-Nachricht leben. Das ist fast immer die richtige Antwort.
- Eine Notiz anhängen, ohne den Commit zu ändern: `git notes add -m "Gemeint war: ..." 7b9e4d1`. Notizen erscheinen in `git log`, müssen aber separat gepusht werden (`git push origin refs/notes/*`) [84].
- Bei inhaltlichen Fehlern: `git revert` (Kapitel 9).

TIPP

Bei Squash-Merges in Gitea kannst du die Nachricht des entstehenden Commits im Merge-Dialog frei bearbeiten. Unordentliche Einzelnachrichten im Pull Request sind dann egal, entscheidend ist nur die Nachricht, die du beim Mergen formulierst.

8.7 Autor und E-Mail korrigieren

Wurde mit falscher Identität committet, etwa mit der privaten statt der dienstlichen Adresse, korrigierst du zuerst die Konfiguration und dann den Commit [49]:

```
git config user.email "philipp.haendl@firma.example"
git commit --amend --reset-author --no-edit          # letzter Commit

# mehrere Commits, z.B. die letzten fünf:
git rebase HEAD~5 --exec "git commit --amend --reset-author --no-edit"
```

Ganz ohne Umschreiben geht es mit einer Datei `.mailmap`: Sie ordnet alte Namen und Adressen den richtigen zu, und `log`, `shortlog` sowie `blame` zeigen die korrigierte Identität, ohne dass sich ein Hash ändert [85]:

.mailmap

```
Philipp Haendl <philipp.haendl@firma.example> <philipp@privat.example>
```

8.8 Große Umbauten: Dateien aus der gesamten Historie entfernen

Muss eine Datei aus *allen* Commits verschwinden (Passwort, riesige Binärdatei), reicht Rebase nicht. Dafür gibt es `git filter-repo` (`brew install git-filter-repo` [86]), den empfohlenen Nachfolger von `git filter-branch` [87], [88]:

```
git filter-repo --path .env --invert-paths          # .env aus allen Commits entfernen
```

Das schreibt **jeden** Commit ab dem ersten Vorkommen um. Danach müssen alle Branches mit Force gepusht werden (Branch-Schutz vorübergehend lockern), und alle Beteiligten klonen neu [88]. Bei Geheimnissen gilt: zuerst austauschen, dann aufräumen.

9 Rückgängig machen und Rettung

Die gute Nachricht vorweg: In Git geht erstaunlich wenig endgültig verloren. Alles, was einmal committet wurde, lässt sich wochenlang wiederfinden [4]. Wirklich gefährlich sind nur zwei Dinge: Änderungen, die **nie committet** wurden, und Befehle, die das Arbeitsverzeichnis überschreiben (`reset --hard`, `restore`, `clean`). Dieses Kapitel ordnet die Werkzeuge danach, was sie verändern.

9.1 Die Werkzeuge im Überblick

Befehl	Wirkt auf	Historie	Nach Push
<code>git restore</code>	Dateien im Arbeitsverzeichnis oder Index	unverändert	unkritisch
<code>git reset</code>	verschiebt den Branch, optional Index und Dateien	umgeschrieben	nur eigene Branches
<code>git revert</code>	neuer Commit, der einen alten umkehrt	ergänzt	sicher
<code>git reflog</code>	zeigt frühere Positionen von <code>HEAD</code>	nur lesend	nur lokal
<code>git stash</code>	legt Änderungen beiseite	unverändert	nur lokal
<code>git cherry-pick</code>	kopiert einen Commit auf den aktuellen Branch	ergänzt	sicher
<code>git clean</code>	löscht unversionierte Dateien	unverändert	unkritisch

9.2 `restore` : Dateien zurücksetzen

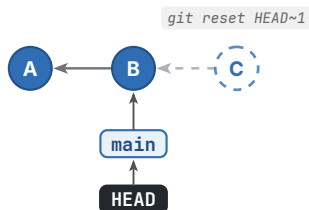
```
git restore src/app.py           # lokale Änderungen verwerfen (Stand aus dem Index)
git restore --staged src/app.py  # Vormerkung aufheben, Änderung bleibt
git restore --source=HEAD~3 src/app.py # Datei auf den Stand von vor drei Commits bringen
git restore -p src/app.py        # nur einzelne Abschnitte verwerfen
```

ACHTUNG

`git restore datei` (ohne `--staged`) verwirft uncommittete Änderungen **unwiderruflich** [64]. Sie waren nie in Git gespeichert, also kann Git sie auch nicht zurückholen. Im Zweifel vorher `git stash` ausführen.

9.3 `reset` : den Branch verschieben

`git reset <commit>` setzt den aktuellen Branch auf einen anderen Commit [89]. Die drei Modi unterscheiden sich darin, was sie außer dem Branch-Zeiger noch anfassen:



Modus	Branch	Index	Dateien
<code>--soft</code>	✓ auf B	bleibt	bleiben
<code>--mixed</code>	✓ auf B	✓ wie B	bleiben
<code>--hard</code>	✓ auf B	✓ wie B	✓ wie B

Abb. 19: Alle drei Modi entfernen `C` aus dem Branch. `--mixed` ist der Standard, wenn kein Modus angegeben wird.

- **`--soft`** : Der Commit ist weg, seine Änderungen liegen aber vorgemerkt im Index [89]. Ideal, um die letzten Commits neu zusammenzufassen: `git reset --soft HEAD~3 && git commit`.
- **`--mixed`** : Commit weg, Änderungen liegen unvorgemerkt im Arbeitsverzeichnis. Gut zum Aufteilen (siehe Kapitel 8). Ohne Commit-Angabe (`git reset`) nimmt es nur alle Vormerkungen zurück [89].
- **`--hard`** : Commit weg, **und alle uncommitteten Änderungen im Arbeitsverzeichnis ebenfalls**. Committetes lässt sich über das Reflog retten, Uncommittetes nicht [89].

Typische Anwendungen:

```
git reset --soft HEAD~1      # "letzten Commit rückgängig, Änderungen behalten"
git reset --hard origin/main  # lokalen Branch exakt auf den Server-Stand bringen
git reset --hard ORIG_HEAD    # missglückten Merge/Rebase direkt danach rückgängig machen
```

9.4 revert : sicher rückgängig nach dem Push

Ist ein fehlerhafter Commit bereits in `main` und damit bei anderen angekommen, darfst du ihn nicht per `reset` entfernen. Stattdessen erzeugt `git revert` einen **neuen** Commit, der die Änderungen des alten exakt umkehrt [76]. Die Historie bleibt intakt und dokumentiert sogar, dass und warum zurückgenommen wurde.

```
git revert 7b9e4d1            # erzeugt "Revert 'Validierung ergänzen'"
git revert -m 1 4f2a9e0       # einen Merge-Commit zurücknehmen (Elternteil 1 = main
behalten)
git revert --no-commit A^..C   # mehrere Commits in einem einzigen Revert-Commit
```

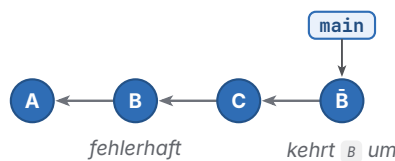


Abb. 20: Revert: $\bar{B}$ hebt die Änderungen von B auf, alle Commits bleiben erhalten.

TIPP

Wird ein per Revert zurückgenommenes Feature später repariert, reicht ein erneuter Merge nicht: Git weiß, dass die ursprünglichen Commits schon enthalten sind [90]. Dann revertierst du den Revert-Commit (`git revert  $\bar{B}$`) und bringst danach die Reparatur ein.

9.5 reflog : das Sicherheitsnetz

Git protokolliert lokal jede Bewegung von `HEAD` und jedes Branches: jeden Commit, Checkout, Reset, Rebase und Merge. Dieses Protokoll ist das *Reflog* [66], und es ist das wichtigste Rettungswerkzeug überhaupt.

```
git reflog
```

```
a90c3d1 (HEAD -> main) HEAD@{0}: reset: moving to HEAD~2
e5f6b72 HEAD@{1}: commit: Tests für Login
7b9e4d1 HEAD@{2}: commit: Validierung ergänzen
3a1f2c0 HEAD@{3}: checkout: moving from feature/login to main
```

Angenommen, `git reset --hard HEAD~2` war ein Versehen. Das Reflog zeigt, dass `HEAD` vorher auf `e5f6b72` stand. Du holst den Stand zurück:

```
git reset --hard HEAD@{1}      # zurück zum Stand vor dem Reset
# oder vorsichtiger: den alten Stand erst als Branch sichern und ansehen
git branch rettung e5f6b72
git log --oneline rettung
```

Das Reflog hilft auch bei gelöschten Branches (den letzten Commit des Branches im Reflog suchen und neu anlegen) und bei Commits aus einem Detached-HEAD-Ausflug [91]. Einträge werden standardmäßig 90 Tage aufbewahrt, für nicht mehr erreichbare Commits 30 Tage [4], [66]. `git reflog show feature/login` zeigt das Protokoll eines einzelnen Branches [66].

MERKE

Das Reflog ist rein lokal. Es wird weder gepusht noch geklont [66], [91]. Auf einem frisch geklonten Rechner gibt es keine Vorgeschichte, auf die man zurückgreifen könnte.

9.6 stash : Änderungen beiseitelegen

Der Stash ist ein Stapel für halbfertige Arbeit, wenn du kurz etwas anderes tun musst [92]:

```
git stash push -m "Login halbfertig" # Änderungen sichern, Arbeitsverzeichnis sauber
git stash push -u -m "inkl. neuer Dateien" # -u nimmt auch untracked Dateien mit
git stash list                        # stash@{0}: On feature/login: Login halbfertig
git stash show -p stash@{0}          # Inhalt als Diff ansehen
git stash pop                        # neuesten Stash anwenden und entfernen
git stash apply stash@{1}            # anwenden, aber behalten
git stash drop stash@{1}             # einzelnen Stash löschen
git stash branch neuer-branch        # Stash in einem neuen Branch wiederherstellen
```

Stashes sind praktisch, geraten aber leicht in Vergessenheit. Für alles, was länger als ein paar Minuten liegen bleibt, ist ein WIP-Commit auf einem eigenen Branch oder ein Worktree (Kapitel 10) die robustere Wahl.

9.7 cherry-pick : einzelne Commits übernehmen

`git cherry-pick` nimmt die Änderung eines einzelnen Commits (seinen Diff zum Elternteil) und wendet sie als **neuen Commit** auf den aktuellen Branch an [93]. Nachricht und Autor bleiben gleich, Elternteil und Hash sind neu. Übernommen wird nur genau diese eine Änderung, nicht die Vorgeschichte des Commits und nicht die übrigen Commits seines Branches.

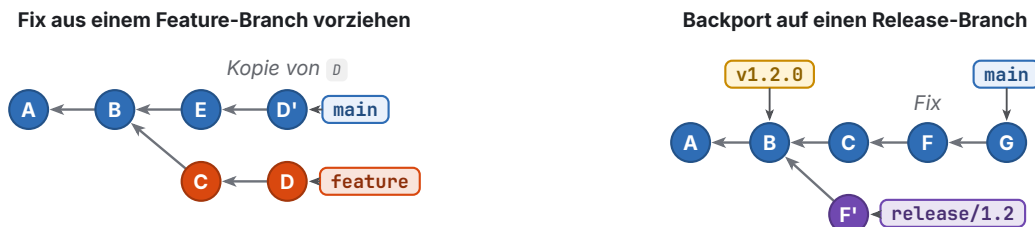


Abb. 21: Links wird ein dringender Fix vorgezogen, obwohl das Feature noch nicht fertig ist. Rechts wird der Fix F aus `main` in die gepflegte Version 1.2 übernommen.

Wann Cherry-Pick das richtige Werkzeug ist

- **Backports:** Du pflegst neben `main` eine ältere Version auf einem Release-Branch (etwa `release/1.2`), und ein Fix aus `main` soll dort ebenfalls ausgeliefert werden, ohne die neuen Funktionen mitzunehmen. Das ist der klassische Anwendungsfall.
- **Dringender Einzelfix:** Ein Commit auf einem unfertigen Feature-Branch behebt einen Fehler, der sofort in `main` gebraucht wird.
- **Rettung:** Commits auf dem falschen Branch gelandet (siehe Notfall-Tabelle am Ende dieses Kapitels).

Die Befehle

```
git switch release/1.2
git cherry-pick 7b9e4d1          # einen Commit übernehmen
git cherry-pick -x 7b9e4d1      # mit Herkunftsvermerk in der Nachricht
git cherry-pick 3a1f2c0 7b9e4d1 # mehrere einzelne Commits, in dieser Reihenfolge
git cherry-pick A^..C           # Bereich von A bis C, A eingeschlossen
git cherry-pick --no-commit A B C # nur Änderungen übernehmen, dann selbst EIN Commit
git commit -m "CSRF-Fixes zurückportieren"
git cherry-pick -m 1 4f2a9e0     # Merge-Commit: Änderungen relativ zum ersten Elternteil
```

`-x` hängt an die Nachricht die Zeile (cherry picked from commit 7b9e4d1...) an [93]. Das ist bei Backports auf öffentliche Branches sehr nützlich, weil man später nachvollziehen kann, woher ein Fix stammt. Bei der Rettung privater Commits, deren Original ohnehin verschwindet, verweist der Vermerk dagegen ins Leere und kann entfallen [93].

Bei Bereichen ist die Schreibweise wichtig: `A^..C` bedeutet „alle Commits bis `C`, die nicht in `A` enthalten sind“, schließt `A` selbst also **aus**. Soll `A` dabei sein, lautet der Bereich `A^..C` [13], [93]. Einen Merge-Commit kann Git nicht ohne Weiteres übernehmen, weil unklar ist, gegenüber welchem Elternteil die

Änderung gemeint ist. `-m 1` wählt das erste Elternteil, also „alles, was dieser Merge in den Ziel-Branch gebracht hat“ [93].

Konflikte beim Cherry-Pick

Weil der Diff auf einer anderen Basis angewendet wird als der, auf der er entstanden ist, kann es Konflikte geben, besonders bei Backports auf ältere Versionen. Die Lösung funktioniert wie beim Merge (Kapitel 6):

```
# Konfliktdateien bearbeiten, dann:
git add src/auth.py
git cherry-pick --continue      # abschließen bzw. mit dem nächsten Commit weiter
git cherry-pick --skip         # diesen Commit des Bereichs auslassen
git cherry-pick --abort         # alles zurück zum Zustand vor dem Cherry-Pick
```

Anders als beim Rebase sind `--ours` und `--theirs` hier **nicht** vertauscht: „Ours“ ist der Branch, auf dem du stehst, „theirs“ der übernommene Commit [29]. Meldet Git beim Übernehmen *The previous cherry-pick is now empty*, ist die Änderung auf dem Ziel-Branch bereits vorhanden. Dann ist `--skip` richtig [93].

Wenn der Quell-Branch später doch gemergt wird

Nach einem Cherry-Pick existiert dieselbe Änderung als zwei verschiedene Commits (`D` und `D'`). Das ist meist harmlos, sollte man aber kennen:

- **Rebase** erkennt, dass eine inhaltsgleiche Änderung schon im Ziel liegt, und lässt den Commit weg [29]. Git meldet das als *skipped previously applied commit*.
- **Merge** führt beide Seiten zusammen. Da beide dieselbe Änderung enthalten, gibt es in der Regel keinen Konflikt. Wurde die betroffene Stelle danach aber auf einer Seite weiter verändert, kann doch einer entstehen.
- **Squash-Merge** eines PRs zeigt die bereits übernommene Änderung nicht mehr im Diff, weil sie im Ziel schon vorhanden ist.

Prüfen, was schon übernommen wurde

Bei Release-Branches stellt sich regelmäßig die Frage, welche Fixes aus `main` dort noch fehlen. Git vergleicht dafür nicht Hashes, sondern den Inhalt der Änderungen [94]:

```
git cherry -v release/1.2 main
```

```
- 7b9e4d1 CSRF-Token bei Logout prüfen
+ 3c55a0e Export für große Dateien beschleunigen
+ 91fe2d4 Neue Berichtsansicht
```

`-` bedeutet: Eine inhaltsgleiche Änderung ist in `release/1.2` schon vorhanden. `+` bedeutet: fehlt noch [94]. Dasselbe zeigt `git log --oneline --cherry-mark release/1.2...main`, das übernommene Commits mit `=` markiert [54]. Musste beim Cherry-Pick ein Konflikt gelöst werden, ist die Änderung nicht mehr exakt gleich und erscheint weiterhin als fehlend. Hier hilft der `-x`-Vermerk: `git log release/1.2 --grep="cherry picked from commit 7b9e4d1"` findet den Backport trotzdem.

Mit Pull Request statt direkt

Auch Backports sollten durch die CI laufen. Statt direkt auf den Release-Branch zu committen, legst du einen eigenen Branch an und öffnest in Gitea einen PR gegen `release/1.2`:

```
git fetch
git switch -c backport/1.2-csrf origin/release/1.2
git cherry-pick -x 7b9e4d1
git push                                # PR in Gitea mit Ziel-Branch release/1.2
```

ACHTUNG

Cherry-Pick ist ein Werkzeug für Einzelfälle. Werden Branches regelmäßig per Cherry-Pick abgeglichen, entstehen viele doppelte Commits und man verliert den Überblick, was wo enthalten ist. Wer von vornherein weiß, dass ein Fix in mehrere Versionen gehört, legt den Fix-Branch besser vom ältesten betroffenen Stand an (etwa von `release/1.2`) und mergt ihn dann in `release/1.2` **und** in `main`. So existiert die Änderung nur als ein einziger Commit.

9.8 `clean` : unversionierte Dateien löschen

```
git clean -n      # Probelauf: zeigt nur, was gelöscht würde
git clean -fd     # unversionierte Dateien und Ordner löschen
git clean -fdx    # zusätzlich ignorierte Dateien (Build-Ordner, .venv, node_modules)
git clean -i      # interaktiv auswählen
```

`git clean` löscht Dateien, die Git nie gesehen hat, und kann sie deshalb auch nicht wiederherstellen [95]. Immer zuerst mit `-n` prüfen.

9.9 Notfall-Tabelle

Ich habe ...	Lösung
... auf <code>main</code> committet statt auf einem Feature-Branch (noch nicht gepusht)	<code>git switch -c feature/x</code> (nimmt die Commits mit), dann <code>git switch main</code> und <code>git reset --hard origin/main</code> .
... auf dem falschen Branch committet	Auf dem richtigen Branch <code>git cherry-pick <hash></code> , auf dem falschen <code>git reset --hard HEAD~1</code> .
... mit <code>reset --hard</code> Commits verloren	<code>git reflog</code> , dann <code>git reset --hard HEAD@{n}</code> oder <code>git branch rettung <hash></code> .
... einen Branch mit ungemergten Commits gelöscht	Git nennt beim Löschen den letzten Hash (was <code>3a1f2c0</code>), sonst im Reflog suchen: <code>git branch name 3a1f2c0</code> .
... einen Merge gemacht, der schiefging (nicht gepusht)	<code>git reset --hard ORIG_HEAD</code> .
... einen fehlerhaften Merge nach <code>main</code> gepusht	<code>git revert -m 1 <merge-hash></code> , dann pushen.
... mitten in einem Merge/Rebase den Überblick verloren	<code>git merge --abort</code> bzw. <code>git rebase --abort</code> .
... ein Passwort committet und gepusht	Sofort das Passwort austauschen, dann ggf. <code>git filter-repo</code> (Kapitel 8).
... eine riesige Datei committet (noch nicht gepusht)	<code>git rm --cached datei</code> , in <code>.gitignore</code> eintragen, <code>git commit --amend</code> .
... uncommittete Änderungen mit <code>restore</code> oder <code>reset --hard</code> verloren	Git kann nicht helfen. Eventuell hat der Editor eine lokale Historie (VS Code: <i>Timeline</i> [96]) oder Time Machine eine Kopie.

10 Worktrees: mehrere Branches gleichzeitig

Ein Repository hat normalerweise genau ein Arbeitsverzeichnis, und darin ist genau ein Branch ausgecheckt. Das wird zum Engpass, sobald du mehrere Dinge parallel tun willst. Worktrees lösen dieses Problem elegant und sind, einmal verstanden, eines der nützlichsten und am wenigsten bekannten Werkzeuge von Git.

10.1 Das Problem

Du bist mitten in einem Feature, der Code kompiliert gerade nicht, Dateien sind halb umgebaut. Da kommt eine dringende Anfrage: ein Hotfix auf `main`, oder ein Kollege bittet um ein Review seines Pull Requests, den du lokal ausprobieren willst. Die klassischen Auswege haben alle Nachteile:

- **Stash:** Funktioniert, aber Build-Artefakte, virtuelle Umgebungen und geöffnete Editor-Tabs passen danach nicht mehr zum Code. Und der Stash gerät leicht in Vergessenheit.
- **WIP-Commit:** Ebenfalls möglich, muss aber später wieder aufgeräumt werden, und der Branch-Wechsel ändert trotzdem alle Dateien unter dem laufenden Editor.
- **Zweiter Klon:** Ein komplett unabhängiges Repository mit doppeltem Speicherbedarf, eigenen Remotes und eigenen Branches, die erst über den Server synchronisiert werden müssen.

10.2 Das Konzept

Ein Worktree ist ein **zusätzliches Arbeitsverzeichnis, das an dasselbe Repository angeschlossen ist**. Alle Worktrees teilen sich eine gemeinsame Objektdatenbank und dieselben Branches. Standardmäßig teilen sie auch die Repository-Konfiguration; bei Bedarf kann Git einzelne Werte pro Worktree speichern. Jeder Worktree hat seinen eigenen ausgecheckten Branch, seinen eigenen Index und seine eigenen Dateien [5], [97].

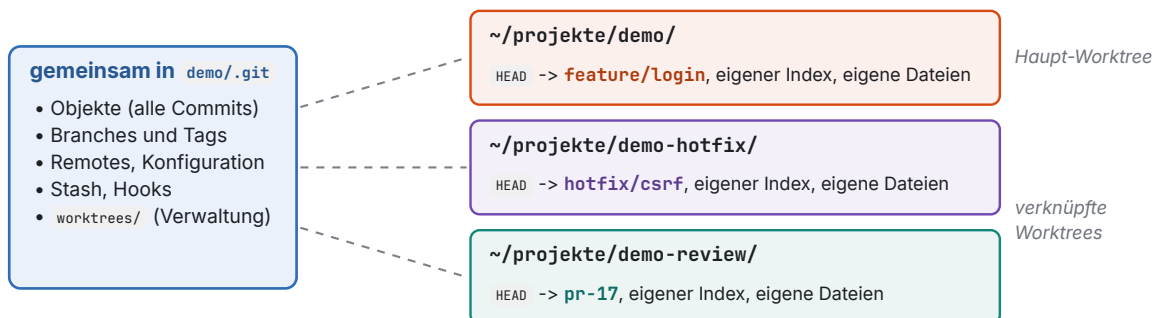


Abb. 22: Ein Repository, drei Arbeitsverzeichnisse. Ein Commit in einem Worktree ist sofort in allen anderen sichtbar.

In einem verknüpften Worktree ist `.git` kein Verzeichnis, sondern eine kleine Textdatei mit dem Verweis auf das Hauptrepository (`gitdir: /Users/.../demo/.git/worktrees/demo-hotfix`) [5], [98]. Deshalb kostet ein zusätzlicher Worktree nur den Platz der ausgecheckten Dateien, nicht den der gesamten Historie.

Gemeinsam für alle Worktrees	Pro Worktree getrennt
Commits und Objektdatenbank [98]	ausgecheckter Branch (<code>HEAD</code>) [5]
Branches, Tags, Remote-Tracking-Branches [97]	Index (Staging Area) [5]
Konfiguration (<code>.git/config</code>) und standardmäßig Hooks [97], [98]	Dateien im Arbeitsverzeichnis
Stash, Reflog der Branches [97], [98]	laufender Merge, Rebase oder Cherry-Pick [5]

10.3 Die Befehle

Befehl	Wirkung
<code>git worktree add ../demo-hotfix -b hotfix/csrf main</code>	neuen Worktree mit neuem Branch <code>hotfix/csrf</code> anlegen, abzweigt von <code>main</code> [97]
<code>git worktree add ../demo-login feature/login</code>	Worktree für einen bestehenden Branch. Existiert der Branch nur auf dem Server, legt Git automatisch einen lokalen Tracking-Branch an [97].
<code>git worktree add --detach ../demo-v1 v1.0</code>	Worktree auf einem Tag oder Commit, ohne Branch (etwa zum Vergleichen) [97]
<code>git worktree list</code>	alle Worktrees mit Pfad, Commit und Branch
<code>git worktree remove ../demo-hotfix</code>	Worktree entfernen. Verweigert, wenn dort uncommittete Änderungen liegen (<code>--force</code> erzwingt es) [97].
<code>git worktree prune</code>	Verwaltungsdaten von Worktrees entfernen, deren Ordner manuell gelöscht wurde [97]
<code>git worktree move alt neu</code>	Worktree-Ordner verschieben
<code>git worktree lock pfad --reason "..."</code>	vor versehentlichem Aufräumen schützen, etwa auf einer externen Platte [97]
<code>git worktree repair</code>	Verknüpfungen reparieren, nachdem Ordner von Hand verschoben wurden [97]

10.4 Beispiel: Hotfix mitten im Feature

```

cd ~/projekte/demo                # hier liegt die halbfertige Arbeit an feature/login
git fetch
git worktree add ../demo-hotfix -b hotfix/csrf origin/main

cd ../demo-hotfix                 # komplett sauberer Stand von main
# ... Fehler beheben, testen ...
git commit -am "CSRF-Token bei Logout prüfen"
git push                          # dann Pull Request in Gitea

cd ../demo                       # zurück, alles wie verlassen
git worktree remove ../demo-hotfix # nach dem Mergen aufräumen
git branch -d hotfix/csrf

```

Das Feature wurde nie angefasst: kein Stash, kein WIP-Commit, keine geänderten Dateien unter dem offenen Editor.

10.5 Beispiel: Pull Request lokal prüfen

Gitea stellt jeden Pull Request unter einer eigenen Referenz bereit (Kapitel 12) [99]. Damit lässt sich ein fremder PR in einem eigenen Worktree ausprobieren, ohne die eigene Arbeit zu unterbrechen:

```

git fetch origin pull/17/head:pr-17    # PR Nr. 17 als lokalen Branch pr-17 holen
git worktree add ../demo-review pr-17
cd ../demo-review && make test

```

10.6 Beispiel: parallele Arbeitsstränge mit KI-Agenten

Worktrees passen hervorragend zu Coding-Agenten wie Claude Code [100]. Zwei Agenten im selben Arbeitsverzeichnis würden sich gegenseitig Dateien überschreiben und Tests des jeweils anderen zer-schießen. In getrennten Worktrees arbeitet jeder auf seinem eigenen Branch, und du führst die Ergebnisse später über Pull Requests zusammen:

```

git worktree add ../demo-agent-a -b agent/export-csv main
git worktree add ../demo-agent-b -b agent/refactor-auth main
# je ein Terminal pro Worktree, dort den Agenten starten

```

10.7 Stolpersteine

- **Ein Branch kann nur in einem Worktree ausgecheckt sein.** Versuchst du es ein zweites Mal, meldet Git *'main' is already used by worktree at ...* [97]. Das ist Absicht, denn zwei Arbeitsverzeichnisse auf demselben Branch würden sich gegenseitig die Grundlage verschieben. Lösung: einen neuen Branch anlegen oder `--detach` verwenden.
- **Unversionierte Dateien werden nicht mitkopiert.** Ein neuer Worktree enthält nur, was in Git ist. `.env`, virtuelle Python-Umgebungen, `node_modules` und Build-Ordner müssen pro Worktree neu angelegt oder kopiert werden. Ein kleines Skript wie `make setup` im Projekt lohnt sich.
- **Ordner nicht einfach löschen.** Wer einen Worktree-Ordner im Finder löscht, hinterlässt Verwaltungsdaten, und der Branch gilt weiterhin als ausgecheckt. `git worktree prune` räumt auf, besser ist gleich `git worktree remove` [97].
- **Stash und Hooks sind gemeinsam.** [97], [98] Ein `git stash pop` im falschen Worktree holt den Stash eines anderen Arbeitsstrangs. Das ist ein weiterer Grund, in Worktrees lieber zu committen.
- **Konfiguration lässt sich bei Bedarf trennen.** Mit `git config extensions.worktreeConfig true` aktivierst du zusätzliche Konfiguration pro Worktree; danach setzt `git config --worktree <schlüssel> <wert>` einen lokalen Wert [18], [97]. Das ist etwa für `core.sparseCheckout` nützlich. Ältere Git-Versionen, die diese Erweiterung nicht kennen, verweigern den Zugriff auf das Repository [97].
- **Submodule** sind pro Worktree auszuchecken: `git submodule update --init --recursive`. Ein Worktree mit Submodulen lässt sich nicht mit `git worktree move` verschieben [97]; notfalls verschiebst du ihn manuell und reparierst die Verknüpfung mit `git worktree repair`.

10.8 Eine sinnvolle Ordnerstruktur

Am übersichtlichsten ist es, verknüpfte Worktrees als Geschwister des Hauptordners anzulegen und den Projektnamen voranzustellen: `demo/`, `demo-hotfix/`, `demo-review/`. So bleiben sie im Finder beieinander und landen nicht versehentlich innerhalb des Hauptprojekts (wo sie als unversionierte Dateien in `git status` auftauchen würden).

	Stash	Worktree	zweiter Klon
Parallel arbeiten	nein	ja	ja
Speicherbedarf	minimal	nur Arbeitsdateien	komplette Historie
Commits sofort überall sichtbar	ja	ja	nein, erst über den Server
Build-Artefakte bleiben erhalten	nein	ja	ja
Geeignet für	kurze Unterbrechungen	Hotfix, Review, Agenten, lange Tests	völlig getrennte Experimente

11 Tags, Referenzen und Werkzeuge

Dieses Kapitel sammelt, was man nicht täglich, aber regelmäßig braucht: Tags für Versionen, die Kurzschreibweisen für Commits und Bereiche sowie die Werkzeuge, mit denen man in der Historie nach Ursachen sucht.

11.1 Tags

Ein Tag ist ein fester Name für einen Commit, typischerweise für eine veröffentlichte Version. Anders als ein Branch bewegt sich ein Tag nie weiter [101].

```
git tag -a v1.2.0 -m "Version 1.2.0" # annotierter Tag (empfohlen)
git tag v1.2.0-rc1                  # leichtgewichtiger Tag (nur ein Name)
git tag -a v1.1.1 -m "Hotfix" 3a1f2c0 # nachträglich an einem älteren Commit
git tag                             # alle Tags auflisten
git show v1.2.0                     # Tag mit Commit anzeigen
git push origin v1.2.0              # einen Tag pushen
git push --follow-tags              # alle annotierten Tags pushen, die zu gepushten Commits
gehören
git tag -d v1.2.0-rc1               # lokal löschen
git push origin --delete v1.2.0-rc1 # auf dem Server löschen
```

Annotierte Tags sind eigene Objekte mit Autor, Datum und Nachricht, **leichtgewichtige** sind nur ein Zeiger [101], [102]. Für Versionen immer annotierte verwenden. `git describe` nutzt sie etwa, um einen Stand wie `v1.2.0-3-g7b9e4d1` zu beschreiben (drei Commits nach v1.2.0) [103].

Tags werden bei einem normalen `git push` **nicht** automatisch übertragen [27], [102]. Das ist eine häufige Stolperfalle, besonders wenn ein Tag eine Pipeline auslösen soll (Kapitel 15). Als Versionsschema hat sich *Semantic Versioning* durchgesetzt: MAJOR.MINOR.PATCH, wobei MAJOR bei inkompatiblen Änderungen, MINOR bei neuen Funktionen und PATCH bei Fehlerbehebungen steigt [104].

Commits und Tags signieren

Eine kryptografische Signatur belegt, dass ein Commit oder Tag mit einem bestimmten privaten Schlüssel erzeugt wurde. Sie verschlüsselt den Inhalt nicht und ersetzt kein Review, erschwert aber das Unterschieben fremder Releases. Git kann dafür neben OpenPGP auch SSH-Schlüssel verwenden [18]. Sinnvoll ist ein eigener Signaturschlüssel statt des Anmeldeschlüssels:

```
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519_git_signing -C "git-signing"
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519_git_signing.pub
git config --global commit.gpgSign true
git config --global tag.gpgSign true
```

Den öffentlichen Schlüssel hinterlegst du in Gitea als Signaturschlüssel. Danach kennzeichnet Gitea passende Commits und annotierte Tags als verifiziert [105]. Lokal benötigt `git verify-commit <hash>` beziehungsweise `git verify-tag <tag>` zusätzlich eine `gpg.ssh.allowedSignersFile`, die vertrauenswürdige Identitäten ihren öffentlichen Schlüsseln zuordnet [18]. Signiere insbesondere Release-Tags; schütze die entsprechenden Tag-Muster zusätzlich serverseitig [106].

11.2 Commits benennen

Fast jeder Befehl akzeptiert statt eines Hashes auch einen relativen Ausdruck [13]:

Ausdruck	Bedeutung
<code>a1b2c3d</code>	Commit über (eindeutigen) Hash-Anfang
<code>HEAD</code>	der aktuelle Commit
<code>HEAD~1</code> , <code>HEAD~3</code>	ein bzw. drei Schritte zurück, immer über das erste Elternteil
<code>HEAD^</code>	das erste Elternteil (gleich <code>HEAD~1</code>)
<code>HEAD^2</code>	das zweite Elternteil, nur bei Merge-Commits sinnvoll
<code>main@{u}</code> , <code>@{u}</code>	der Upstream des Branches (z.B. <code>origin/main</code>)
<code>main@{yesterday}</code>	wo <code>main</code> gestern stand (aus dem Reflog)
<code>HEAD@{2}</code>	die vorletzte Position von <code>HEAD</code> (Reflog)
<code>:/Login</code>	der jüngste Commit, dessen Nachricht „Login“ enthält

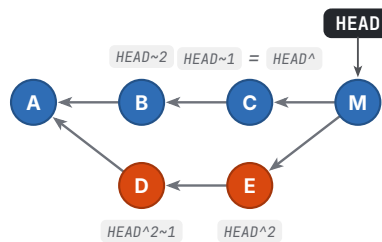


Abb. 23: `~` geht entlang der ersten Eltern zurück, `^2` wählt beim Merge die zweite Linie.

11.3 Bereiche: zwei und drei Punkte

Viele Fragen an die Historie lauten „Was hat der eine Branch, was der andere nicht hat?“. Dafür gibt es zwei Schreibweisen, deren Unterschied man einmal verstanden haben sollte [13]:



nur C, D: was `feature` hat und `main` nicht

C, D, E, F: was jeweils nur eine Seite hat

Abb. 24: Zwei Punkte: Differenz. Drei Punkte: symmetrische Differenz seit der Merge-Basis.

```
git log main..feature      # Commits, die in den PR gehen würden
git log feature..main      # Commits, die main inzwischen neu hat
git log @{u}..            # meine noch nicht gepushten Commits
git diff main...feature    # nur die Änderungen des Features seit dem Abzweigen
```

Achtung, bei `git diff` ist die Bedeutung leicht verschoben: `git diff main...feature` vergleicht die Merge-Basis mit `feature` [52] und zeigt damit genau das, was ein Pull Request in Gitea als Änderungen anzeigt. `git diff main feature` vergleicht dagegen die beiden Enden direkt und enthält auch die Änderungen, die nur in `main` passiert sind.

11.4 Suchen in der Historie

```
git log -S "max_retries"    # Commits, die das Vorkommen dieses Textes verändert haben
git log -G "retr(y|ies)"    # dasselbe mit regulärem Ausdruck
git log --grep="Login" -i   # in Commit-Nachrichten suchen
git grep "TODO" v1.2.0      # im Dateistand eines beliebigen Commits suchen
```

Die Suche mit `-s` (*pickaxe*) ist besonders nützlich für die Frage „Wann ist diese Funktion verschwunden?“. Sie findet genau die Commits, in denen sich die Anzahl der Vorkommen des Textes geändert hat [54].

blame : Wer hat diese Zeile zuletzt geändert?

```
git blame src/login.py           # jede Zeile mit Commit, Autor und Datum
git blame -L 40,60 src/login.py  # nur Zeilen 40 bis 60
git blame -w -C src/login.py     # Leerzeichen ignorieren, verschobenen Code erkennen
```

`blame` beantwortet nicht „wer ist schuld“, sondern „in welchem Zusammenhang entstand diese Zeile“. Mit dem gefundenen Hash führt `git show` zur Commit-Nachricht und damit oft zum Warum. Reine Formatierungs-Commits (etwa nach Einführung eines Formaters) verstellen den Blick. Trägt man ihre Hashes in eine Datei `.git-blame-ignore-revs` ein und setzt `git config blame.ignoreRevsFile .git-blame-ignore-revs`, überspringt `blame` sie [18], [107].

11.5 bisect : den schuldigen Commit finden

Ein Fehler ist da, vor zwei Wochen war er es noch nicht, dazwischen liegen 80 Commits. `git bisect` findet den verursachenden Commit per binärer Suche in etwa sieben Schritten [78]:

```
git bisect start
git bisect bad           # der aktuelle Stand ist fehlerhaft
git bisect good v1.1.0   # dieser Stand war in Ordnung
# Git checkt einen Commit in der Mitte aus. Testen, dann:
git bisect good          # oder: git bisect bad
# ... wiederholen, bis Git meldet: "a1b2c3d is the first bad commit"
git bisect reset         # zurück zum Ausgangspunkt
```

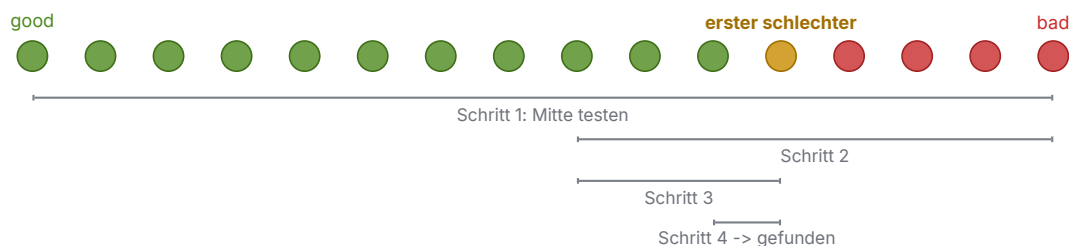


Abb. 25: Jeder Schritt halbiert den Suchbereich. 1000 Commits brauchen höchstens zehn Tests.

Lässt sich der Test automatisieren, erledigt Git die ganze Suche selbst: `git bisect run make test` führt den Befehl an jedem Kandidaten aus und wertet den Rückgabewert aus (0 = gut, 1 bis 127 außer 125 = schlecht, 125 = überspringen) [78]. Das funktioniert umso besser, je kleiner und in sich funktionsfähiger die einzelnen Commits sind. Das ist ein handfester Grund für die sauberen Commits aus Kapitel 3.

11.6 Kleine Helfer

Befehl	Wirkung
<code>git shortlog -sn</code>	Anzahl der Commits pro Autor [108]
<code>git describe --tags</code>	lesbare Versionsbezeichnung für den aktuellen Stand [103]
<code>git archive -o demo.zip HEAD</code>	Projektstand ohne <code>.git</code> als ZIP exportieren [109]
<code>git show HEAD~5:src/app.py</code>	eine Datei in einem alten Stand ausgeben [55]
<code>git log --oneline --first-parent main</code>	nur die Hauptlinie: ein Eintrag pro gemergtem Feature [54]
<code>git count-objects -vH</code>	Größe des Repositories [110]
<code>git maintenance start</code>	regelmäßige Hintergrundoptimierung für große Repositories einrichten [111]

12 Gitea und Pull Requests

Gitea ist ein selbst gehosteter Git-Server mit Weboberfläche, vergleichbar mit GitHub oder GitLab, aber schlank genug für einen kleinen Server oder eine NAS. Neben Repositories bietet es Organisationen und Teams [112], Issues, Pull Requests, eine Paket- und Container-Registry [113] sowie mit *Gitea Actions* ein CI/CD-System, das mit GitHub Actions weitgehend kompatibel ist [114] (Kapitel 13). Die Angaben in diesem Teil beziehen sich auf Gitea 1.27.3 und Gitea Runner 3.5.

ACHTUNG

Für öffentlich erreichbare Instanzen ist mindestens Gitea 1.27.3 wichtig: Diese Patch-Version schließt mehrere Sicherheitslücken, darunter eine Umgehung der Freigabe für Workflow-Läufe aus erstmaligen Fork-Beiträgen [115]. Sicherheitsupdates von Gitea und Runner zeitnah einspielen und Release Notes vor jedem Upgrade lesen.

12.1 Ein lokales Projekt mit Gitea verbinden

Für ein bestehendes lokales Projekt legst du in Gitea zuerst ein **leeres** Repository an (über + -> *Neues Repository*). Wichtig: Dabei keine README, `.gitignore` oder Lizenz erzeugen lassen, sonst haben Server und lokales Projekt zwei unabhängige Anfangscommits und der erste Push schlägt fehl.

```
cd ~/projekte/demo
git remote add origin gitea:team/demo.git # Kurzname aus ~/.ssh/config (Kapitel 2)
git push -u origin main
```

Ist das Repository in Gitea schon mit Inhalt angelegt, klonst du es stattdessen einfach. Organisationen (etwa `team`) bündeln Repositories mehrerer Personen. Rechte vergibst du über Teams in der Organisation, nicht pro Repository und Person [112].

12.2 Der Branch-basierte Arbeitsablauf

Fast alle modernen Teams arbeiten nach demselben einfachen Muster, oft *GitHub Flow* [116] oder *trunk-based development* [117] genannt:

1. `main` ist immer lauffähig und geschützt. Niemand pusht direkt darauf.
2. Jede Änderung beginnt als kurzlebiger Branch von `main`.
3. Über einen **Pull Request** (PR) wird die Änderung zur Integration vorgeschlagen.
4. Die CI prüft automatisch, ein Mensch prüft inhaltlich (Review).
5. Nach dem Mergen wird der Branch gelöscht, `main` wird ausgeliefert.

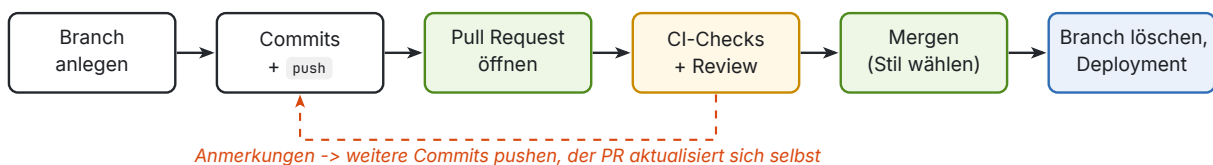


Abb. 26: Lebenszyklus eines Pull Requests.

12.3 Einen Pull Request erstellen

```
git switch main && git pull
git switch -c feature/42-login-sperre
```

```
# ... arbeiten, committen ...
git push # mit push.autoSetupRemote genügt das
```

```
remote:
remote: Create a new pull request for 'feature/42-login-sperre':
remote: https://gitea.example.com/team/demo/pulls/new/feature/42-login-sperre
remote:
```

Gitea gibt beim ersten Push eines Branches direkt den Link zum Anlegen des PRs aus [118] (⌘-Klick im Terminal öffnet ihn). Alternativ findest du im Repository unter *Pull Requests* -> *Neuer Pull Request* die Auswahl von Ziel- und Quell-Branch.

Beim Ausfüllen lohnt sich Sorgfalt, denn Titel und Beschreibung sind die Dokumentation der Änderung:

- **Titel** wie eine gute Commit-Titelzeile. Beim Squash-Merge wird er zur Commit-Nachricht.
- **Beschreibung:** Was ändert sich, warum, wie kann man es testen, was ist bewusst nicht enthalten.
- **Verknüpfung mit Issues:** Schlüsselwörter wie `Closes #42`, `Fixes #42` oder `Resolves #42` in der Beschreibung schließen das Issue automatisch, sobald der PR gemergt wird [119].
- **Noch nicht fertig?** Beginnt der Titel mit `WIP:` oder `[WIP]`, markiert Gitea den PR als *Work in Progress* und verhindert das Mergen [120]. So lässt sich früh Feedback einholen.
- Reviewer, Labels und Meilensteine setzt du in der rechten Seitenleiste.

TIPP

Eine Vorlage für PR-Beschreibungen legst du als `.gitea/PULL_REQUEST_TEMPLATE.md` ins Repository. Gitea füllt das Beschreibungsfeld jedes neuen PRs damit vor [121], etwa mit den Abschnitten *Was*, *Warum*, *Testen* und einer kleinen Checkliste.

12.4 Review

Im Tab *Geänderte Dateien* zeigt Gitea den Diff zwischen Merge-Basis und PR-Branch, also genau `git diff main...feature` aus Kapitel 11 [52]. Über das Plus-Symbol neben einer Zeile hinterlässt du einen Kommentar. Statt jeden Kommentar einzeln abzuschicken, sammelst du sie mit *Review beginnen* und schließt das Review mit einer Entscheidung ab:

Entscheidung	Bedeutung
<i>Kommentieren</i>	Feedback ohne Urteil.
<i>Zustimmen (Approve)</i>	Aus Sicht des Reviewers mergebar. Zählt für die geforderte Anzahl an Freigaben [83].
<i>Änderungen anfordern</i>	Blockiert den Merge, wenn der Branch-Schutz das so vorsieht (siehe unten) [83].

Erledigte Diskussionen markiert man als *aufgelöst*. Bei größeren PRs lohnt es sich, im Diff-Menü die Ansicht auf einzelne Commits umzuschalten oder Leerzeichenänderungen auszublenden.

12.5 Einen Pull Request aktualisieren

Ein PR ist keine Kopie, sondern ein Verweis auf den Branch. Jeder weitere Push auf den Branch aktualisiert den PR automatisch, und die CI läuft erneut [122]. Für Review-Anmerkungen gibt es zwei Stile:

- **Einfach weitere Commits pushen.** Übersichtlich für Reviewer, die Historie wird beim Squash-Merge ohnehin zusammengefasst.
- **Fixup-Commits pushen** (Kapitel 7) und vor dem Mergen mit `git rebase -i --autosquash main` einfallen [29], dann `git push --force-with-lease` [27]. Empfehlenswert, wenn per Rebase gemergt wird und die Einzelcommits erhalten bleiben sollen.

Ist `main` inzwischen weitergelaufen, zeigt Gitea an, dass der Branch veraltet ist, und bietet *Branch aktualisieren* an, per Merge oder (falls erlaubt) per Rebase [123]. Lokal erreichst du dasselbe mit `git pull --rebase origin main` [26] und anschließendem `git push --force-with-lease`.

12.6 Mergen: die Gitea-Merge-Stile

Beim Mergen bietet Gitea mehrere Stile an [77]. Sie entsprechen genau den Strategien aus Kapitel 6. Die Tabelle nennt die englischen Originalbeschriftungen, in der deutschen Oberfläche sind sie sinngemäß übersetzt:

Gitea-Schaltfläche	entspricht	Ergebnis
Create merge commit	<code>git merge --no-ff</code> [30]	① Merge-Commit, alle Einzelcommits bleiben unverändert
Rebase then fast-forward	<code>rebase + merge --ff-only</code>	③ lineare Historie mit allen (neu geschriebenen) Einzelcommits
Rebase then create merge commit	<code>rebase + merge --no-ff</code>	④ semi-linear: Einzelcommits plus Klammer
Create squash commit	<code>merge --squash</code> [30]	② ein Commit pro PR, Nachricht im Dialog editierbar
Fast-forward only	<code>merge --ff-only</code> [30]	nur möglich, wenn der Branch schon auf <code>main</code> aufsetzt
Manually merged	(lokal erledigt)	markiert den PR als gemergt, wenn du lokal gemergt und gepusht hast

In den Repository-Einstellungen unter *Pull Requests* legst du fest, welche Stile erlaubt sind, welcher vorausgewählt ist und ob der Branch nach dem Mergen standardmäßig gelöscht wird [73]. Für die meisten Projekte empfiehlt sich: Squash als Standard, Rebase + Merge-Commit zusätzlich erlaubt, Merge-Commit und Fast-Forward-only abgeschaltet, Branch nach dem Mergen löschen.

Sind die Checks noch nicht durchgelaufen, kannst du den PR über die Option *Automatisch mergen, wenn alle Checks erfolgreich sind* vormerken. Gitea mergt dann selbstständig, sobald die Pipeline grün ist [77].

Nach dem Mergen räumst du lokal auf:

```
git switch main
git pull
git branch -D feature/42-login-sperre # -D, weil Squash neue Commits erzeugt (Kapitel 4)
```

12.7 Branch-Schutz

Die wichtigste Einstellung für ein geordnetes Repository findest du unter *Einstellungen -> Branches -> Branch-Schutzregel hinzufügen*. Eine Regel gilt für ein Namensmuster, meist `main`:

Option	Empfehlung und Wirkung
Push	<i>Push deaktivieren</i> : Änderungen an <code>main</code> nur über PRs [83]. Wer allein arbeitet, kann sich selbst in eine Ausnahmeliste eintragen, verliert dann aber die Garantie, dass alles durch die CI lief.
Force-Push	deaktiviert lassen, damit die Historie von <code>main</code> nie umgeschrieben wird [83]
Statusprüfungen	aktivieren und Muster wie <code>ci / *</code> eintragen: Merge nur, wenn die passenden CI-Jobs erfolgreich waren [83] (Kapitel 15)
Erforderliche Freigaben	z.B. 1: Mindestanzahl an Zustimmungen [83]. Alleine sinnlos, im Team sehr wertvoll.
Veraltete Freigaben verwerfen	Neue Commits nach einer Freigabe erfordern eine erneute Freigabe [83].
Bei abgelehnten Reviews blockieren	Ein <i>Änderungen angefordert</i> verhindert den Merge [83].
Bei veraltetem Branch blockieren	Der PR muss erst auf den aktuellen Stand von <code>main</code> gebracht werden. Garantiert, dass die CI den tatsächlichen Endzustand geprüft hat [83].
Geschützte Dateimuster	z.B. <code>.gitea/workflows/*</code> : Änderungen an diesen Dateien nur durch berechtigte Personen [83].

Schütze zusätzlich Release-Tags, etwa das Muster `v*`, und beschränke das Erzeugen und Löschen auf Release-Verantwortliche [124]. Ein Tag ist sonst nur ein frei verschiebbarer Name [101] und kann —

wenn er ein Produktions-Deployment auslöst — den normalen PR-Weg umgehen. Kapitel 15 prüft deshalb zusätzlich technisch, dass ein Release-Tag auf einen Commit aus `main` zeigt.

12.8 Pull Requests lokal auschecken

Gitea stellt jeden PR zusätzlich unter der Referenz `refs/pull/<Nummer>/head` bereit [125], auch wenn der Branch aus einem Fork stammt. Damit holst du jeden PR mit einem Befehl [28]:

```
git fetch origin pull/17/head:pr-17
git switch pr-17 # oder als Worktree, siehe Kapitel 10
```

Wer das oft macht, kann Git anweisen, alle PRs bei jedem Fetch mitzuholen. Danach heißen sie `origin/pr/17` usw. [126]:

```
git config --add remote.origin.fetch "+refs/pull/*/head:refs/remotes/origin/pr/*"
```

13 Gitea Actions: die `ci.yml` verstehen

CI/CD steht für *Continuous Integration* und *Continuous Delivery/Deployment*: Bei jedem Push prüft eine Maschine automatisch, ob der Code gebaut werden kann und die Tests bestehen (CI), und liefert ihn bei Erfolg aus (CD). In Gitea übernimmt das *Gitea Actions*. Die Workflows werden als YAML-Dateien im Repository beschrieben, und die Syntax ist bewusst mit GitHub Actions kompatibel [114]. Viele Anleitungen und fertige Bausteine aus der GitHub-Welt funktionieren deshalb auch hier — Unterschiede müssen aber geprüft werden.

13.1 Wie Gitea Actions funktioniert

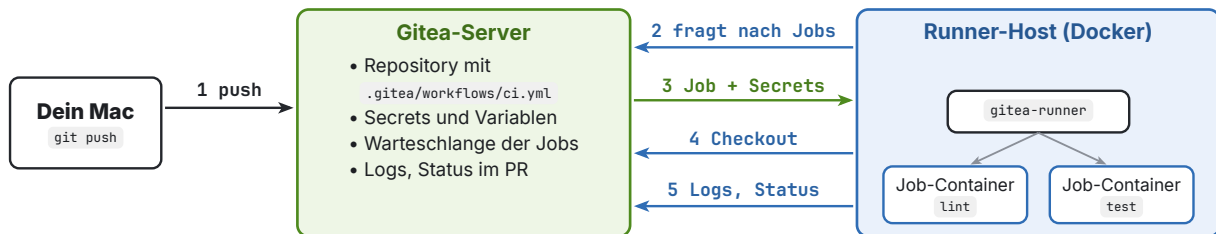


Abb. 27: Ablauf eines Workflow-Laufs. Der Runner baut alle Verbindungen selbst auf, Gitea muss ihn nicht erreichen können.

1. Du pushst einen Commit. Gitea liest die Workflow-Dateien des Commits und prüft, welche davon auf das Ereignis reagieren (etwa *push auf main* oder *Pull Request geöffnet*). Für jeden passenden Job legt es einen Eintrag in einer Warteschlange an.
2. Ein **Runner** fragt regelmäßig bei Gitea nach Arbeit [127]. Der Runner ist ein eigenes Programm (`gitea-runner`, früher `act_runner` [128]), das auf einem beliebigen Rechner mit Docker läuft [127] (Kapitel 14).
3. Passt ein Job zu den *Labels* des Runners, bekommt der Runner die Job-Beschreibung samt der benötigten Secrets [129].
4. Bei einem Docker-Label startet der Runner für den Job einen frischen Container, der den Code aus Gitea auscheckt und die Schritte ausführt [129], [130]. Ein `host`-Label führt die Schritte dagegen direkt auf dem Runner-Rechner aus und bietet deutlich weniger Isolation [130].
5. Logs und Ergebnis gehen zurück an Gitea und erscheinen im Tab *Actions* sowie als Häkchen oder Kreuz am Commit und im Pull Request.

MERKE

Weil der Runner die Verbindung zu Gitea aufbaut und nicht umgekehrt, kann er hinter einem Router, in einem Heimnetz oder in einem VPN stehen. Er braucht nur ausgehenden Zugriff auf Gitea (und auf das Internet, wenn Actions oder Docker-Images heruntergeladen werden) [127].

Voraussetzungen

- Actions ist in aktuellen Gitea-Versionen serverweit aktiv. Falls nicht, muss in der `app.ini` im Abschnitt `[actions]` der Wert `ENABLED = true` stehen [73], [131].
- Im Repository muss die Einheit *Actions* eingeschaltet sein (*Einstellungen -> Erweiterte Einstellungen*) [131]. Dann erscheint der Tab *Actions*.
- Mindestens ein Runner muss registriert und online sein, dessen Labels zu `runs-on` passen.
- Workflows liegen als `.yml`- oder `.yaml`-Dateien im Ordner `.gitea/workflows/`. Existiert dieser Ordner nicht, liest Gitea ersatzweise `.github/workflows/` [73]. Der Dateiname ist frei wählbar, `ci.yml` ist nur eine Konvention.

13.2 YAML in zehn Minuten

Workflow-Dateien sind in YAML geschrieben, einem Format für verschachtelte Daten, dessen Struktur allein durch **Einrückung** entsteht [132]. Fast alle Fehler in Workflows sind Einrückungsfehler.

`yml-grundlagen.yml`

```
# Kommentar
name: ci                # Zuordnung: Schlüssel, Doppelpunkt, Leerzeichen, Wert
timeout: 10             # Zahl
aktiv: true             # Wahrheitswert

branches: [main, develop] # Liste in Kurzform
labels:                 # Liste in Langform: jedes Element mit "- "
  - ubuntu-latest
  - docker

job:                    # verschachtelte Zuordnung: alles darunter um 2 Leerzeichen
  eingerückt
    name: Test
    steps:              # Liste von Zuordnungen
      - name: Erster Schritt
        run: echo "Hallo"
      - name: Zweiter Schritt
        run: |           # "|" = mehrzeiliger Text, Zeilenumbrüche bleiben erhalten
          echo "Zeile 1"
          echo "Zeile 2"
version: "3.10"         # Anführungszeichen, sonst würde daraus die Zahl 3.1
```

Die wichtigsten Regeln: Einrückung **nur mit Leerzeichen**, nie mit Tabulatoren [132]. Alles auf derselben Einrückungsebene gehört zusammen. Ein `-` leitet ein Listenelement ein, und alles, was zu diesem Element gehört, steht um zwei Leerzeichen weiter eingerückt als das `-`. Im Zweifel Werte in Anführungszeichen setzen, besonders Versionsnummern, Zeiten und alles mit `:` oder `#`.

TIPP

VS Code mit der Erweiterung **YAML** von Red Hat markiert Einrückungsfehler sofort [133]. Mit dem Schema für GitHub-Workflows (das für Gitea weitgehend passt) gibt es sogar Autovervollständigung für Schlüssel wie `runs-on` und `steps`.

13.3 Der erste Workflow

`.gitea/workflows/ci.yml`

```
name: ci                # Name des Workflows, erscheint in Gitea und in
                        # Statusprüfungen

on:                    # WANN läuft der Workflow?
  push:
    branches: [main]   # bei jedem Push auf main
  pull_request:        # bei jedem PR (Öffnen und jedem neuen Push darauf)

jobs:                 # WAS läuft? Eine Sammlung von Jobs
  test:               # frei gewählte Job-ID
    runs-on: ubuntu-latest # WO läuft er? Muss zu einem Runner-Label passen
    steps:            # die Schritte, der Reihe nach
      - name: Code auschecken
        uses: actions/checkout@v4 # fertige Action: klonet das Repository in den Container
      - name: Tests ausführen
        run: make test             # Shell-Befehl; Rückgabewert ungleich 0 = Fehler
```

Zeile für Zeile passiert Folgendes: Bei einem Push auf `main` oder einem PR startet Gitea den Workflow `ci`. Er besteht aus einem Job `test`, der auf einem Runner mit dem Label `ubuntu-latest` in einem frischen Container läuft. Der erste Schritt holt den Code, der zweite führt die Tests aus. Schlägt ein Schritt

fehl, bricht der Job ab und wird rot markiert. Im Pull Request erscheint die Prüfung unter dem Namen `ci / test (pull_request)`, also *Workflow / Job (Ereignis)* [134].

ACHTUNG

Ohne `actions/checkout` ist der Container leer! Der Code wird nicht automatisch bereitgestellt [135]. Der Checkout-Schritt ist deshalb in fast jedem Job der erste.

13.4 Die Anatomie: Workflow, Jobs, Steps

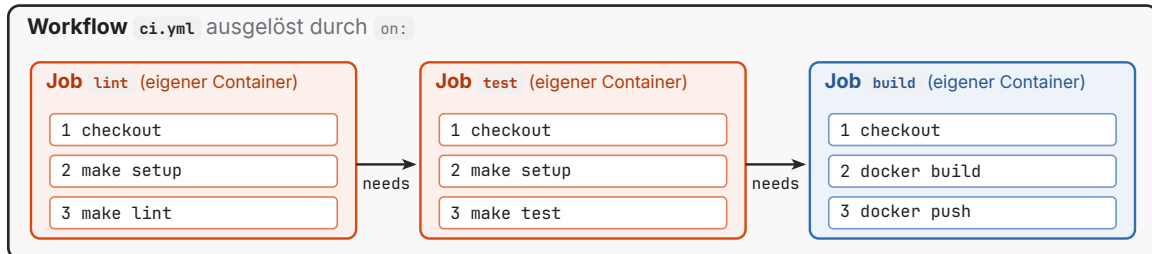


Abb. 28: Jobs laufen in getrennten Containern und ohne `needs` parallel. Steps eines Jobs laufen nacheinander im selben Container.

Diese Struktur ist das Wichtigste am ganzen Kapitel:

- Ein **Workflow** ist eine Datei. Er wird durch Ereignisse ausgelöst und enthält einen oder mehrere Jobs [136].
- Ein **Job** läuft auf einem Runner in einem **eigenen, frischen Container**. Jobs laufen standardmäßig **parallel**. Mit `needs` legst du Abhängigkeiten fest [136]. Jobs teilen **keine Dateien** miteinander, dafür gibt es Artefakte, Caches und Job-Ausgaben [137].
- Ein **Step** ist entweder ein Shell-Befehl (`run`) oder eine wiederverwendbare **Action** (`uses`). Steps eines Jobs laufen nacheinander im selben Container und teilen sich das Arbeitsverzeichnis. Schlägt ein Step fehl, werden die folgenden übersprungen [136].

13.5 Auslöser: `on`

Auslöser	Beispiel und Bedeutung
<code>push</code>	<code>branches: [main], tags: ['v*'], paths: ['src/**'], paths-ignore: ['docs/**']</code> . Filter werden kombiniert: Push auf passenden Branch und passende Pfade [122].
<code>pull_request</code>	<code>types: [opened, synchronize, reopened]</code> (Standard). <code>branches: [main]</code> filtert nach dem Ziel -Branch des PRs [122].
<code>workflow_dispatch</code>	Manuell starten über eine Schaltfläche im Tab <i>Actions</i> , optional mit Eingaben (<code>inputs</code>) [122].
<code>schedule</code>	Zeitgesteuert: <code>- cron: '0 3 * * 1'</code> (montags 3 Uhr, UTC) [122]. Gitea versteht zusätzlich <code>@daily</code> , <code>@weekly</code> usw. [114]
<code>release</code>	beim Veröffentlichen eines Releases in Gitea [122]

```
on:
  push:
    branches: [main]
    tags: ['v*']           # Tags wie v1.2.0 lösen ebenfalls aus
    paths-ignore: ['**/*.md'] # reine Doku-Änderungen ignorieren
  pull_request:
  workflow_dispatch:
    inputs:
      ziel:
        description: 'Ziellumgebung'
        default: 'staging'
```

13.6 Jobs im Detail

```
jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 20 # Job nach 20 Minuten abbrechen
    strategy:
      matrix: # Job für jede Kombination einmal starten
        version: ["3.12", "3.13"]
    services: # Hilfscontainer für die Dauer des Jobs
      db:
        image: postgres:17
        env:
          POSTGRES_PASSWORD: test
    env:
      DATABASE_URL: postgres://postgres:test@db:5432/postgres
    steps:
      - uses: actions/checkout@v4
      - run: make test VERSION=${{ matrix.version }}

  deploy:
    needs: [test] # erst nach erfolgreichem test
    if: github.ref == 'refs/heads/main' # nur auf main, nicht in PRs
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deployment ..."
```

- **runs-on** wählt den Runner über sein Label. Gitea unterstützt hier nur einen einfachen Namen oder eine einfache Liste, keine Gruppen-Syntax [114].
- **needs** baut aus Jobs einen Ablaufgraphen. Schlägt ein benötigter Job fehl, werden die abhängigen übersprungen [136].
- **if** entscheidet, ob ein Job oder Step überhaupt läuft. Innerhalb von **if** darf man die `{{ }}`-Klammern weglassen [136].
- **strategy.matrix** vervielfacht einen Job, etwa für mehrere Sprachversionen [136].
- **services** startet Hilfscontainer (Datenbank, Redis). Der Job erreicht sie über ihren Namen als Hostnamen, hier `db` [136].
- **container** (nicht gezeigt) lässt den Job statt im Standard-Image in einem eigenen Image laufen, etwa `container: python:3.13-slim` [136].

13.7 Steps im Detail

```
steps:
  - name: Code auschecken
    uses: actions/checkout@v4 # Action in Version v4
    with: # Eingaben für die Action
      fetch-depth: 0 # komplette Historie statt nur des letzten Commits

  - name: Version bestimmen
    id: version # ID, um später auf Ausgaben zuzugreifen
    run: echo "tag=$(git describe --tags --always)" >> "$GITHUB_OUTPUT"

  - name: Bauen
    working-directory: ./app # in einem Unterordner ausführen
    env:
      VERSION: ${{ steps.version.outputs.tag }}
    run: |
      echo "Baue Version $VERSION"
      make build

  - name: Aufräumen
    if: always() # auch ausführen, wenn vorher etwas fehlschlug
    run: make clean
```

`run`-Schritte werden mit `bash -e` ausgeführt: Der erste fehlschlagende Befehl beendet den Schritt mit Fehler [136]. Ausgaben eines Schrittes schreibt man in die Datei, auf die `$GITHUB_OUTPUT` zeigt, im Format `name=wert`. Folgende Schritte lesen sie über `steps.<id>.outputs.<name>` [138]. Auf dieselbe Weise setzt `echo "NAME=wert" >> "$GITHUB_ENV"` eine Umgebungsvariable für alle folgenden Schritte [138].

TIPP

Actions werden über `uses: besitzer/name@version` eingebunden und standardmäßig von `github.com` geladen [114]. Die Beispiele verwenden Hauptversions-Tags, damit sie lesbar bleiben. In produktiven Workflows pinnt man fremde Actions nach Prüfung auf einen vollen Commit-Hash (`actions/checkout@<40 Zeichen>`) [139] und lässt Abhängigkeitsupdates bewusst prüfen. Gitea erlaubt außerdem absolute URLs, etwa `uses: https://gitea.example.com/team/meine-action@v1`, womit man geprüfte Actions auf dem eigenen Server spiegeln kann [114].

13.8 Ausdrücke und Kontexte

Alles zwischen `${{` und `}}` ist ein Ausdruck, den Gitea vor dem Ausführen auswertet [140]. Die Daten kommen aus *Kontexten* [141]:

Kontext	Inhalt (Beispiele)
<code>gitea / github</code>	Metadaten des Laufs, beide Namen funktionieren [142]: <code>gitea.ref (refs/heads/main)</code> , <code>gitea.ref_name (main)</code> , <code>gitea.sha</code> , <code>gitea.event_name (push, pull_request)</code> , <code>gitea.actor</code> , <code>gitea.repository (team/demo)</code> , <code>gitea.server_url</code> , <code>gitea.run_number</code>
<code>secrets</code>	geheime Werte, z.B. <code>secrets.SONAR_TOKEN</code> , dazu automatisch <code>secrets.GITEA_TOKEN</code> [143] (Kapitel 14)
<code>vars</code>	nicht geheime Konfigurationsvariablen, z.B. <code>vars.DEPLOY_HOST</code> [144]
<code>env</code>	Umgebungsvariablen aus <code>env: -</code> Blöcken
<code>steps</code>	Ausgaben und Ergebnis früherer Schritte: <code>steps.version.outputs.tag</code> , <code>steps.x.outcome</code>
<code>needs</code>	Ausgaben und Ergebnis benötigter Jobs: <code>needs.build.outputs.image</code> , <code>needs.test.result</code>
<code>matrix</code>	aktuelle Matrix-Werte: <code>matrix.version</code>
<code>runner</code>	Informationen über den Runner: <code>runner.os</code> , <code>runner.temp</code>

Dazu gibt es Operatoren (`==`, `!=`, `&&`, `||`, `!`) und Funktionen wie `contains(...)`, `startsWith(gitea.ref, 'refs/tags/v')` und `format(...)` [140]. Für Bedingungen auf den bisherigen Verlauf dient `always()` [140]. Für die übrigen Statusfunktionen nennt die Gitea-Dokumentation Einschränkungen. Ungewöhnliche Bedingungen testest du am besten einmal gezielt mit einem kleinen Workflow und `workflow_dispatch`.

Zusätzlich stehen in jedem Schritt Umgebungsvariablen bereit, darunter `GITHUB_SHA`, `GITHUB_REF`, `GITHUB_REF_NAME`, `GITHUB_WORKSPACE` (Arbeitsverzeichnis mit dem ausgecheckten Code) [145], `CI=true` und `GITEA_ACTIONS=true`, an dem ein Skript erkennen kann, dass es in Gitea läuft [144].

13.9 Daten zwischen Jobs: Artefakte und Caches

Weil jeder Job in einem neuen Container startet, muss alles, was ein späterer Job braucht, ausdrücklich weitergereicht werden:

```
- uses: actions/upload-artifact@v4      # im Job "build"
  with:
    name: paket
    path: dist/
    retention-days: 7
# ...
- uses: actions/download-artifact@v4   # im Job "deploy" (needs: build)
  with:
    name: paket
```

Artefakte erscheinen auch zum Herunterladen in der Oberfläche des Laufs, ideal für Testberichte. **Caches** (`actions/cache@v4`) sind dagegen dafür da, Abhängigkeiten wie Paket-Downloads zwischen Läufen wiederzuverwenden [146]. Der Runner bringt dafür einen eigenen Cache-Server mit [147]. Gitea Runner

3.x verwendet das Cache-v2-Protokoll; die normalen `actions/cache@v4` sowie `actions/upload-artifact` und `actions/download-artifact` ab v4.4 funktionieren damit [147], [148]. Beim Umstieg von `act_runner` oder Runner 2.x zuerst die Upgrade-Hinweise lesen: Runner 3.0 änderte Cache-Verhalten, Sicherheitsfilter für Container-Optionen und die Verwaltung mehrerer Registrierungen [149].

13.10 Unterschiede zu GitHub Actions

Gitea Actions ist kompatibel, aber nicht identisch. Stand Gitea 1.27.3 und Runner 3.5 gilt laut Dokumentation:

- `jobs.<id>.environment` (Deployment-Umgebungen mit Freigaben) wird ignoriert [114]. Unterschiedliche Ziele bildet man über getrennte Variablen oder Workflows ab (Kapitel 15).
- `runs-on` akzeptiert nur einfache Labels, keine Gruppen oder komplexen Ausdrücke [114].
- *Problem Matchers* und Fehler-Annotationen im Code werden ignoriert [114].
- Der automatische `GITEA_TOKEN` darf keine Pakete in die Gitea-Registry hochladen. Dafür braucht man einen persönlichen Zugriffstoken als Secret [114].
- `concurrency` und `permissions` werden seit Gitea 1.26 unterstützt [150], `continue-on-error` seit Gitea 1.27 [151], `timeout-minutes` seit Runner 2.0 (unabhängig von der Gitea-Version) [152]. **Ältere Versionen ignorieren diese Schlüssel stillschweigend.** Wer eine ältere Gitea-Version betreibt, sollte sich nicht darauf verlassen.
- Zusätzlich kann Gitea Actions per absoluter URL aus beliebigen Git-Repositories laden und versteht `@daily` & Co. bei `schedule` [114].

Gitea 1.27 zeigt mit Runner 2.0 oder neuer außerdem Job-Zusammenfassungen aus `$GITHUB_STEP_SUMMARY`, einen eigenen Zustand *Cancelling* und korrekt aggregiertes `continue-on-error` [151]. Server und Runner werden unabhängig versioniert [128]; eine hohe Runner-Version bedeutet daher nicht, dass die Gitea-Instanz alle Syntaxmerkmale unterstützt.

13.11 Vertrauensgrenzen von Workflows

Ein Workflow ist ausführbarer Code. Wer eine Workflow-Datei, ein eingebundenes Skript oder eine nicht gepinnte Action verändern kann, kann grundsätzlich alle Rechte des Jobs nutzen. Auf einem selbst gehosteten Runner mit Docker-Socket reicht das bis zu Root-Rechten auf dem Runner-Host [153].

- Workflows aus Forks erhalten keine Secrets [154], können aber trotzdem schädlichen Code auf dem Runner ausführen [155]. Öffentliche Repositories deshalb nur auf kurzlebigen, isolierten Runnern ausführen [139] und Läufe neuer Fork-Beiträge freigabepflichtig lassen [155].
- Niemals untrusted PR-Code in einem privilegierten Deployment-Job ausführen. Build/Tests und Deployment auf getrennte Runner-Gruppen oder mindestens getrennte Workflows und Zugangsdaten aufteilen.
- Actions auf unveränderliche Commit-Hashes pinnen [139], Workflow-Dateien per Branch-Schutz absichern und ausgehenden Netzwerkzugriff sowie erlaubte Volumes minimieren.
- Die Log-Maskierung ist nur ein Auffangnetz. Auch wenn aktuelle Runner mehrere übliche Kodierungen erkennen [148], können umgeformte, aufgeteilte oder in Artefakten gespeicherte Secrets weiterhin sichtbar werden [139].

Mit `concurrency` lassen sich überholte Läufe automatisch abbrechen, etwa wenn du mehrmals kurz hintereinander auf denselben PR-Branch pushst [136]:

```
concurrency:
  group: ci-${{ gitea.ref }}
  cancel-in-progress: true
```

13.12 Das Beispielprojekt

Damit Workflows sprachunabhängig bleiben, kapselt das Beispielprojekt `demo` alle Werkzeuge in einem `Makefile`. Die Pipeline ruft nur `make`-Ziele auf. Wechselt die Sprache oder das Test-Framework, ändert sich nur das Makefile, nicht die Pipeline. Ein weiterer Vorteil: Dieselben Befehle laufen lokal am Mac exakt so wie in der CI.

Makefile

```
.PHONY: setup lint test build

setup:          # Abhängigkeiten installieren, z.B. pip install -r requirements-dev.txt / npm ci
    ./scripts/setup.sh

lint:           # statische Prüfungen, z.B. ruff check . / npm run lint
    ./scripts/lint.sh

test:           # Tests mit Coverage-Bericht nach reports/coverage.xml
    ./scripts/test.sh

build:          # Container-Image bauen
    docker build -t demo:${or ${VERSION},dev} .
```

Wie aus diesem Baustein eine vollständige Pipeline mit Lint, Tests, SonarQube, Image-Build und Deployment wird, zeigt Kapitel 15. Vorher geht es im nächsten Kapitel um die Infrastruktur dahinter: Runner, Secrets, SSH-Zugang und SonarQube.

14 Runner, Secrets, Deployment und SonarQube

Ein Workflow ist nur eine Beschreibung. Damit er läuft, braucht es einen Runner. Damit er etwas Nützliches tun kann, braucht er Zugangsdaten. Und damit das Ergebnis irgendwo ankommt, braucht es einen sicheren Weg auf den Zielservers. Dieses Kapitel baut diese Infrastruktur Schritt für Schritt auf und ergänzt zum Schluss SonarQube für die statische Codeanalyse.

14.1 Der Runner

Der Runner ist ein kleines Go-Programm, das bei Gitea nach Jobs fragt [129] und sie in Docker-Containern oder direkt auf dem Host ausführt [130]. Bis 2026 hieß er `act_runner` (Docker-Image `gitea/act_runner`), seit der Umbenennung mit Version 1.0.0 heißt er `gitea-runner` (Image `gitea/runner`) [128]. Das Buch verwendet Version 3.5.0 vom 14. September 2026; seit dem 24. September gibt es 4.0 [156]. Server und Runner werden unabhängig versioniert. Beim Wechsel einer Runner-Hauptversion sind die Upgrade-Hinweise verbindlich; 2.0, 3.0 und 4.0 brachten absichtliche Inkompatibilitäten [149], [156].

Wo ein Runner registriert wird

Ein Runner wird mit einem **Registrierungstoken** an Gitea gebunden. Wo du das Token holst, bestimmt, wem der Runner dient [131], [157]:

Ebene	Token unter	Runner nimmt Jobs von
Instanz	<i>Administration -> Actions -> Runner</i>	allen Repositories des Servers
Organisation	<i>Organisation -> Einstellungen -> Actions -> Runner</i>	allen Repositories dieser Organisation
Repository	<i>Repository -> Einstellungen -> Actions -> Runner</i>	nur diesem Repository

Für ein kleines Team ist ein Runner auf Organisationsebene meist die richtige Wahl.

Einrichtung mit Docker Compose

Der einfachste Betrieb ist ein Container, der den Docker-Daemon des Host-Rechners mitbenutzt:

`~/runner/docker-compose.yml`

```
services:
  runner:
    image: gitea/runner:3.5.0           # reproduzierbar; Updates bewusst nach Release Notes
    container_name: gitea-runner
    restart: unless-stopped
    environment:
      CONFIG_FILE: /data/config.yaml
      GITEA_INSTANCE_URL: https://gitea.example.com
      GITEA_RUNNER_REGISTRATION_TOKEN: ${RUNNER_TOKEN} # aus der Datei .env, nur beim ersten
Start nötig
      GITEA_RUNNER_NAME: build-01
      GITEA_RUNNER_LABELS: ubuntu-latest:docker://docker.gitea.com/runner-images:ubuntu-latest
    volumes:
      - ./data:/data                  # .runner (Registrierung) und
config.yaml
      - /var/run/docker.sock:/var/run/docker.sock # Jobs laufen auf dem Docker des Hosts
```

```
cd ~/runner
mkdir -p data
docker run --rm --entrypoint="" gitea/runner:3.5.0 gitea-runner config generate > data/
```

```
config.yaml
# ältere Versionen: ... gitea/act_runner:latest act_runner generate-config > data/config.yaml
echo "RUNNER_TOKEN=<Token aus Gitea>" > .env
docker compose up -d
docker compose logs -f          # auf "Runner registered successfully" bzw. "declared
successfully" achten
```

In der von `gitea-runner` erzeugten Datei sind alle Optionen auskommentiert, es gelten also die Standardwerte [158]. Was du ändern willst, kommentierst du ein. Nach dem ersten Start liegt die Registrierung in `data/.runner` [157]; behandle diese Datei wie ein Geheimnis und sichere sie nicht öffentlich. Entferne danach `GITEA_RUNNER_REGISTRATION_TOKEN` aus Compose und `.env`, denn das Registrierungstoken wird nicht mehr gebraucht [157]; ein Job mit Zugriff auf den Docker-Socket könnte es sonst per `docker inspect` aus dem Runner-Container lesen [159]. In Gitea erscheint der Runner in der Runner-Liste mit Status *Idle*.

Labels: Wie Jobs zum Runner finden

Ein Label hat die Form `name:schema:argument` [130].
`ubuntu-latest:docker://docker.gitea.com/runner-images:ubuntu-latest` bedeutet: Ein Job mit `runs-on: ubuntu-latest` läuft in einem Container aus dem Image `docker.gitea.com/runner-images:ubuntu-latest` [130]. Diese offiziellen Images enthalten die üblichen Werkzeuge (Git, Node.js für Actions, Docker-CLI, Build-Werkzeuge). Mit dem Schema `host` (etwa `macos:host`) laufen Jobs direkt auf dem Rechner des Runners, ohne Container [130]. So lassen sich zum Beispiel Swift- oder Xcode-Builds auf einem Mac ausführen.

Eigene Labels sind jederzeit möglich, etwa `build:docker://python:3.13` für Jobs mit `runs-on: build`. Ein Job läuft nur auf einem Runner, der alle Labels aus `runs-on` besitzt [142]. Findet ein Job keinen Runner mit passendem Label, bleibt er dauerhaft im Zustand *Wartend*. Das ist der häufigste Grund für „die Pipeline startet nicht“.

Die wichtigsten Einstellungen

`data/config.yaml` (Auszug)

```
runner:
  capacity: 2          # wie viele Jobs gleichzeitig laufen dürfen
  timeout: 1h          # Obergrenze pro Job
container:
  network: ""          # leer = eigenes Netz pro Job; bei Problemen mit Services ggf. "bridge"
  privileged: false    # Jobs nicht privilegiert starten
  valid_volumes: []    # welche Host-Pfade Workflows einbinden dürfen (leer = keine)
```

Mit `gitea-runner config generate` bekommst du die vollständige, kommentierte Liste aller Optionen [160]. Mit `capacity` solltest du bei kleinen Rechnern wie einer NAS vorsichtig sein: Zwei parallele Builds mit Docker können 4 bis 8 GB RAM belegen.

ACHTUNG

Wer den Docker-Socket in einen Container einbindet, gibt diesem Container und damit jedem Job faktisch Root-Rechte auf dem Host [161]. Ein Workflow kann über den Socket beliebige Container starten und das Host-Dateisystem einbinden. Deshalb gehört ein Runner auf einen eigenen Rechner oder in eine eigene VM, nicht auf den Server mit den produktiven Daten. Er sollte nur Workflows aus vertrauenswürdigen Repositories ausführen. Das Image existiert zusätzlich in den Varianten `latest-dind` (eigener Docker-Daemon im Container, privilegiert) und `latest-dind-rootless` (eigener Daemon ohne Root-Rechte) [159], [160]. Die rootless-Variante reduziert das Risiko, bringt aber die üblichen Einschränkungen von rootless Docker (Netzwerk, cgroups, Storage-Treiber) mit [159].

14.2 Secrets und Variablen

Passwörter, Tokens und private Schlüssel gehören nie in eine Workflow-Datei, denn die liegt im Repository und ist für alle lesbar. Stattdessen hinterlegst du sie in Gitea als **Secrets**, nicht geheime Einstellungen als **Variablen** [144], [162]:

	Secret	Variable
Zugriff im Workflow	<code>\${{ secrets.NAME }}</code>	<code>\${{ vars.NAME }}</code>
Nach dem Speichern lesbar	nein, nur überschreibbar	ja
In Logs	nach Möglichkeit durch <code>***</code> ersetzt	im Klartext
Typische Inhalte	SSH-Schlüssel, API-Tokens, Passwörter	Hostnamen, Benutzernamen, URLs, <code>known_hosts</code>

Beide lassen sich auf Ebene des Benutzers, der Organisation und des Repositories anlegen (*Einstellungen* -> *Actions* -> *Secrets* bzw. *Variablen*) [144], [162]. Gibt es denselben Namen auf mehreren Ebenen, gewinnt die spezifischere [162]. Namen dürfen nur Buchstaben, Ziffern und Unterstriche enthalten, nicht mit einer Ziffer und nicht mit `GITEA_` oder `GITHUB_` beginnen (bei Variablen zusätzlich nicht mit `CI`). Groß- und Kleinschreibung wird nicht unterschieden [144], [162].

Zusätzlich erzeugt Gitea für jeden Lauf automatisch ein kurzlebiges Token, erreichbar als `secrets.GITEA_TOKEN` (oder `secrets.GITHUB_TOKEN`). Es erlaubt dem Workflow Zugriff auf das eigene Repository über die Gitea-API [143], aber, wie in Kapitel 13 erwähnt, nicht das Hochladen von Paketen und Container-Images [114].

Regeln für den sicheren Umgang:

- **Secrets über `env`: übergeben, nicht direkt in Befehle einsetzen.** `run: tool --token ${{ secrets.X }}` macht das Token für andere Prozesse in der Prozessliste sichtbar und kann in Fehlermeldungen landen. Besser: `env: TOKEN: ${{ secrets.X }}` und im Befehl `$TOKEN` [139].
- **Nie ausgeben** oder in Dateien schreiben, die als Artefakt hochgeladen werden. Die Maskierung erkennt exakte und einige häufig kodierte Formen [163], aber keine beliebige Umformung, Aufteilung oder Verschlüsselung eines Secrets [139].
- **Pull Requests aus Forks bekommen keine Secrets.** Das ist eine Schutzmaßnahme: Sonst könnte jeder per PR einen Workflow einschleusen, der die Secrets ausliest.
- **Minimale Rechte und eigene Zugangsdaten pro Zweck.** Ein Token nur für die Registry, ein SSH-Schlüssel nur für das Deployment. Regelmäßig austauschen [139].

14.3 Deployment per SSH

Das Ziel: Nach einem erfolgreichen Build auf `main` soll der Runner auf dem Zielserver `app.example.com` die neue Version starten. Der naheliegende Weg, dem Runner einen normalen SSH-Zugang zu geben, ist gefährlich: Wer den Schlüssel aus dem Runner erbeutet, hat eine Shell auf dem Produktivserver. Deshalb wird der Schlüssel so eingeschränkt, dass er **genau einen Befehl** auslösen kann und sonst nichts.

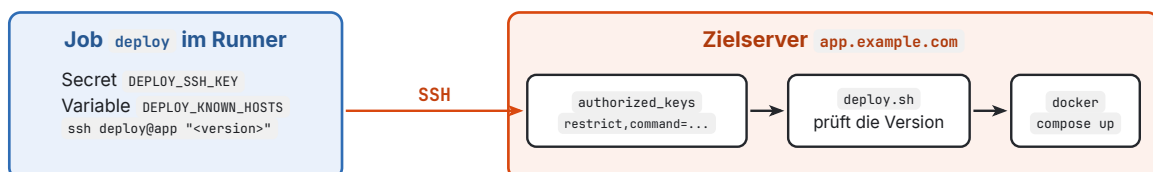


Abb. 29: Der Deploy-Schlüssel kann auf dem Server nur ein einziges, vorher festgelegtes Skript starten.

Schritt 1: Ein eigenes Schlüsselpaar für das Deployment

```
# am Mac, ohne Passphrase, weil der Runner sie nicht eingeben kann
ssh-keygen -t ed25519 -N "" -C "ci-deploy demo" -f ./deploy_demo
```

Das erzeugt `deploy_demo` (privat, kommt als Secret nach Gitea) und `deploy_demo.pub` (öffentlich, kommt auf den Server) [164]. Nach dem Einrichten löschst du beide Dateien vom Mac.

Schritt 2: Der Server lässt nur ein Skript zu

Auf dem Zielservers legst du einen eigenen Benutzer `deploy` an und trägst den öffentlichen Schlüssel mit Einschränkungen ein:

```
/home/deploy/.ssh/authorized_keys
```

```
restrict,command="/srv/demo/deploy.sh" ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAA... ci-deploy demo
```


`restrict` verbietet Port-Weiterleitungen, Agent-Weiterleitung und ein Terminal [165]. `command=` erzwingt, dass bei jeder Anmeldung mit diesem Schlüssel ausschließlich das angegebene Skript läuft, egal welchen Befehl der Client mitschickt. Den mitgeschickten Befehl findet das Skript in der Variable `SSH_ORIGINAL_COMMAND` und kann ihn als Parameter nutzen, hier für die Version [165]. Dabei muss es ihn streng prüfen:

`/srv/demo/deploy.sh`

```
#!/usr/bin/env bash
set -euo pipefail

VERSION="${SSH_ORIGINAL_COMMAND:-}"
# nur Commit-Hashes oder Versions-Tags zulassen, nichts anderes
if [[ ! "$VERSION" =~ ^([0-9a-f]{7,40}|v[0-9]+\.[0-9]+\.[0-9]+)$ ]]; then
    echo "Ungültige Version: '$VERSION'" >&2
    exit 1
fi

cd /srv/demo
exec 9>deploy.lock
flock -n 9 || { echo "Ein Deployment läuft bereits" >&2; exit 1; }

PREVIOUS="$(cat .deployed-version 2>/dev/null || true)"
export IMAGE_TAG="$VERSION"
docker compose pull
if ! docker compose up -d --remove-orphans --wait --wait-timeout 120; then
    echo "Deployment fehlgeschlagen; Rollback auf $PREVIOUS" >&2
    if [[ -n "$PREVIOUS" ]]; then
        export IMAGE_TAG="$PREVIOUS"
        docker compose up -d --remove-orphans --wait --wait-timeout 120
    fi
fi
exit 1
fi
printf '%s\n' "$VERSION" > .deployed-version.tmp
mv .deployed-version.tmp .deployed-version
docker image prune -f
echo "Deployed $VERSION"
```

```
sudo chmod 755 /srv/demo/deploy.sh
sudo usermod -aG docker deploy # siehe Hinweis unten
```

ACHTUNG

Mitglieder der Gruppe `docker` haben faktisch Root-Rechte [166]. Wegen der `command=`-Einschränkung kann der Deploy-Schlüssel diese Rechte aber nur über `deploy.sh` nutzen. Noch strenger ist eine `sudo`-Regel, die dem Benutzer `deploy` nur genau dieses Skript erlaubt, statt ihn in die Docker-Gruppe aufzunehmen. `--wait` wartet, bis die Dienste laufen bzw. gesund sind [167], und setzt für Letzteres funktionierende `healthcheck`-Einträge im Compose-Modell voraus; den Rollback-Pfad vor dem ersten echten Release absichtlich testen.

Schritt 3: Den Server-Schlüssel festnageln

Damit sich niemand als Zielsystem ausgeben kann, muss der Runner den Host-Schlüssel des Servers vorab kennen. Den ermittelst du einmal und prüfst den Fingerabdruck gegen den Server selbst, denn `ssh-keyscan` allein kann die Echtheit der Schlüssel nicht prüfen [168]:

```
ssh-keyscan -t ed25519 app.example.com # Zeile für known_hosts
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub # auf dem Server: Fingerabdruck zum Vergleich
```

Die Ausgabe von `ssh-keyscan` speicherst du als Variable `DEPLOY_KNOWN_HOSTS` in Gitea. Im Workflow wird dann mit `StrictHostKeyChecking=yes` geprüft [38]. Das häufig zu sehende `StrictHostKeyChecking=no` schaltet genau diesen Schutz ab [38].

Schritt 4: Secrets und Variablen in Gitea

Name	Art	Inhalt
DEPLOY_SSH_KEY	Secret	kompletter Inhalt von <code>deploy_demo</code> inklusive der BEGIN/END-Zeilen
DEPLOY_KNOWN_HOSTS	Variable	Ausgabe von <code>ssh-keyscan</code>
DEPLOY_HOST	Variable	<code>app.example.com</code>
DEPLOY_USER	Variable	<code>deploy</code>

Schritt 5: Der Deploy-Job

Auszug aus `.gitea/workflows/deploy.yml`

```
deploy:
  needs: [build]
  runs-on: ubuntu-latest
  steps:
    - name: Über SSH ausrollen
      env:
        SSH_KEY: ${ secrets.DEPLOY_SSH_KEY }
        KNOWN_HOSTS: ${ vars.DEPLOY_KNOWN_HOSTS }
      run: |
        install -m 700 -d "$RUNNER_TEMP/ssh"
        printf '%s\n' "$SSH_KEY" > "$RUNNER_TEMP/ssh/deploy_key"
        chmod 600 "$RUNNER_TEMP/ssh/deploy_key"
        printf '%s\n' "$KNOWN_HOSTS" > "$RUNNER_TEMP/ssh/known_hosts"
        trap 'rm -rf "$RUNNER_TEMP/ssh"' EXIT
        ssh -i "$RUNNER_TEMP/ssh/deploy_key" \
          -o UserKnownHostsFile="$RUNNER_TEMP/ssh/known_hosts" \
          -o StrictHostKeyChecking=yes \
          ${ vars.DEPLOY_USER }@${ vars.DEPLOY_HOST } "${ gitea.sha }"
```

Es gibt fertige Actions für SSH-Deployments (etwa `appleboy/ssh-action`). Sie sind bequem, bedeuten aber, dass fremder Code deinen privaten Schlüssel verarbeitet [139]. Die vier Zeilen Shell oben sind transparent und brauchen keine Abhängigkeit.

AUS DER PRAXIS

Liegt der Zielservers in einem privaten Netz oder ist er nur über WireGuard erreichbar, muss der **Runner** in diesem Netz sein, nicht Gitea. Da der Runner seine Verbindungen selbst aufbaut (Kapitel 13) [129], genügt es, den Runner-Host ins VPN zu nehmen. Gitea selbst braucht keinen Zugang zum Zielservers.

14.4 Container-Images in der Gitea-Registry

Gitea hat eine eingebaute Container-Registry. Images heißen dort `gitea.example.com/<besitzer>/<image>:<tag>`, also etwa `gitea.example.com/team/demo:3f2a9c1` [169]. Weil der automatische `GITEA_TOKEN` keine Pakete hochladen darf [114], legst du unter *Einstellungen* -> *Anwendungen* einen persönlichen Zugriffstoken mit Schreibrecht für Pakete an [169], [170] und speicherst ihn als Secret `REGISTRY_TOKEN`:

```
build:
  needs: [test]
  runs-on: ubuntu-latest
  env:
    IMAGE: gitea.example.com/team/demo
  steps:
    - uses: actions/checkout@v4
    - name: An der Registry anmelden
      env:
        TOKEN: ${ secrets.REGISTRY_TOKEN }
      run: echo "$TOKEN" | docker login gitea.example.com -u ${ gitea.actor } --password-stdin
    - name: Bauen und hochladen
```

```
run: |
  docker build -t "$IMAGE:${{ gitea.sha }}" -t "$IMAGE:latest" .
  docker push "$IMAGE:${{ gitea.sha }}"
  docker push "$IMAGE:latest"
```

Damit `docker build` im Job funktioniert, braucht der Job Zugriff auf einen Docker-Daemon. Beim Runner-Setup mit eingebundenem Socket ist das gegeben, die Runner-Images bringen die Docker-CLI mit. Auf dem Zielsystem steht in der `docker-compose.yml` dann `image: gitea.example.com/team/demo:${IMAGE_TAG}`, und der Server braucht einmalig ein `docker login` mit einem Token, das nur Leserechte für Pakete hat [170]. Ausgerollt wird immer der unveränderliche Commit- oder Versions-Tag; `latest` ist höchstens ein bequemer Hinweis und darf nie die Grundlage für reproduzierbare Deployments oder Rollbacks sein.

14.5 SonarQube: statische Codeanalyse

SonarQube untersucht den Quellcode, ohne ihn auszuführen, auf Fehlermuster, Sicherheitsprobleme (*Vulnerabilities* und *Security Hotspots*), Wartbarkeitsmängel (*Code Smells*), Duplikate und Testabdeckung [171]. Das Ergebnis wird an einem **Quality Gate** gemessen, einem Satz von Schwellwerten wie „keine neuen kritischen Probleme“ und „mindestens 80 % Abdeckung im neuen Code“ [171], [172].

MERKE

Die kostenlose *Community Build* von SonarQube analysiert nur **einen** Branch pro Projekt, den Hauptbranch. Analysen von Feature-Branche und Pull Requests samt Kommentaren im PR gibt es erst in den kostenpflichtigen Editionen, und auch dort ist Gitea keine offiziell unterstützte Plattform für PR-Kommentare [173]. Praktische Folge: In der Community Build lässt du Sonar **nur bei Pushes auf main** laufen. Eine Analyse eines Feature-Branche würde sonst die Ergebnisse von `main` überschreiben.

Server und Projekt

SonarQube läuft als Docker-Container mit einer PostgreSQL-Datenbank und braucht spürbar Ressourcen (laut Dokumentation mindestens 4 GB RAM für kleine Installationen) [174]. Weil es intern Elasticsearch verwendet, muss auf dem Host ein Kernel-Parameter gesetzt sein, sonst startet es nicht [175]:

```
sudo sysctl -w vm.max_map_count=524288 # dauerhaft in /etc/sysctl.conf eintragen
```

Nach dem Start legst du in der Weboberfläche ein Projekt manuell an (*Create Project* -> *Local project*, Projektschlüssel z.B. `demo`) und erzeugst dafür ein **Project Analysis Token**. Dieses kommt als Secret `SONAR_TOKEN` nach Gitea, die Server-URL als Variable `SONAR_HOST_URL`.

Konfiguration im Repository

sonar-project.properties

```
sonar.projectKey=demo
sonar.projectName=demo
sonar.sources=src
sonar.tests=tests
sonar.exclusions=**/migrations/**,**/*.min.js
# Coverage-Bericht aus "make test", Schlüssel je nach Sprache, z.B.:
sonar.python.coverage.reportPaths=reports/coverage.xml
# sonar.javascript.lcov.reportPaths=reports/lcov.info
```

Der Sonar-Job

```
sonar:
  if: gitea.ref == 'refs/heads/main' # Community Build: nur main analysieren
  needs: [test]
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
      with:
        fetch-depth: 0 # volle Historie für "neuer Code" und blame
```

```
- run: make setup && make test          # erzeugt den Coverage-Bericht
- name: SonarQube-Analyse
  uses: SonarSource/sonarqube-scan-action@v7
  env:
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
    SONAR_HOST_URL: ${ vars.SONAR_HOST_URL }
- name: Quality Gate prüfen
  uses: SonarSource/sonarqube-quality-gate-action@v1
  timeout-minutes: 5
  env:
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
    SONAR_HOST_URL: ${ vars.SONAR_HOST_URL }
```

Die Scan-Action lädt beim ersten Lauf den *SonarScanner* samt Java-Laufzeit herunter, der Runner braucht dafür Internetzugang [176]. Der Quality-Gate-Schritt wartet auf das Ergebnis der Auswertung und lässt den Job fehlschlagen, wenn das Gate rot ist [177]. Hängen spätere Jobs wie das Deployment per `needs` an `sonar`, wird bei rotem Gate nicht ausgeliefert. Die Versionsangaben `@v7` und `@v1` sind die hier verwendeten Hauptversionen; die Scan-Action gibt es inzwischen als `v8` (Stand 30. September 2026: 8.3.0), das die Signaturprüfung des Scanners standardmäßig aktiviert [178]. Neue Hauptversionen können Konfigurationsänderungen mitbringen, deshalb vor einem Update die Release Notes lesen.

15 Die komplette Pipeline

Zum Abschluss werden alle Bausteine zu einer vollständigen Pipeline für das Beispielprojekt zusammengesetzt. Sie besteht aus zwei Workflow-Dateien mit klar getrennten Aufgaben: `ci.yml` prüft jede Änderung, `deploy.yml` liefert aus, was in `main` angekommen ist.

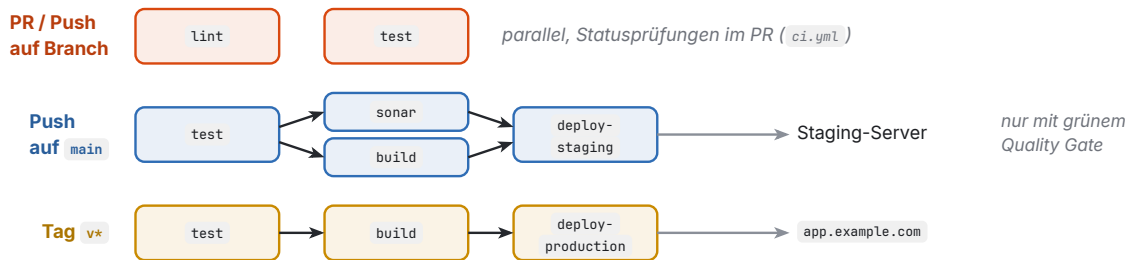


Abb. 30: Drei Auslöser, drei Abläufe. Produktion wird nur durch einen bewusst gesetzten Versions-Tag erreicht.

15.1 ci.yml : jede Änderung prüfen

`.gitea/workflows/ci.yml`

```
name: ci

on:
  pull_request:           # jeder PR und jeder weitere Push darauf
  push:
    branches-ignore: [main] # Pushes auf Feature-Branches auch ohne PR prüfen
  workflow_dispatch:

concurrency:
  group: ci-${{ gitea.ref }}
  cancel-in-progress: true

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: make setup
      - run: make lint

  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: make setup
      - run: make test
      - name: Testbericht sichern
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: testbericht
          path: reports/
          retention-days: 14
```

`lint` und `test` laufen parallel, weil keiner vom anderen abhängt. So kommt das Ergebnis schneller. In Gitea trägst du im Branch-Schutz von `main` die Statusprüfungen mit dem Muster `ci / *` als erforderlich ein (Kapitel 12) [83]. Ab dann lässt sich kein PR mehr mergen, dessen Lint oder Tests rot sind [83].

Pushes auf einen Branch mit offenem PR lösen hier beide Ereignisse aus (`push` und `pull_request`), die Prüfung läuft dann doppelt. Wer das vermeiden will, lässt den `push`-Block weg und prüft nur PRs.

15.2 `deploy.yml` : ausliefern

`.gitea/workflows/deploy.yml`

```
name: deploy

on:
  push:
    branches: [main]
    tags: ['v*']

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: make setup && make lint && make test

  sonar:
    if: gitea.ref == 'refs/heads/main'
    needs: [test]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - run: make setup && make test
      - uses: SonarSource/sonarqube-scan-action@v7
        env:
          SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
          SONAR_HOST_URL: ${ vars.SONAR_HOST_URL }
      - uses: SonarSource/sonarqube-quality-gate-action@v1
        env:
          SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
          SONAR_HOST_URL: ${ vars.SONAR_HOST_URL }

  build:
    needs: [test]
    runs-on: ubuntu-latest
    outputs:
      tag: ${ steps.meta.outputs.tag }
    env:
      IMAGE: gitea.example.com/team/demo
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - name: Release-Tag muss aus main stammen
        if: startsWith(gitea.ref, 'refs/tags/v')
        run: |
          git fetch --no-tags origin main:refs/remotes/origin/main
          git merge-base --is-ancestor "$GITHUB_SHA" refs/remotes/origin/main || {
            echo "Release-Tag zeigt nicht auf einen Commit aus main" >&2
            exit 1
          }
      - name: Image-Tag bestimmen (Versions-Tag oder Commit-Hash)
        id: meta
        run: |
```

```

    if [[ "$GITHUB_REF" == refs/tags/* ]]; then TAG="$GITHUB_REF_NAME"; else
TAG="$GITHUB_SHA"; fi
    echo "tag=$TAG" >> "$GITHUB_OUTPUT"
  - name: Anmelden, bauen, hochladen
  env:
    TOKEN: ${ secrets.REGISTRY_TOKEN }
    TAG: ${ steps.meta.outputs.tag }
  run: |
    echo "$TOKEN" | docker login gitea.example.com -u ${ gitea.actor } --password-stdin
    docker build -t "$IMAGE:$TAG" .
    docker push "$IMAGE:$TAG"

deploy-staging:
  if: gitea.ref == 'refs/heads/main'
  needs: [build, sonar] # nur mit grünem Quality Gate
  runs-on: ubuntu-latest
  steps:
  - name: Über SSH ausrollen
  env:
    SSH_KEY: ${ secrets.STAGING_SSH_KEY }
    KNOWN_HOSTS: ${ vars.STAGING_KNOWN_HOSTS }
  run: |
    install -m 700 -d "$RUNNER_TEMP/ssh"
    printf '%s\n' "$SSH_KEY" > "$RUNNER_TEMP/ssh/deploy_key"
    chmod 600 "$RUNNER_TEMP/ssh/deploy_key"
    printf '%s\n' "$KNOWN_HOSTS" > "$RUNNER_TEMP/ssh/known_hosts"
    trap 'rm -rf "$RUNNER_TEMP/ssh"' EXIT
    ssh -i "$RUNNER_TEMP/ssh/deploy_key" \
      -o UserKnownHostsFile="$RUNNER_TEMP/ssh/known_hosts" \
      -o StrictHostKeyChecking=yes \
      deploy@${ vars.STAGING_HOST } "${ needs.build.outputs.tag }"

deploy-production:
  if: startsWith(gitea.ref, 'refs/tags/v')
  needs: [build]
  runs-on: ubuntu-latest
  steps:
  - name: Über SSH ausrollen
  env:
    SSH_KEY: ${ secrets.PROD_SSH_KEY }
    KNOWN_HOSTS: ${ vars.PROD_KNOWN_HOSTS }
  run: |
    install -m 700 -d "$RUNNER_TEMP/ssh"
    printf '%s\n' "$SSH_KEY" > "$RUNNER_TEMP/ssh/deploy_key"
    chmod 600 "$RUNNER_TEMP/ssh/deploy_key"
    printf '%s\n' "$KNOWN_HOSTS" > "$RUNNER_TEMP/ssh/known_hosts"
    trap 'rm -rf "$RUNNER_TEMP/ssh"' EXIT
    ssh -i "$RUNNER_TEMP/ssh/deploy_key" \
      -o UserKnownHostsFile="$RUNNER_TEMP/ssh/known_hosts" \
      -o StrictHostKeyChecking=yes \
      deploy@${ vars.PROD_HOST } "${ needs.build.outputs.tag }"

```

Einige Entscheidungen in dieser Datei sind bewusst getroffen:

- **Staging und Produktion haben getrennte Schlüssel und Variablen.** Weil Gitea `environment` ignoriert [114], übernehmen die Namenspräfixe `STAGING_` und `PROD_` diese Trennung. Ein erbeuteter Staging-Schlüssel öffnet nicht die Produktion.
- **Produktion nur über Tags.** Ein Merge nach `main` landet automatisch auf Staging. Für die Produktion setzt du bewusst einen Versions-Tag. Das Image wird dabei unter dem Versionsnamen gebaut, sodass `deploy.sh` auf dem Server genau diese Version startet.
- **Ein Release-Tag muss aus `main` stammen.** Der Build holt `origin/main` [28] und prüft mit `git merge-base --is-ancestor`, dass der getaggte Commit darin enthalten ist [74]. Zusammen mit einem

geschützten Tag-Muster `v*` [106] verhindert das, dass ein beliebiger Commit den PR- und Review-Weg umgeht.

- **Sonar nur auf `main`**, passend zur Einschränkung der Community Build aus Kapitel 14 [173]. Auf Tags wird nicht erneut analysiert, der getaggte Commit hat die Analyse schon auf `main` durchlaufen.
- **Jeder Job checkt selbst aus.** Jobs teilen keine Dateien. Das Image ist das Artefakt, das vom Build zum Deployment wandert, und zwar über die Registry.

15.3 Ein Feature von Anfang bis Ende

```
# 1. Branch anlegen
git switch main && git pull
git switch -c feature/43-csv-export

# 2. Arbeiten, lokal genau das prüfen, was die CI prüft
make lint && make test
git add -p && git commit -m "CSV-Export für Berichte ergänzen"

# 3. Pushen und in Gitea den PR öffnen (Link steht in der Ausgabe)
git push
#   -> ci.yml läuft: lint + test als Statusprüfungen im PR

# 4. Review-Anmerkungen einarbeiten
git commit --fixup=HEAD && git push
#   -> nach Freigabe in Gitea: "Create squash commit", Branch wird gelöscht
#   -> deploy.yml läuft auf main: test, sonar, build, deploy-staging

# 5. Lokal aufräumen
git switch main && git pull && git branch -D feature/43-csv-export

# 6. Nach dem Test auf Staging: Release (auf aktuellem main)
git switch main && git pull --ff-only
git tag -s v1.3.0 -m "Version 1.3.0: CSV-Export"
git push origin v1.3.0
#   -> deploy.yml läuft für den Tag: test, build, deploy-production
```


15.4 Fehlersuche

Symptom	Ursache und Lösung
Workflow erscheint nicht im Tab <i>Actions</i>	Datei nicht in <code>.gitea/workflows/</code> oder falsche Endung [131]. Actions im Repository deaktiviert [131]. YAML-Fehler (Gitea zeigt ihn im Tab <i>Actions</i> an). Das Ereignis passt nicht zu den Filtern unter <code>on:</code> .
Job bleibt auf <i>Wartend</i>	Kein Runner mit passendem Label online. <code>runs-on</code> vertippt. Runner nur für ein anderes Repository oder eine andere Organisation registriert.
Checkout schlägt fehl (<i>could not resolve host, connection refused</i>)	Der Job-Container erreicht Gitea nicht unter dessen <code>ROOT_URL</code> . Häufig steht dort <code>localhost</code> , oder der Name ist nur im Heimnetz auflösbar [129], [131]. <code>ROOT_URL</code> korrigieren oder <code>container.network</code> des Runners anpassen.
Secret ist leer	Name vertippt, auf der falschen Ebene angelegt, als Variable statt als Secret gespeichert (oder umgekehrt), oder der PR stammt aus einem Fork.
<i>Permission denied (publickey)</i>	Privater Schlüssel unvollständig eingefügt (BEGIN/END-Zeilen fehlen), öffentlicher Schlüssel nicht in <code>authorized_keys</code> , falsche Rechte auf <code>~/.ssh</code> beim Zielbenutzer.
<i>Host key verification failed</i>	<code>known_hosts</code> -Variable fehlt oder ist veraltet, etwa nach einer Neuinstallation des Servers. Fingerabdruck neu prüfen und Variable aktualisieren.
<i>Cannot connect to the Docker daemon</i>	Der Runner hat keinen Zugriff auf einen Docker-Daemon, oder das Job-Image enthält keine Docker-CLI.
Sonar: <i>Not authorized</i> oder Timeout beim Quality Gate	Token falsch oder abgelaufen, <code>projectKey</code> passt nicht, oder <code>SONAR_HOST_URL</code> ist aus dem Job-Container nicht erreichbar.
Tag löst keinen Workflow aus	Tag nur lokal angelegt (Tags werden nicht automatisch gepusht [102]), oder das Muster unter <code>tags:</code> passt nicht.
<code>concurrency</code> , <code>timeout-minutes</code> wirken nicht	Gitea-Version zu alt (<code>concurrency</code> gibt es ab Gitea 1.26 [150], <code>timeout-minutes</code> ab Runner 2.0 [179]), die Schlüssel werden dort stillschweigend ignoriert.

Lokal testen, bevor gepusht wird

Jeder Fehlversuch in der CI kostet einen Commit und ein paar Minuten. Drei Hilfen verkürzen die Schleife:

- Die Prüfungen über das Makefile lokal ausführen (`make lint && make test`). Das fängt die meisten Fehler ab.
- Workflows lokal in Docker ausführen, ohne Gitea: mit `gitea-runner exec` (früher `act_runner exec`) [160] oder dem Werkzeug `act` (`brew install act`) [180], auf dem der Runner ursprünglich basiert [181]. Am Mac braucht beides einen kompatiblen laufenden Docker-Daemon, etwa Colima, Docker Desktop oder OrbStack; Details liefert `--help`. Das ist ein schneller Test, aber keine vollständige Simulation von Giteas Ereignisdaten, Berechtigungen und Netzwerk.
- Neue Workflows zuerst nur mit `workflow_dispatch` anlegen [142] und über die Schaltfläche im Tab *Actions* starten, bis sie funktionieren. Erst dann die echten Auslöser eintragen.

16 Dependency-Bots mit Renovate

Abhängigkeiten veralten auch dann, wenn sich der eigene Code nicht ändert. Bibliotheken, Container-Images, Actions und Tool-Versionen bekommen Fehlerkorrekturen und Sicherheitsupdates. Ein *Dependency-Bot* sucht diese Änderungen, aktualisiert Versionsangaben und Lockfiles und eröffnet normale Pull Requests [182]. Er ersetzt weder Tests noch Reviews: Er sorgt dafür, dass Updates klein, sichtbar und regelmäßig eintreffen.

Dieses Kapitel verwendet Renovate als Beispiel. Renovate unterstützt Gitea direkt [183], kann Repositories automatisch finden [184] und versteht viele Paketmanager sowie Docker-, Compose-, Terraform- und Workflow-Dateien [185], [186].

16.1 Der sichere Ablauf

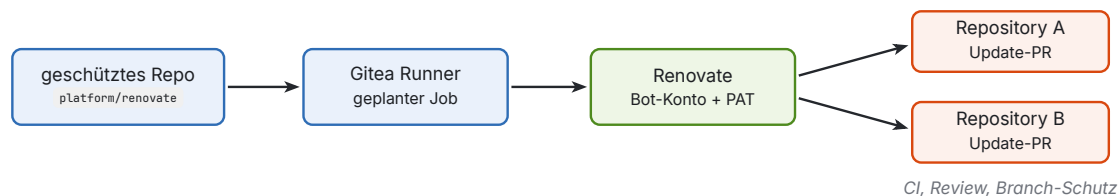


Abb. 31: Renovate arbeitet als eigener Dienstbenutzer. Seine Änderungen nehmen denselben PR- und CI-Weg wie menschliche Änderungen.

Die wichtigste Trennung lautet: **Der Runner führt Renovate aus; das Bot-Konto ändert Repositories.** Das Registrierungstoken des Runners, das automatische `GITEA_TOKEN` eines Jobs und der persönliche Zugriffstoken (PAT) des Bots sind drei verschiedene Zugangsdaten:

Zugangsdaten	Zweck	Reichweite
Runner-Registrierungstoken	einen Runner bei Gitea registrieren	Instanz, Organisation oder ein Repository; nach Registrierung nicht mehr im laufenden Job nötig [157]
<code>GITEA_TOKEN</code>	API- und Git-Zugriff des aktuellen Jobs	standardmäßig das Repository des Jobs; Rechte über <code>permissions:</code> begrenzen [143]
Renovate-PAT	Branches, Issues und Pull Requests des Bot-Kontos	alle Repositories, auf die das Bot-Konto tatsächlich Zugriff hat

ACHTUNG

Ein organisationsweiter Renovate-PAT gehört nicht als Organisations-Secret in alle Projekt-Repositories. Ein veränderbarer Projekt-Workflow könnte ihn sonst verwenden. Das Secret liegt ausschließlich in einem kleinen, besonders geschützten Automatisierungs-Repository, auf das nur die Plattformverantwortlichen Schreibzugriff haben.

16.2 Ein Bot-Konto statt eines persönlichen Kontos

Lege in Gitea einen eigenen Benutzer `renovate-bot` mit Name und E-Mail-Adresse an. Gib diesem Konto nur Zugriff auf die Organisationen und Repositories, die es pflegen soll. Für den PAT nennt Renovate bei aktuellem Gitea folgende Rechte [183]:

- `repo`: Lesen und Schreiben
- `user`: Lesen

- `issue`: Lesen und Schreiben
- `organization`: Lesen, falls Organisationslabels oder Teams gelesen werden
- zusätzlich `read:packages`, wenn Pakete aus der Gitea-Registry ausgewertet werden

Der PAT kommt als `RENOVATE_TOKEN` [183] in ein Repository-Secret des geschützten Automatisierungs-Repositories. Er steht weder in `config.js` noch in `renovate.json`, `.env`, Compose-Dateien oder Shell-History. Für Changelogs und Release Notes von github.com kann optional ein eigener, nur lesender `RENOVATE_GITHUB_COM_TOKEN` nötig sein [187]; auch dieser Token bekommt ein eigenes Secret.

AUS DER PRAXIS

Für mehrere Sicherheitszonen sind mehrere Bot-Konten besser als ein allmächtiges Konto: etwa `renovate-internal` für interne Dienste und `renovate-public` für öffentliche Projekte. Ein kompromittierter Token erreicht dann nicht automatisch alle Repositories.

16.3 Zentraler Lauf auf einem Runner

Renovate kann als CronJob, geplanter Container oder Gitea-Workflow laufen [187]. Für eine kleine Gitea-Installation ist ein geplanter Workflow in einem geschützten Repository gut nachvollziehbar: Konfiguration, Laufprotokoll und verwendete Image-Version sind versioniert. Der Job läuft auf einem vertrauenswürdigen Organisations-Runner, nicht auf einem öffentlichen PR-Runner [181].

`platform/renovate/.gitea/workflows/renovate.yml`

```
name: renovate

on:
  schedule:
    - cron: '17 */4 * * *'          # UTC: alle vier Stunden
  workflow_dispatch:

permissions:
  contents: read                  # betrifft nur das automatische GITEA_TOKEN

jobs:
  renovate:
    runs-on: trusted-automation
    timeout-minutes: 60
    container: renovate/renovate:44.115.2
    env:
      RENOVATE_CONFIG_FILE: ${GITEA_WORKSPACE}/config.js
      RENOVATE_TOKEN: ${GITEA_SECRETS.RENOVATE_TOKEN}
      RENOVATE_GITHUB_COM_TOKEN: ${GITEA_SECRETS.RENOVATE_GITHUB_COM_TOKEN}
      LOG_LEVEL: info
    steps:
      - uses: actions/checkout@v4
      - name: Konfiguration prüfen
        run: renovate-config-validator config.js
      - name: Repositories aktualisieren
        run: renovate
```

Das Beispiel pinnt eine lesbare Renovate-Version. Produktiv wird das Image zusätzlich auf einen geprüften Digest festgelegt; anschließend darf Renovate genau diesen Pin über einen normalen PR aktualisieren [188]. `schedule` bestimmt, wann der Backend-Prozess startet [189]. Zeitfenster in einer Repository-Konfiguration können Updates weiter einschränken, aber keinen häufigeren Lauf erzwingen [187], [190].

MERKE

Ein dauerhafter Cron-Container ist genauso möglich und entkoppelt Renovate vollständig von Actions. Der Sicherheitsgrundsatz bleibt gleich: eigener Dienstbenutzer, minimaler PAT, feste Image-Version, eingeschränkter Netzwerkzugriff und eine Konfiguration, die Projekt-Repositories nicht überschreiben können.

16.4 Die globale Konfiguration

Selbst gehostete Optionen gehören in die vom Plattformteam kontrollierte `config.js` [184]. Projekt-Repositories bekommen nur die dort ausdrücklich erlaubte Repository-Konfiguration.

platform/renovate/config.js

```
module.exports = {
  platform: 'gitea',
  endpoint: 'https://gitea.example.com/api/v1',
  autodiscover: true,
  autodiscoverTopics: ['managed-by-renovate'],
  onboarding: true,
  onboardingConfig: {
    extends: ['config:recommended'],
  },
  dependencyDashboard: true,
  prHourlyLimit: 4,
  prConcurrentLimit: 10,
  branchConcurrentLimit: 10,
  useCloudMetadataServices: false,
};
```

Nur Repositories mit dem Topic `managed-by-renovate` werden gefunden [184]. Das ist sicherer und übersichtlicher als ein ungefiltertes `autodiscover` über alles, was das Bot-Konto sehen kann [184]. Alternativ begrenzen `autodiscoverNamespaces` oder `autodiscoverFilter` die Menge [184]. Mirror-Repositories, Repositories ohne Push-/Pull-Recht und Repositories ohne Pull Requests überspringt die Gitea-Integration [183].

`useCloudMetadataServices: false` verhindert unnötige Versuche, Cloud-Metadatendienste zu verwenden [184]. Zusätzlich sollte der Runner nur die wirklich nötigen Ziele erreichen: Gitea, die verwendeten Paketregistries und gegebenenfalls github.com für Changelogs. Interne Registries erhalten eng passende `hostRules` [184]; Anmeldedaten kommen dabei aus Umgebungsvariablen, nicht aus dem Repository.

16.5 Konfiguration pro Repository

Die erste Ausführung eröffnet normalerweise einen Onboarding-PR mit `renovate.json`. Erst nach dessen Merge beginnt Renovate mit regulären Update-PRs [191]. Eine vernünftige Ausgangskonfiguration hält die Zahl paralleler PRs klein und behandelt Hauptversionen sichtbar anders:

renovate.json

```
{
  "$schema": "https://docs.renovatebot.com/renovate-schema.json",
  "extends": ["config:recommended"],
  "timezone": "Europe/Vienna",
  "schedule": ["* 20-23,0-5 * * 1-5"],
  "dependencyDashboard": true,
  "labels": ["dependencies"],
  "packageRules": [
    {
      "matchUpdateTypes": ["patch", "minor"],
      "groupName": "regelmäßige Abhängigkeitsupdates"
    },
    {
      "matchUpdateTypes": ["major"],
      "addLabels": ["breaking-change"],
      "minimumReleaseAge": "7 days"
    },
    {
      "matchManagers": ["dockerfile", "docker-compose", "github-actions"],
      "pinDigests": true
    }
  ]
}
```

Die Zeitangabe ist ein erlaubtes Renovate-Zeitfenster, kein exakter Starttermin [190]. Lockfiles werden im PR mitaktualisiert; dafür muss das Renovate-Image die passenden Werkzeuge enthalten oder sie kontrolliert bereitstellen können [184], [187]. Nach Änderungen an `renovate.json` läuft `renovate-config-validator` als normale CI-Prüfung [192].

Automerge nur als bewusste Ausnahme

Automatisches Mergen ist erst sinnvoll, wenn Branch-Schutz, erforderliche Statusprüfungen und reproduzierbare Tests stabil sind [190]. Ein möglicher enger Einstieg sind Patch-Updates reiner Entwicklungsabhängigkeiten:

```
{
  "matchDepTypes": ["devDependencies"],
  "matchUpdateTypes": ["patch"],
  "automerge": true,
  "platformAutomerge": true
}
```

Auch dann muss der PR alle erforderlichen Checks bestehen. Hauptversionen, Runtime-Abhängigkeiten, Basis-Images, Datenbanken, Compiler, Actions und Sicherheitswerkzeuge bleiben Review-pflichtig. Wenn ein Paket kompromittiert wird, beschleunigt blindes Automerge sonst gerade den Angriff.

16.6 Betrieb und Fehlersuche

Symptom	Prüfung
Repository wird nicht gefunden	Topic, Pull-Request-Einheit und Rechte des Bot-Kontos prüfen [183]; bei Filtern den vollständigen Pfad <code>organisation/repo</code> verwenden.
Onboarding-PR kommt immer wieder	Onboarding-Branch oder Titel wurde geändert beziehungsweise der alte PR geschlossen [191]; Verlauf und globale Konfiguration prüfen.
Lockfile wird nicht aktualisiert	Benötigter Paketmanager fehlt im Image oder darf nicht heruntergeladen werden; vollständiges Renovate-Image beziehungsweise kontrolliertes Werkzeug-Image verwenden [184], [187].
Zu viele PRs	<code>prHourlyLimit</code> , <code>prConcurrentLimit</code> , Gruppierung und Zeitfenster enger setzen [190]; alte Dashboard-Einträge aufräumen.
Private Registry liefert 401/403	<code>hostRules</code> , Hostname einschließlich Port, Token-Reichweite und Netzwerkzugriff des Runners prüfen.
PR ist grün, aber riskant	Automerge stoppen; Changelog, Transitivabhängigkeiten, Lizenz, Migrationshinweise und reale Integrationstests prüfen.

Für den Alltag genügen vier Kontrollen: Der Bot lief zuletzt erfolgreich, sein PAT läuft nicht unbemerkt ab, offene Sicherheitsupdates altern nicht, und Renovate selbst wird regelmäßig aktualisiert. Die Dependency-Dashboard-Issue dient dabei als Arbeitsliste, nicht als Ersatz für Monitoring [193].

17 CI-Gates, Secrets und Repository-Struktur

Eine gute Pipeline sammelt nicht möglichst viele Werkzeuge. Sie beantwortet für jede Änderung wenige klare Fragen: Ist der Code korrekt? Ist er wartbar? Enthält er bekannte Schwachstellen oder Geheimnisse? Ist das gebaute Artefakt das geprüfte Artefakt? Und darf genau dieser Stand in genau diese Umgebung ausgerollt werden?

17.1 Vier Stufen statt eines einzigen großen Jobs



schnell links, privilegiert und selten rechts

Abb. 32: Je weiter rechts ein Job steht, desto stärker sind seine Rechte und desto strenger muss seine Ausführung begrenzt sein.

Die Stufen dürfen auf mehrere Workflow-Dateien verteilt sein. Dadurch bleiben Statusprüfungen verständlich und ein PR wartet nicht auf einen Deployment-Job, der dort ohnehin nichts zu tun hat.

Gate	Typische Prüfungen	Werkzeuge	Wann
Check	Format, Lint, Typen, Unit- und Integrationstests, Buildbarkeit	projektnahe Befehle über <code>make check</code> und <code>make test</code>	jeder PR
Quality	Duplikate, Komplexität, Coverage, Bugs, Wartbarkeit	SonarQube oder sprachspezifische Linter	<code>main</code> , bei passender Edition zusätzlich PR [173]
Security	SAST, Abhängigkeiten, Secrets, IaC und Container-Image	Semgrep und Trivy [194], [195]	PR sowie gebautes Image
Build	reproduzierbares Artefakt, SBOM, unveränderlicher Digest	Docker/BuildKit [196], Trivy, Registry	nach grünen Quellcode-Checks
Deploy	Version, Ziel, Migration, Healthcheck, Smoke-Test, Rollback	eingeschränkter SSH-/API-Zugang	<code>main</code> nach Staging, geschützter Tag nach Produktion

17.2 Welche Checks wirklich sinnvoll sind

Check: schnell, deterministisch und lokal ausführbar

Der wichtigste Vertrag ist kein CI-Anbieter, sondern das Repository selbst:

```

.PHONY: check test
check:
    ./scripts/format-check
    ./scripts/lint
    ./scripts/typecheck

test:
    ./scripts/unit-test
    ./scripts/integration-test
  
```

CI führt `make check` und `make test` aus; Entwickler führen dieselben Befehle lokal aus. Formatter, Linter und Compiler werden über Lockfile, Tool-Datei oder ein gepinntes CI-Image versioniert. Ein flüchtiger Fehler wird nicht durch drei automatische Wiederholungen versteckt: Flaky Tests werden repariert oder ausdrücklich isoliert.

SonarQube: übergreifende Codequalität

SonarQube eignet sich für langfristige Trends, Duplikate, Coverage und ein zentrales Quality Gate. Die kostenlose Community Build analysiert den Hauptbranch; Branch- und Pull-Request-Analysen gehören zu anderen SonarQube-Varianten [173], [197]. Deshalb läuft die Community Build typischerweise nach dem Merge auf `main`. Das Quality Gate blockiert dort den Build oder das Staging-Deployment.

Ein Quality Gate sollte vor allem **neuen Code** schützen [198]: keine neuen Blocker-/Critical-Befunde, ausreichende Coverage für neuen Code und keine unreviewten Security Hotspots. Starre Gesamt-Coverage auf einem Altprojekt erzeugt dagegen oft Arbeit ohne Risikogewinn.

Semgrep: SAST mit verständlichen Regeln

Semgrep sucht Sprachmuster und Sicherheitsfehler direkt im Quellcode. Die Community-Variante kann ohne Cloudkonto laufen [194]:

```
semgrep scan --config auto --error .
```

`auto` lädt passende Regeln aus der Semgrep Registry [199]. Für reproduzierbare, vertrauliche oder offline betriebene Pipelines werden freigegebene Regeln in einem internen Repository versioniert und mit `--config .semgrep/` verwendet [199]. Zunächst läuft Semgrep sichtbar, aber nicht blockierend; nach Bereinigung und Baseline werden neue High-Confidence-Befunde zum Gate. Jede Ausnahme braucht Regel-ID, Begründung, Verantwortlichen und Ablaufdatum.

Trivy: Abhängigkeiten, IaC, Secrets und Images

Trivy deckt mehrere andere Risikoflächen ab. Vor dem Build prüft es Repository, Lockfiles und Infrastrukturcode [195]:

```
trivy fs \
  --scanners vuln,misconfig,secret \
  --severity HIGH,CRITICAL \
  --ignore-unfixed \
  --exit-code 1 .
```

Nach dem Build wird das echte Container-Image geprüft [200]:

```
trivy image \
  --scanners vuln,misconfig,secret \
  --severity HIGH,CRITICAL \
  --ignore-unfixed \
  --exit-code 1 "$IMAGE:$GITHUB_SHA"

trivy image --format cyclonedx --output sbom.cdx.json "$IMAGE:$GITHUB_SHA"
```

Die beiden Läufe sind nicht redundant: Der Dateisystemscan sieht Lockfiles und IaC [195], der Image-Scan zusätzlich Betriebssystempakete und den tatsächlichen Inhalt aller Schichten [200]. `--ignore-unfixed` ist eine bewusste Policy-Entscheidung [201]; bei besonders kritischen Produkten werden auch nicht behobene Critical-Befunde sichtbar gehalten, selbst wenn sie den Build noch nicht blockieren.

MERKE

SonarQube, Semgrep und Trivy überschneiden sich teilweise, haben aber verschiedene Schwerpunkte: SonarQube bewertet Codequalität und Trends, Semgrep findet quelltextnahe Sicherheitsmuster, Trivy prüft bekannte Schwachstellen, Konfiguration, Secrets und Images. Zwei Werkzeuge mit demselben Zweck erhöhen ohne abgestimmte Zuständigkeit nur das Rauschen.

17.3 Referenzworkflow für Pull Requests

Der PR-Workflow bekommt keine Deployment-Secrets und nur Leserechte. Die drei Jobs laufen parallel und liefern stabile Namen für den Branch-Schutz:

`.gitea/workflows/check.yml`

```
name: check

on:
  pull_request:
  push:
    branches-ignore: [main]

permissions:
  contents: read

jobs:
  code:
    runs-on: untrusted-checks
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v4
      - run: make setup
      - run: make check

  test:
    runs-on: untrusted-checks
    timeout-minutes: 25
    steps:
      - uses: actions/checkout@v4
      - run: make setup
      - run: make test

  security-source:
    runs-on: untrusted-checks
    timeout-minutes: 20
    steps:
      - uses: actions/checkout@v4
      - run: semgrep scan --config .semgrep/ --error .
      - run: >-
          trivy fs --scanners vuln,misconfig,secret
          --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 .
```

Das Label `untrusted-checks` gehört zu einem Runner-Pool ohne Produktionsnetz, ohne Organisations-Secrets und möglichst ohne Host-Docker-Socket [161]. Fremder PR-Code ist ausführbarer Fremdcode. Besonders bei öffentlichen Repositories darf ein PR nicht automatisch auf einem privilegierten, dauerhaften Runner starten [139]. Gitea 1.27.3 schließt eine konkrete Umgehung der Freigabe für erstmalige Fork-Beiträge [115]; ältere 1.27-Versionen gehören deshalb nicht mehr in eine solche Umgebung.

Actions und Scanner werden in echten Workflows nicht nur mit einem Hauptversions-Tag, sondern mit vollständigem Commit- beziehungsweise Image-Digest gepinnt [139], [202]. Die lesbaren Versionen im Beispiel machen nur den Aufbau deutlich. Renovate aktualisiert diese Pins über Review-PRs [186], [188].

Das Runner-Image hinter `untrusted-checks` stellt die im Beispiel verwendeten, versionierten Werkzeuge bereit. Alternativ bekommt jeder Scanner einen eigenen Job mit einem auf Digest gepinnten Job-Container. Werkzeuge während jedes Laufs ungeprüft mit `curl | sh` nachzuladen, ist weder reproduzierbar noch eine sinnvolle Lieferkettenkontrolle.

17.4 Build und Deployment: nur das geprüfte Artefakt

Ein sauberer Build erzeugt das Image genau einmal. Nach dem Scan wird es in die Registry übertragen, sein Digest festgehalten und genau dieser Digest ausgerollt. `latest` ist kein Deployment-Nachweis [202].


```

IMAGE="gitea.example.com/team/demo"
TAG="$GITHUB_SHA"

docker build --pull -t "$IMAGE:$TAG" .
trivy image --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 "$IMAGE:$TAG"
trivy image --format cyclonedx --output sbom.cdx.json "$IMAGE:$TAG"
docker push "$IMAGE:$TAG"

DIGEST="$(docker inspect --format='{{index .RepoDigests 0}}' "$IMAGE:$TAG")"
printf 'image=%s\n' "$DIGEST" >> "$GITHUB_OUTPUT"

```

Das Deployment erhält `needs.build.outputs.image` und startet diesen Digest. Danach folgen Datenbankmigrationen mit eigener Rückrollstrategie, Healthcheck und ein kleiner Smoke-Test. Erst wenn alles grün ist, wird der neue Stand als aktiv markiert. Kapitel 14 zeigt den eingeschränkten SSH-Zugang und Kapitel 15 den vollständigen Ablauf für Staging und Produktion.

17.5 Secrets, Variablen und Job-Tokens

Gitea speichert Actions-Secrets und Variablen auf Benutzer-, Organisations- und Repository-Ebene [144], [162]. Bei gleichem Namen gewinnt die spezifischere Ebene; ein Repository-Wert überschreibt also den Organisationswert [144], [162].

Mittel	Geeignet für	Nicht geeignet für
Variable (<code>vars</code>)	Hostnamen, Feature-Schalter, nicht geheime Pfade	Passwörter, Tokens, private Schlüssel
Actions-Secret	API-Token, Registry-Passwort, privater Deployment-Schlüssel	Werte, die jedes Repository der Organisation nicht wirklich braucht
<code>GITEA_TOKEN</code>	Repository-Inhalte und API-Aufrufe des aktuellen Jobs [143]	organisationsweiter Bot oder dauerhaftes Deployment-Konto
Deploy Key	maschinelles Klonen genau eines Gitea-Repositories [203]	Zugriff auf viele Repositories oder den Zielerver

Für das automatische Job-Token werden die Rechte ausdrücklich gesetzt:

```

permissions:
  contents: read
  packages: read

```

Job- und Workflow- `permissions` wirken nur auf `GITEA_TOKEN`, nicht auf selbst angelegte PATs [143]. Zugriff auf andere private Repositories ist standardmäßig verweigert; ausgewählte Leserechte können unter *Actions -> General -> Cross-Repository Access* freigegeben werden [143]. Ein PAT bleibt die Ausnahme, weil seine Laufzeit und Reichweite nicht automatisch an einen einzelnen Job gebunden sind.

Secrets werden nicht in Befehlszeilenargumente, Artefakte oder Debug-Ausgaben geschrieben. Sie gehen über Standardeingabe oder eine Datei mit Modus `0600` an das Werkzeug und werden mit `trap` entfernt [139]. Log-Maskierung hilft nur gegen versehentliche Ausgabe [139]; bösartiger Workflow-Code kann ein Secret verändern, stückeln oder an einen fremden Server senden.

ACHTUNG

Organisations-Secrets sind bequem, vergrößern aber den Explosionsradius [139]. Gemeinsame lesende Tokens dürfen organisationsweit liegen. Produktionsschlüssel, Signierschlüssel und Renovate-PATs bleiben in einem dedizierten Repository oder besser in einem externen Secret-Manager. Rotation, Ablaufdatum, Besitzer und letzter Test werden dokumentiert.

17.6 Deploy Keys richtig einordnen

Ein Gitea-Deploy Key ist ein öffentlicher SSH-Schlüssel in den Einstellungen **eines Repositories**. Er ist standardmäßig nur lesend [204] und eignet sich etwa für einen Server, der genau dieses Repository auschecken muss. Schreibzugriff wird nur aktiviert, wenn der konkrete Automat tatsächlich pushen soll.

Für Renovate ist ein Deploy Key ungeeignet: Der Bot braucht API-Zugriff, Issues und Pull Requests in mehreren Repositories und verwendet deshalb ein eigenes Konto mit PAT [183]. Auch der SSH-Schlüssel zum Zielsystem aus Kapitel 14 ist kein Gitea-Deploy-Key; er wird im `authorized_keys` des Zielsystems auf einen einzigen Deployment-Befehl beschränkt [165].

Für jeden Verbraucher wird ein eigenes Schlüsselpaar erzeugt. So lässt sich ein kompromittierter Server sperren, ohne alle Deployments zu unterbrechen. Private Schlüssel werden nie zwischen Repositories kopiert. Ein schreibender Deploy Key umgeht keine Branch-Schutzregeln absichtlich [83]; wenn der technische Weg Regeln umgehen könnte, ist ein Bot-Konto mit normalem PR-Verfahren die bessere Wahl.

17.7 Einzelne Repositories oder Organisation?

Variante	Passt gut für	Grenzen
persönliches Repository	Experiment, Kursdemo, privates Einzelprojekt	Besitz hängt an einer Person; keine Teamrollen, weniger zentrale Runner-/Secret-/Workflow-Verwaltung
Organisations-Repository	Teamprodukt, gemeinsamer Betrieb, mehrere Dienste	Organisationsweite Runner und Secrets brauchen klare Vertrauenszonen
mehrere Organisationen	getrennte Kunden, Sicherheitszonen oder Verantwortlichkeiten	mehr Verwaltungsaufwand; gemeinsame Automatisierung muss bewusst geteilt werden

Für produktive Team-Repositories ist eine Organisation normalerweise die bessere Heimat. Rechte werden über Teams statt über einzelne Collaborators vergeben [112]. Organisationseigene Runner, Variablen, Secrets und zentrale *Scoped Workflows* können mehrere Repositories konsistent versorgen [205], [206]. Scoped Workflows laufen dabei im Kontext des jeweiligen Ziel-Repositories: mit dessen Runnern, Secrets, Job-Token und Commit [206].

Eine kleine Struktur kann so aussehen:

organisation/	
app-web	Produktcode
app-api	Produktcode
platform-ci	geprüfte Scoped Workflows und Regeln
renovate	geschützter Renovate-Runner und globale Konfiguration
deploy-manifests	gewünschter Betriebsstand, besonders streng geschützt

Teams wie `developers`, `maintainers` und `platform` bekommen nur die benötigten Repository-Einheiten und Rechte. Das Plattformteam verwaltet Runner, gemeinsame Workflows und Produktionszugänge; Entwickler ändern Produktcode über PRs. Ein organisationsweiter Runner ist keine Isolation [181], [205]: Repositories mit unterschiedlichem Vertrauen bekommen getrennte Runner-Pools oder getrennte Organisationen.

17.8 Eine pragmatische Zielarchitektur

Bereich	Runner/Identität	Pflicht-Gates
Pull Requests	ephemerer oder stark isolierter <code>untrusted-checks</code> -Runner, keine Secrets	Format, Lint, Typen, Tests, Semgrep, Trivy-Dateisystemscan
<code>main</code>	vertrauenswürdiger Build-Runner, Registry-Token nur für diesen Job	erneute Tests, Sonar-Quality-Gate, Build, Image-Scan, SBOM
Staging	eingeschränkter Deployment-Schlüssel für Staging	Digest-Deployment, Migration, Healthcheck, Smoke-Test
Produktion	separater Produktionsschlüssel, geschützter Tag und Review	bereits geprüftes Image, Freigabe, Healthcheck, Rollback
Renovate	geschütztes Automatisierungs-Repository, eigenes Bot-Konto	Konfigurationsprüfung, PR statt Direkt-Push, normale CI-Gates

A Befehlsübersicht

EINRICHTEN

<code>git config --global user.name "..."</code>	Name setzen
<code>git config --list --show-origin</code>	alle Einstellungen mit Herkunft
<code>ssh-keygen -t ed25519</code>	SSH-Schlüssel erzeugen
<code>ssh -T gitea</code>	SSH-Verbindung testen
<code>git init / git clone url</code>	Repository anlegen / klonen

ALLTAG

<code>git status -sb</code>	Zustand kompakt
<code>git add -p</code>	stückweise vormerken
<code>git diff / --staged</code>	unvorgemerkt / vorgemerkt
<code>git commit -m "..."</code>	committen
<code>git log --oneline --graph --all</code>	Graph aller Branches
<code>git show <commit></code>	einen Commit ansehen
<code>git restore --staged datei</code>	Vormerkung aufheben
<code>git rm --cached datei</code>	nicht mehr versionieren

BRANCHES

<code>git switch -c name</code>	anlegen und wechseln
<code>git switch -</code>	zum vorherigen Branch
<code>git branch -vv</code>	Branches mit Upstream
<code>git branch -d / -D name</code>	löschen / erzwingen
<code>git branch -m neu</code>	aktuellen umbenennen
<code>git branch --merged</code>	bereits integrierte Branches

REMOTES

<code>git fetch --prune</code>	Server-Stand holen, aufräumen
<code>git pull (rebase)</code>	holen und eigene Commits obenauf
<code>git push -u origin name</code>	erster Push mit Upstream
<code>git push --force-with-lease</code>	umgeschriebenen Branch pushen
<code>git push origin --delete name</code>	Remote-Branch löschen
<code>git log @{u}..</code>	noch nicht gepushte Commits

MERGEN UND REBASE

<code>git merge name</code>	in aktuellen Branch mergen
<code>git merge --no-ff / --squash</code>	Merge-Commit / Squash
<code>git merge --abort</code>	Merge abbrechen
<code>git rebase main</code>	Branch auf main setzen
<code>git rebase -i main</code>	Commits umbauen
<code>git rebase --onto neu alt branch</code>	Branch umhängen
<code>git rebase --continue / --abort</code>	weiter / abbrechen
<code>git checkout --ours datei</code>	Konflikt: unsere Seite

COMMITTS ÄNDERN

<code>git commit --amend</code>	letzte Nachricht ändern
<code>git commit --amend --no-edit</code>	Datei nachreichen
<code>git rebase -i HEAD~n + reword</code>	ältere Nachricht ändern
<code>git commit --fixup=<hash></code>	Korrektur markieren
<code>git rebase -i --autosquash main</code>	Fixups einfallen
<code>git commit --amend --reset-author</code>	Author korrigieren

RÜCKGÄNGIG UND RETTUNG

<code>git restore datei</code>	Änderungen verworfen (!)
<code>git reset --soft HEAD~1</code>	Commit auflösen, Änderungen behalten
<code>git reset --hard origin/main</code>	exakt auf Server-Stand (!)
<code>git revert <hash></code>	Commit sicher umkehren
<code>git revert -m 1 <merge></code>	Merge umkehren
<code>git reflog</code>	frühere Positionen von HEAD
<code>git branch rettung <hash></code>	verlorenen Stand sichern
<code>git stash push -u -m "..."</code>	beiseitelegen
<code>git stash pop</code>	zurückholen
<code>git cherry-pick -x <hash></code>	Commit übernehmen
<code>git cherry -v ziel quelle</code>	was schon übernommen ist
<code>git clean -n / -fd</code>	Probelauf / löschen (!)

WORKTREES

<code>git worktree add ../x -b neu main</code>	Worktree mit neuem Branch
<code>git worktree add ../x branch</code>	für bestehenden Branch
<code>git worktree list</code>	alle anzeigen
<code>git worktree remove ../x</code>	entfernen
<code>git worktree prune</code>	Reste aufräumen
<code>git config --worktree ...</code>	Wert nur für diesen Worktree

TAGS UND SUCHE

<code>git tag -a v1.0.0 -m "..."</code>	annotierten Tag setzen
<code>git push origin v1.0.0</code>	Tag pushen
<code>git tag -s v1.0.0 -m "..."</code>	signierten Release-Tag setzen
<code>git log main..feature</code>	was feature zusätzlich hat
<code>git log -S "text"</code>	wann wurde Text geändert
<code>git blame -L 10,20 datei</code>	Herkunft von Zeilen
<code>git bisect start/good/bad</code>	fehlerhaften Commit suchen

GITEA UND CI

<code>git fetch origin pull/17/head:pr-17</code>	PR lokal holen
<code>.gitea/workflows/*.yaml</code>	Ort der Workflows
<code>\${{ secrets.X }} / \${{ vars.X }}</code>	Secret / Variable
<code>\$GITHUB_OUTPUT</code>	Ausgaben eines Steps
<code>needs: / if:</code>	Abhängigkeit / Bedingung
<code>git merge-base --is-ancestor A B</code>	prüfen, ob A in B enthalten ist
<code>renovate-config-validator</code>	Renovate-Konfiguration
<code>renovate.json</code>	prüfen
<code>semgrep scan --config .semgrep / --error .</code>	versionierte SAST-Regeln ausführen
<code>trivy fs --scanners vuln,misconfig,secret .</code>	Repository und IaC prüfen
<code>trivy image image@sha256:...</code>	gebautes Image prüfen

B Glossar und Quellen

Action

Wiederverwendbarer Baustein in einem Workflow-Step, eingebunden mit `uses:`, z.B. `actions/checkout`.

Artefakt

Datei, die ein Job hochlädt, damit spätere Jobs oder Menschen sie herunterladen können.

Branch

Verschiebbarer Zeiger auf einen Commit. Wandert bei jedem Commit mit.

Branch-Schutz

Gitea-Regel, die Pushes, Force-Pushes und Merges auf einen Branch an Bedingungen knüpft.

Cherry-Pick

Übernahme der Änderung eines einzelnen Commits als neuer Commit auf einem anderen Branch, typisch für Backports.

Commit

Unveränderlicher Schnappschuss des Projekts mit Verweis auf seine Eltern, Autor und Nachricht.

Detached HEAD

Zustand, in dem `HEAD` direkt auf einen Commit statt auf einen Branch zeigt.

Dependency-Bot

Dienst wie Renovate, der Versionsänderungen als normale Pull Requests vorschlägt.

Deploy Key

Öffentlicher SSH-Schlüssel, der maschinellen Zugriff auf genau ein Gitea-Repository gewährt, möglichst nur lesend.

Fast-Forward

Merge ohne neuen Commit: Der Branch-Zeiger wird nur nach vorne geschoben.

Fetch

Neue Commits vom Server holen, ohne eigene Branches zu verändern.

Fixup

Korrektur-Commit, der beim interaktiven Rebase in einen früheren Commit eingefaltet wird.

Force-with-lease

Überschreiben eines Remote-Banches, nur wenn er seit dem letzten Fetch unverändert ist.

HEAD

Zeiger auf den aktuell ausgecheckten Branch (oder Commit).

Index

Staging Area: Entwurf des nächsten Commits zwischen Arbeitsverzeichnis und Repository.

Job

Teil eines Workflows, läuft in einem eigenen Container auf einem Runner.

Label

Name, über den `runs-on` einen passenden Runner und dessen Ausführungsumgebung wählt.

Merge-Basis

Jüngster gemeinsamer Vorfahre zweier Branches.

Merge-Commit

Commit mit zwei (oder mehr) Eltern, entsteht beim Zusammenführen auseinandergegangener Branches.

PAT

Personal Access Token eines Benutzers oder Dienstkontos für Git- und API-Zugriffe; dauerhaft und deshalb eng zu begrenzen.

Pull Request

Vorschlag in Gitea, einen Branch in einen anderen zu integrieren, mit Review und CI-Prüfung.

Quality Gate

Schwellwerte in SonarQube, an denen eine Analyse als bestanden oder nicht bestanden gilt.

Rebase

Neues Abspielen von Commits auf einer anderen Basis. Erzeugt neue Commits mit neuen Hashes.

Reflog

Lokales Protokoll aller Bewegungen von `HEAD` und Branches, das zentrale Rettungswerkzeug.

Remote

Benannte Verbindung zu einem anderen Repository, meist `origin`.

Remote-Tracking-Branch

Lokale Kopie des zuletzt bekannten Server-Stands, z.B. `origin/main`.

Signatur

Kryptografischer Nachweis, dass ein Commit oder Tag mit einem bestimmten privaten Schlüssel erzeugt wurde; keine Verschlüsselung.

Runner

Programm (`gitea-runner`, früher `act_runner`), das Jobs von Gitea abholt und ausführt.

SAST

Statische Anwendungssicherheitsanalyse: sucht Sicherheitsmuster im Quellcode, ohne das Programm auszuführen.

SBOM

Maschinenlesbare Stückliste der enthaltenen Softwarebestandteile, etwa im CycloneDX-Format.

Scoped Workflow

Zentral verwalteter Gitea-Workflow, der in mehreren Ziel-Repositories mit deren Kontext läuft.

Secret

Geschützt gespeicherter Wert in Gitea, im Workflow über `secrets.NAME` erreichbar. Die Log-Maskierung ist nur ein zusätzliches Auffangnetz.

Squash

Zusammenfassen mehrerer Commits zu einem einzigen.

Stash

Stapel für vorübergehend beiseitegelegte, uncommittete Änderungen.

Step

Einzelner Schritt in einem Job: Shell-Befehl (`run`) oder Action (`uses`).

Tag

Fester, nicht wandernder Name für einen Commit, meist eine Versionsnummer.

Upstream

Remote-Branch, mit dem ein lokaler Branch standardmäßig abgeglichen wird.

Workflow

YAML-Datei in `.gitea/workflows/`, die beschreibt, wann welche Jobs laufen.

Worktree

Zusätzliches Arbeitsverzeichnis, das an dasselbe Repository angeschlossen ist.

Weiterführende Quellen

- **Pro Git** von Scott Chacon und Ben Straub, frei online und auch auf Deutsch: <https://git-scm.com/book/de/v2>. Kapitel 3 (Branches) und 10 (Git-Interna) vertiefen die Kapitel 1 und 4 dieses Handbuchs.
- **Offizielle Git-Referenz** mit allen Optionen: <https://git-scm.com/docs>, lokal über `git help <befehl>`.
- **Gitea-Dokumentation**, insbesondere *Usage* -> *Actions* mit dem Vergleich zu GitHub Actions: <https://docs.gitea.com>
- **Gitea Runner**: README und Beispielkonfigurationen unter <https://gitea.com/gitea/runner>
- **Gitea Runner 3.x**: Installation, Upgrade-Hinweise und Sicherheitsoptionen unter <https://docs.gitea.com/runner/>
- **Syntax für Workflows**: Die GitHub-Dokumentation zu *Workflow syntax* gilt mit den Einschränkungen aus Kapitel 13 auch für Gitea.
- **SonarQube**: <https://docs.sonarsource.com>, Abschnitt zur CI-Integration mit GitHub Actions.
- **Gitea Actions**: Runner, Job-Token-Rechte, Secrets und Scoped Workflows unter <https://docs.gitea.com/usage/actions/>
- **Renovate für Gitea**: Plattformintegration und Self-Hosting unter <https://docs.renovatebot.com/modules/platform/gitea/> und <https://docs.renovatebot.com/examples/self-hosting/>
- **Trivy**: Dateisystem-, IaC-, Secret- und Image-Scans unter <https://trivy.dev/docs/latest/>
- **Semgrep**: lokale Scans und CI-Konfiguration unter <https://semgrep.dev/docs/>

Literaturverzeichnis

Die Nummern verweisen auf die Quellenangaben im Text. Online-Quellen wurden am 1. Oktober 2026 abgerufen. Das Verzeichnis enthält nur Quellen, die im Text zitiert werden.

- [1] S. Chacon und B. Straub, „Git Objects“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/book/en/v2/Git-Internals-Git-Objects>
- [2] Git Project, „git Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git>
- [3] S. Chacon und B. Straub, „What is Git?“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/book/en/v2/Getting-Started-What-is-Git%3F>
- [4] Git Project, „git-gc Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-gc>
- [5] Git Project, „gitglossary Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/gitglossary>
- [6] Git Project, „git-add Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-add>
- [7] Git Project, „git-status Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-status>
- [8] S. Chacon und B. Straub, „Git Tools - Revision Selection“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/book/en/v2/Git-Tools-Revision-Selection>
- [9] S. Chacon und B. Straub, „Git Tools - Rewriting History“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/book/en/v2/Git-Tools-Rewriting-History>
- [10] Git Project, „BreakingChanges Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/BreakingChanges>
- [11] S. Chacon und B. Straub, „Git Branching - Branches in a Nutshell“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell>
- [12] Git Project, „git-branch Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-branch>
- [13] Git Project, „gitrevisions Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/gitrevisions>
- [14] S. Chacon und B. Straub, „Git References“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/book/en/v2/Git-Internals-Git-References>
- [15] Apple Inc., „Use zsh as the default shell on Mac“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://support.apple.com/en-us/102360>
- [16] Microsoft, „Installing Visual Studio Code on macOS“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://code.visualstudio.com/docs/setup/mac>
- [17] S. Chacon und B. Straub, „Git in Zsh“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/book/en/v2/Appendix-A%3A-Git-in-Other-Environments-Git-in-Zsh>
- [18] Git Project, „git-config Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-config>
- [19] Linux man-pages project, „less(1) - Linux manual page“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://man7.org/linux/man-pages/man1/less.1.html>
- [20] Git Project, „gitcli Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/gitcli>
- [21] Git Project, „git-help Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-help>
- [22] S. Chacon und B. Straub, „Installing Git“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/book/en/v2/Getting-Started-Installing-Git>
- [23] Git Project, „Install for macOS“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/install/mac>
- [24] Homebrew, „Installation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.brew.sh/Installation>
- [25] Git Project, „git-init Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-init>
- [26] Git Project, „git-pull Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-pull>

- [27] Git Project, „git-push Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-push>
- [28] Git Project, „git-fetch Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-fetch>
- [29] Git Project, „git-rebase Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-rebase>
- [30] Git Project, „git-merge Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-merge>
- [31] Git Project, „git-rerere Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-rerere>
- [32] Git Project, „diff-options Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/diff-options>
- [33] Git Project, „git-var Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-var>
- [34] Microsoft, „Command Line Interface (CLI)“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://code.visualstudio.com/docs/editor/command-line>
- [35] GNU nano, „Cheatsheet for GNU nano“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.nano-editor.org/dist/latest/cheatsheet.html>
- [36] Apple Inc., „Technical Note TN2449: OpenSSH updates in macOS 10.12.2“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://developer.apple.com/library/archive/technotes/tn2449/_index.html
- [37] GitHub Inc., „Generating a new SSH key and adding it to the ssh-agent“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent>
- [38] OpenBSD Project, „ssh_config(5)“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://man.openbsd.org/ssh_config
- [39] Gitea, „FAQ“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/help/faq/>
- [40] Git Project, „gitignore Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/gitignore>
- [41] Zsh Development Group, „User Contributions (zsh manual)“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://zsh.sourceforge.io/Doc/Release/User-Contributions.html>
- [42] Starship Contributors, „Starship“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://starship.rs/>
- [43] Apple Inc., „File system formats available in Disk Utility on Mac“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://support.apple.com/guide/disk-utility/file-system-formats-dsku19ed921c/mac>
- [44] Git Project, „git-clone Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-clone>
- [45] S. Chacon und B. Straub, „Getting a Git Repository“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/book/en/v2/Git-Basics-Getting-a-Git-Repository>
- [46] S. Chacon und B. Straub, „Recording Changes to the Repository“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/book/en/v2/Git-Basics-Recording-Changes-to-the-Repository>
- [47] S. Chacon und B. Straub, „Interactive Staging“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/book/en/v2/Git-Tools-Interactive-Staging>
- [48] S. Chacon und B. Straub, „Contributing to a Project“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/book/en/v2/Distributed-Git-Contributing-to-a-Project>
- [49] Git Project, „git-commit Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-commit>
- [50] Git Project, „git-interpret-trailers Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-interpret-trailers>
- [51] Conventional Commits, „Conventional Commits“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.conventionalcommits.org/en/v1.0.0/>
- [52] Git Project, „git-diff Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-diff>
- [53] Free Software Foundation, „Detailed Description of Unified Format“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://www.gnu.org/software/diffutils/manual/html_node/Detailed-Unified.html
- [54] Git Project, „git-log Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-log>
- [55] Git Project, „git-show Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-show>
- [56] Git Project, „git-rm Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-rm>
- [57] Git Project, „git-check-ignore Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-check-ignore>

- [58] GitHub Inc., „Removing sensitive data from a repository“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/removing-sensitive-data-from-a-repository>
- [59] Git Project, „gitattributes Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/gitattributes>
- [60] GitHub Inc., „Git Large File Storage“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-lfs.com/>
- [61] Gitea, „Git LFS setup“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/administration/git-lfs-setup>
- [62] Git LFS Authors, „git-lfs-track(1)“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/git-lfs/git-lfs/blob/main/docs/man/git-lfs-track.adoc>
- [63] Git LFS Authors, „git-lfs-migrate(1)“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/git-lfs/git-lfs/blob/main/docs/man/git-lfs-migrate.adoc>
- [64] Git Project, „git-restore Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-restore>
- [65] Git Project, „git-checkout Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-checkout>
- [66] Git Project, „git-reflog Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-reflog>
- [67] S. Chacon und B. Straub, „Git Branching - Branch Management“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/book/en/v2/Git-Branching-Branch-Management>
- [68] Git Project, „git-switch Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-switch>
- [69] Git Project, „git-check-ref-format Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-check-ref-format>
- [70] S. Chacon und B. Straub, „Git Branching - Remote Branches“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/book/en/v2/Git-Branching-Remote-Branches>
- [71] Git Project, „git-remote Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-remote>
- [72] S. Chacon und B. Straub, „Git Basics - Working with Remotes“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/book/en/v2/Git-Basics-Working-with-Remotes>
- [73] Gitea Authors, „Configuration Cheat Sheet“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/administration/config-cheat-sheet/>
- [74] Git Project, „git-merge-base Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-merge-base>
- [75] S. Chacon und B. Straub, „Git Branching - Basic Branching and Merging“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging>
- [76] Git Project, „git-revert Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-revert>
- [77] Gitea Authors, „Merge a pull request“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/api/operations/repo-merge-pull-request/>
- [78] Git Project, „git-bisect Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-bisect>
- [79] S. Chacon und B. Straub, „Git Branching - Rebasing“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/book/en/v2/Git-Branching-Rebasing>
- [80] Microsoft, „Resolve merge conflicts in VS Code“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://code.visualstudio.com/docs/sourcecontrol/merge-conflicts>
- [81] Git Project, „git-mergetool Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-mergetool>
- [82] Git Project, „mergetools/opendiff“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/git/git/blob/master/mergetools/opendiff>
- [83] Gitea Authors, „Protected branches“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/usage/access-control/protected-branches/>
- [84] Git Project, „git-notes Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-notes>
- [85] Git Project, „gitmailmap Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/gitmailmap>
- [86] Homebrew, „git-filter-repo“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://formulae.brew.sh/formula/git-filter-repo>
- [87] Git Project, „git-filter-branch Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-filter-branch>
- [88] E. Newren, „git-filter-repo: Quickly rewrite git repository history (git-filter-branch replacement)“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/newren/git-filter-repo>

- [89] Git Project, „git-reset Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-reset>
- [90] Git Project, „Revert a faulty merge“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/git/git/blob/master/Documentation/howto/revert-a-faulty-merge.adoc>
- [91] S. Chacon und B. Straub, „Git Internals - Maintenance and Data Recovery“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/book/en/v2/Git-Internals-Maintenance-and-Data-Recovery>
- [92] Git Project, „git-stash Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-stash>
- [93] Git Project, „git-cherry-pick Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-cherry-pick>
- [94] Git Project, „git-cherry Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-cherry>
- [95] Git Project, „git-clean Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-clean>
- [96] Microsoft, „Source control in VS Code“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://code.visualstudio.com/docs/sourcecontrol/overview>
- [97] Git Project, „git-worktree Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-worktree>
- [98] Git Project, „gitrepository-layout Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/gitrepository-layout>
- [99] go-gitea/gitea, „Pull Request ref not available immediatly after pushing“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/go-gitea/gitea/issues/12074>
- [100] Anthropic, „Common workflows“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://code.claude.com/docs/en/common-workflows>
- [101] Git Project, „git-tag Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-tag>
- [102] S. Chacon und B. Straub, „Git Basics - Tagging“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/book/en/v2/Git-Basics-Tagging>
- [103] Git Project, „git-describe Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-describe>
- [104] T. Preston-Werner, „Semantic Versioning 2.0.0“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://semver.org/lang/de/>
- [105] Gitea, „Signing“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/administration/signing>
- [106] Gitea, „Protected Tags“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/usage/access-control/protected-tags>
- [107] Git Project, „git-blame Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-blame>
- [108] Git Project, „git-shortlog Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-shortlog>
- [109] Git Project, „git-archive Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-archive>
- [110] Git Project, „git-count-objects Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-count-objects>
- [111] Git Project, „git-maintenance Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-maintenance>
- [112] Gitea Authors, „Permissions“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/usage/permissions>
- [113] Gitea Authors, „What is Gitea?“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/>
- [114] Gitea Authors, „Compared to GitHub Actions“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/usage/actions/comparison>
- [115] Gitea, „Gitea 1.27.3 is released“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://blog.gitea.com/release-of-1.27.3/>
- [116] GitHub Inc., „GitHub flow“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.github.com/en/get-started/using-github/github-flow>
- [117] trunkbaseddevelopment.com, „Trunk Based Development“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://trunkbaseddevelopment.com/>
- [118] Gitea Authors, „gitea/routers/private/hook_post_receive.go (main)“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://raw.githubusercontent.com/go-gitea/gitea/main/routers/private/hook_post_receive.go
- [119] Gitea Authors, „Automatically Linked References“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/usage/issues-prs/automatically-linked-references>

- [120] Gitea Authors, „Pull Request“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/usage/pull-request>
- [121] Gitea Authors, „Issue and Pull Request templates“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/usage/issue-pull-request-templates>
- [122] GitHub Inc., „Events that trigger workflows“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows>
- [123] Gitea Authors, „Add option to update pull request by rebase · Pull Request #16125 · go-gitea/gitea“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/go-gitea/gitea/pull/16125>
- [124] Gitea Authors, „Protected tags“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/usage/protected-tags>
- [125] Gitea Authors, „gitea/modules/git/ref.go (main)“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://raw.githubusercontent.com/go-gitea/gitea/main/modules/git/ref.go>
- [126] S. Chacon und B. Straub, „Git Internals - The Refspec“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/book/en/v2/Git-Internals-The-Refspec>
- [127] Gitea Authors, „Gitea Runner“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/usage/actions/act-runner>
- [128] bircni, „Gitea Runner 1.0.0 is released“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://blog.gitea.com/release-of-runner-1.0.0/>
- [129] Gitea Authors, „Design of Gitea Actions“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/usage/actions/design>
- [130] Gitea Authors, „Labels“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/runner/labels>
- [131] Gitea Authors, „Quick Start“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/usage/actions/quickstart>
- [132] O. Ben-Kiki, C. Evans, und I. döt Net, „YAML Ain't Markup Language (YAML) Version 1.2.2“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://yaml.org/spec/1.2.2/>
- [133] Red Hat, „YAML - Visual Studio Marketplace“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://marketplace.visualstudio.com/items?itemName=redhat.vscode-yaml>
- [134] Gitea Authors, „Refactor commit status for Actions jobs · Pull Request #23786 · go-gitea/gitea“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/go-gitea/gitea/pull/23786>
- [135] GitHub Inc., „actions/checkout: Action for checking out a repo“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/actions/checkout>
- [136] GitHub Inc., „Workflow syntax for GitHub Actions“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax>
- [137] GitHub Inc., „Workflow artifacts“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.github.com/en/actions/concepts/workflows-and-actions/workflow-artifacts>
- [138] GitHub Inc., „Workflow commands for GitHub Actions“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-commands>
- [139] GitHub, Inc., „Secure use reference“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.github.com/en/actions/reference/security/secure-use>
- [140] GitHub Inc., „Evaluate expressions in workflows and actions“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.github.com/en/actions/reference/workflows-and-actions/expressions>
- [141] GitHub Inc., „Contexts reference“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.github.com/en/actions/reference/workflows-and-actions/contexts>
- [142] Gitea Authors, „Frequently Asked Questions“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/usage/actions/faq>
- [143] Gitea Authors, „Actions job token permissions (GITEA_TOKEN)“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/usage/actions/token-permissions>
- [144] Gitea Authors, „Variables“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/usage/actions/actions-variables>
- [145] GitHub Inc., „Variables reference“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.github.com/en/actions/reference/workflows-and-actions/variables>
- [146] GitHub Inc., „actions/cache: Cache dependencies and build outputs in GitHub Actions“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/actions/cache>
- [147] Gitea Authors, „Caching“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/runner/cache>
- [148] Gitea Authors, „Gitea Runner 3.0.0 is released“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://blog.gitea.com/release-of-runner-3.0.0/>
- [149] Gitea Authors, „Upgrading“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/runner/upgrade>
- [150] lunny und bircni, „Gitea 1.26.0 is released“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://blog.gitea.com/release-of-1.26.0/>

- [151] Gitea Authors, „Gitea 1.27.0 is released“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://blog.gitea.com/release-of-1.27.0/>
- [152] Gitea Authors, „Gitea Runner 2.0.0 is released“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://blog.gitea.com/release-of-runner-2.0.0/>
- [153] Docker Inc., „Docker Engine security“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/security/>
- [154] Gitea Authors, „[Intentional] Secrets are not visible in forks · Issue #35145 · go-gitea/gitea“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/go-gitea/gitea/issues/35145>
- [155] Gitea Authors, „Require approval to run actions for fork pull request · Pull Request #22803 · go-gitea/gitea“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/go-gitea/gitea/pull/22803>
- [156] Gitea, „Gitea Runner v4.0.0 Release Notes“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://gitea.com/gitea/runner/releases/tag/v4.0.0>
- [157] Gitea, „Registering a runner“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/runner/registration>
- [158] Gitea, „Configuration“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/runner/configuration>
- [159] Gitea, „Install with Docker“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/runner/installation/docker>
- [160] Gitea, „gitea/runner: Official runner for Gitea“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://gitea.com/gitea/runner>
- [161] OWASP Foundation, „Docker Security Cheat Sheet“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html
- [162] Gitea, „Secrets“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/usage/actions/secrets>
- [163] Gitea, „Gitea Runner v3.0.0 Release Notes“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://gitea.com/gitea/runner/releases/tag/v3.0.0>
- [164] OpenBSD Project, „ssh-keygen(1)“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://man.openbsd.org/ssh-keygen>
- [165] OpenBSD, „sshd(8) - OpenBSD manual pages“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://man.openbsd.org/sshd.8>
- [166] Docker Inc., „Linux post-installation steps for Docker Engine“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/engine/install/linux-postinstall/>
- [167] Docker Inc., „docker compose up“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/reference/cli/docker/compose/up/>
- [168] OpenBSD Project, „ssh-keyscan(1)“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://man.openbsd.org/ssh-keyscan>
- [169] Gitea, „Container Registry“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/usage/packages/container>
- [170] Gitea, „OAuth2 Provider“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/development/oauth2-provider>
- [171] Sonar, „SonarQube glossary“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.sonarsource.com/sonarqube-community-build/glossary>
- [172] Sonar, „Introduction to quality gates“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.sonarsource.com/sonarqube-community-build/quality-standards-administration/managing-quality-gates/introduction-to-quality-gates>
- [173] R. Curlee, „SonarQube Compare Community vs Developer vs Enterprise vs Data Center“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.sonarsource.com/blog/sonarqube-compare-editions/>
- [174] Sonar, „Server host requirements“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.sonarsource.com/sonarqube-community-build/server-installation/server-host-requirements>
- [175] Sonar, „Linux“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.sonarsource.com/sonarqube-community-build/server-installation/pre-installation/linux>
- [176] SonarSource, „SonarSource/sonarqube-scan-action“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/SonarSource/sonarqube-scan-action>
- [177] SonarSource, „SonarSource/sonarqube-quality-gate-action“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/SonarSource/sonarqube-quality-gate-action>
- [178] SonarSource, „Releases: SonarSource/sonarqube-scan-action“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/SonarSource/sonarqube-scan-action/releases>
- [179] Gitea, „Gitea Runner v2.0.0 Release Notes“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://gitea.com/gitea/runner/releases/tag/v2.0.0>
- [180] nektos, „nektos/act: Run your GitHub Actions locally“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/nektos/act>

- [181] Gitea, „Overview“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/usage/actions/overview>
- [182] Renovate, „Renovate documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.renovatebot.com/>
- [183] Renovate, „Gitea“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.renovatebot.com/modules/platform/gitea/>
- [184] Renovate, „Self-Hosted configuration“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.renovatebot.com/self-hosted-configuration/>
- [185] Renovate, „Managers“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.renovatebot.com/modules/manager/>
- [186] Renovate, „Automated Dependency Updates for GitHub Actions“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.renovatebot.com/modules/manager/github-actions/>
- [187] Renovate, „Running Renovate“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.renovatebot.com/getting-started/running/>
- [188] Renovate, „Docker“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.renovatebot.com/docker/>
- [189] Gitea Authors, „models/actions/schedule_spec.go“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://raw.githubusercontent.com/go-gitea/gitea/main/models/actions/schedule_spec.go
- [190] Renovate, „Configuration Options“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.renovatebot.com/configuration-options/>
- [191] Renovate, „Installing & onboarding“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.renovatebot.com/getting-started/installing-onboarding/>
- [192] Renovate, „Config Validation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.renovatebot.com/config-validation/>
- [193] Renovate, „Dependency Dashboard“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.renovatebot.com/key-concepts/dashboard/>
- [194] Semgrep, Inc., „Local scans with Semgrep“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.semgrep.dev/getting-started/cli>
- [195] Aqua Security, „Filesystem“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://trivy.dev/latest/docs/target/filesystem/>
- [196] Docker, Inc., „docker buildx build“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/reference/cli/docker/buildx/build/>
- [197] Sonar, „SonarQube Server Download & Install“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.sonarsource.com/products/sonarqube/downloads/>
- [198] Sonar, „Quality standards and new code“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.sonarsource.com/sonarqube-community-build/user-guide/about-new-code>
- [199] Semgrep, Inc., „CLI Reference“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.semgrep.dev/cli-reference>
- [200] Aqua Security, „Container Image“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://trivy.dev/latest/docs/target/container_image/
- [201] Aqua Security, „Filtering“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://trivy.dev/latest/docs/configuration/filtering/>
- [202] Docker, Inc., „Building best practices“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/build/building/best-practices/>
- [203] Gitea, „Add a key to a repository“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/api/operations/repo-create-key/>
- [204] Gitea Authors, „templates/repo/settings/deploy_keys.tmpl“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://raw.githubusercontent.com/go-gitea/gitea/main/templates/repo/settings/deploy_keys.tmpl
- [205] Gitea, „Gitea Runner“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/usage/actions/runner>
- [206] Gitea, „Scoped Workflows“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.gitea.com/usage/actions/scoped-workflows>